**FESP — FUNDAÇÃO DE ESTUDOS SOCIAIS DO PARANÁ**

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Júlia Barbosa de Almeida — 302255

Luís Gustavo Bueno Taquete — 302347

Gabriel Weber da Rocha — 302169

Nityananda Portellada — 302093

Ana Carolina de Proença — 302284

Danilo Depetris Soares — 302217

# IA PARA TODOS — CONECTAIA

## PROJETO 4 — ENGENHARIA DE SOFTWARE, MODELAGEM, AGILIDADE E PROTÓTIPO FUNCIONAL

CURITIBA — PARANÁ

2026

Júlia Barbosa de Almeida — 302255
Luís Gustavo Bueno Taquete — 302347
Gabriel Weber da Rocha — 302169
Nityananda Portellada — 302093
Ana Carolina de Proença — 302284
Danilo Depetris Soares — 302217

**IA PARA TODOS — CONECTAIA**

# PROPOSTA DE SOLUÇÃO DE SOFTWARE DOCUMENTADA

Trabalho acadêmico organizado conforme o enunciado “PROMPT do trabalho — Projeto 4 — 2026/2”, com cinco etapas de Engenharia de Software, requisitos, modelagem, planejamento ágil, prototipação, testes e documentação. O conteúdo reaproveita dados documentados no relatório ConectaIA de IHC/UX, distinguindo evidências, proposições técnicas e atividades pendentes.

CURITIBA — PARANÁ

2026

Júlia Barbosa de Almeida — 302255
Luís Gustavo Bueno Taquete — 302347
Gabriel Weber da Rocha — 302169
Nityananda Portellada — 302093
Ana Carolina de Proença — 302284
Danilo Depetris Soares — 302217

**IA PARA TODOS — CONECTAIA**

# PROJETO 4 — ENGENHARIA DE SOFTWARE, MODELAGEM, AGILIDADE E PROTÓTIPO FUNCIONAL

Trabalho acadêmico apresentado ao Bacharelado em Sistemas de Informação da Fundação de Estudos Sociais do Paraná (FESP), no âmbito do Projeto 4 — 2026/2.

Data de aprovação: ____ / ____ / _____

Banca examinadora / docente responsável:

_____

Nome e assinatura: _____

_____

Nome e assinatura: _____

## RESUMO

Este trabalho apresenta a reformulação técnico-acadêmica do projeto IA para Todos, implementado como protótipo web ConectaIA, para atender às cinco etapas de Engenharia de Software estabelecidas no Projeto 4 — 2026/2. O problema abordado é o acesso seguro, acessível e compreensível à inteligência artificial por pessoas idosas e usuários com baixa familiaridade digital. O relatório integra fundamentação de processos de software, requisitos funcionais e não funcionais, modelagem BPMN e UML, rastreabilidade, proposta de organização Scrum e Kanban, backlog priorizado, arquitetura e critérios de qualidade. O documento de origem registra três entrevistas (P1–P3), seis tarefas de avaliação e escores SUS relatados como 100/100 por participante, totalizando 18/18 tarefas concluídas segundo a equipe. Esses resultados permanecem classificados como relatos sem os formulários originais, não sendo convertidos em evidência auditada. A inspeção prévia do código identificou React/TypeScript, integração Gemini prevista, recursos de voz dependentes do navegador e controles de acessibilidade; não comprovou execução em navegador nem validação em produção. Esta versão apresenta desenhos conceituais e planejamento proposto de forma distinta dos elementos já documentados, além de reproduzir integralmente os cinco materiais didáticos (54 páginas, Anexos A–E) e identificar a falta das respostas dos exercícios e da comprovação experimental. A versão integrada acrescenta vinte respostas-modelo aplicadas às cinco etapas e um formulário de registro de evidências, mantendo explícita a distinção entre proposta, relato e realização comprovada.


Palavras-chave: Engenharia de Software. Engenharia de Requisitos. BPMN. UML. Scrum. Kanban. Acessibilidade. Inteligência Artificial.

## ABSTRACT

This academic study presents the technical restructuring of the AI for All project, implemented as the ConectaIA web prototype, to address the five Software Engineering stages established in Project 4 — 2026/2. It addresses safe, accessible and understandable access to artificial intelligence for older adults and users with limited digital familiarity. The report combines foundations of software processes, functional and non-functional requirements, BPMN and UML modeling, traceability, proposed Scrum and Kanban organization, a prioritized backlog, architecture, and quality criteria. The source report records three interviews (P1–P3), six evaluation tasks, and SUS scores reported as 100/100 for each participant, totaling 18/18 completed tasks according to the team. Those outcomes remain categorized as unverified reports because the original forms were unavailable; they are not treated as audited evidence. A previous code inspection identified React/TypeScript, a planned Gemini integration, browser-dependent voice features and accessibility controls, but did not establish browser execution or production validation. This version distinguishes conceptual designs and proposed planning from already documented elements, reproduces all five educational support texts (54 pages, Annexes A–E), and identifies missing exercise answers and experimental verification. The integrated version also includes twenty model answers covering the five stages and an evidence-recording form, explicitly distinguishing proposals, reports, and demonstrated results.



Keywords: Software Engineering. Requirements Engineering. BPMN. UML. Scrum. Kanban. Accessibility. Artificial Intelligence.

## SUMÁRIO

1 INTRODUÇÃO.....7
2 ETAPA 1 — FUNDAMENTOS DA ENGENHARIA DE SOFTWARE.....9
3 ETAPA 2 — REQUISITOS E MODELAGEM INICIAL.....16
4 ETAPA 3 — METODOLOGIAS ÁGEIS E PLANEJAMENTO.....30
5 ETAPA 4 — DESENVOLVIMENTO COLABORATIVO E PROTÓTIPOS.....41
6 ETAPA 5 — VALIDAÇÃO, QUALIDADE E APRESENTAÇÃO FINAL.....48
7 MATRIZ DE ADERÊNCIA E PENDÊNCIAS.....57
8 CONSIDERAÇÕES FINAIS.....58
REFERÊNCIAS.....59
APÊNDICE A — EVIDÊNCIAS E IMAGENS DA VERSÃO ANTERIOR.....61
APÊNDICE B — INSTRUMENTOS DE TESTES E RASTREABILIDADE.....69
APÊNDICE C — CORRESPONDÊNCIA COM OS TEXTOS DE APOIO DIDÁTICOS.....71
APÊNDICE D — PLANO DE COMPROVAÇÕES E REGISTRO DAS ATIVIDADES.....72
ANEXOS — REPRODUÇÃO INTEGRAL DOS CINCO TEXTOS DIDÁTICOS.....74
ANEXO A — Aula 1 — Fundamentos da Engenharia de Software, Ciclos de Vida e Qualidade.75
ANEXO B — Aula 2 — Engenharia de Requisitos e Modelagem Inicial de Sistemas.....85
ANEXO C — Aula 3 — Metodologias Ágeis, Engenharia de Fluxo e Planejamento.....105
ANEXO D — Aula 4 — Desenvolvimento Colaborativo, Controle de Versão e Prototipação….124
ANEXO E — Aula 5 — Validação, Garantia da Qualidade e Apresentação Técnica.....129

# 1 INTRODUÇÃO

A Engenharia de Software procura transformar necessidades humanas, organizacionais e técnicas em produtos verificáveis, sustentáveis e de qualidade, mediante processos sistemáticos de análise, desenvolvimento, verificação e evolução. A questão norteadora do Projeto 4 é: “Como planejar, organizar e desenvolver soluções de software de forma colaborativa, utilizando fundamentos da Engenharia de Software e metodologias ágeis para atender necessidades reais de usuários e organizações?”. A presente aplicação dessa questão é o ConectaIA, projeto educacional voltado à inclusão digital no uso da inteligência artificial.

A versão original estava centrada em IHC, UX, acessibilidade e testes de usabilidade. Esses conteúdos continuam relevantes como evidências sobre pessoas e interface, mas não substituem os artefatos de Engenharia de Software exigidos no enunciado atual. Assim, esta versão registra o problema, o modelo de processo, os critérios de qualidade, os requisitos, a modelagem, a proposta de backlog e de gestão de fluxo, a organização da implementação e o estado verificável das validações, sem presumir execuções ou resultados ausentes dos arquivos.

## 1.1 Problema real e justificativa

Usuários com baixa familiaridade digital podem encontrar obstáculos para formular perguntas, identificar informações confiáveis, compreender riscos de privacidade e utilizar recursos de leitura e voz. Três entrevistas semiestruturadas relatadas pela equipe com P1, P2 e P3 identificaram dificuldades de expressão, receio de erros, dúvidas sobre dados pessoais e necessidades de legibilidade. A solução proposta oferece orientação progressiva, exemplos de perguntas, trilhas de aprendizagem, navegação clara e controles de acessibilidade. O problema é sociotécnico, porque envolve tecnologia, cognição, comportamento, autonomia e proteção de dados (ROGERS; SHARP; PREECE, 2019).

## 1.2 Objetivo geral e objetivos específicos

Objetivo geral: planejar, documentar e estruturar a evolução colaborativa de um protótipo web inclusivo para iniciação segura ao uso da inteligência artificial, aplicando processos, requisitos, modelagem, métodos ágeis e práticas de qualidade de software.

- Explicitar público, contexto de uso, partes interessadas e restrições do ConectaIA.
- Especificar requisitos funcionais e não funcionais, suas prioridades e critérios de aceitação.
- Modelar o processo de interação em BPMN e as funcionalidades e entidades em UML.
- Propor backlog priorizado, sprints, fluxo Kanban, responsabilidades e critérios de pronto.
- Documentar arquitetura, MVP, uso de versionamento e riscos de implantação.
- Separar evidências de testes efetivamente relatadas de planos e validações ainda pendentes.

### 1.3 Método, fontes e critérios de evidência

A elaboração confrontou o relatório ConectaIA previamente apresentado, o enunciado do Projeto 4 — 2026/2 e os cinco Textos de Apoio Didáticos da disciplina, recebidos integralmente (Aulas 1 a 5). Os conteúdos das aulas e os exemplos gerais são preservados em suas 54 páginas originais, reproduzidas nos Anexos A a E. Também foram integradas vinte respostas-modelo, distribuídas pelas cinco etapas, a partir das entregas indicadas no prompt; essas respostas não devem ser confundidas com um arquivo original de exercícios do professor, que não foi fornecido. O relatório diferencia documentação elaborada, relatos dos participantes e evidências de execução efetivamente disponíveis.

# 2 ETAPA 1 — FUNDAMENTOS DA ENGENHARIA DE SOFTWARE

## 2.1 Introdução à Engenharia de Software e cenário profissional

### 2.1.1 Evolução epistemológica da construção de software

O desenvolvimento de programas evoluiu de práticas predominantemente individuais e dependentes de conhecimento tácito para uma disciplina que integra requisitos, arquitetura, configuração, testes, implantação e manutenção. O termo Engenharia de Software enfatiza reprodutibilidade, gestão da complexidade, qualidade e capacidade de evolução. Além da correção do código, o sucesso depende de necessidades bem definidas, manutenção e adequação do sistema ao contexto de uso (SOMMERVILLE, 2016).

Conforme a periodização didática da Aula 1, a evolução pode ser sintetizada em quatro marcos: práticas artesanais (1950–1965), crise do software (1968–1975), modelos clássicos prescritivos (1975–1995) e paradigma ágil, contínuo e sociotécnico (desde 2001). Para o ConectaIA, a principal consequência é a substituição de decisões isoladas por requisitos rastreáveis, verificação técnica e validação com usuários (GUERRA, 2026).

### 2.1.2 Trilha de integração: horizonte sociotécnico

No ConectaIA, a solução não se resume ao chat. O público apresenta repertórios diferentes, dificuldades de leitura e preocupação com privacidade. Portanto, elicitação, arquitetura, conteúdo educativo, segurança, acessibilidade e suporte devem ser tratados como partes do sistema. A integração sociotécnica vincula decisões de interface aos requisitos de confiabilidade das respostas, transparência sobre IA e autonomia do usuário; o sucesso não é medido exclusivamente pela existência de uma funcionalidade.

### 2.1.3 O engenheiro de software no cenário contemporâneo

O profissional de Engenharia de Software articula produto, stakeholders, requisitos, código, infraestrutura, QA, segurança e operações. Em equipes multidisciplinares, o trabalho envolve negociação de escopo, decomposição de problemas, comunicação técnica, revisão por pares e identificação de riscos. Para o ConectaIA, essas atribuições podem ser distribuídas por papéis de produto,

desenvolvimento, UX/acessibilidade e qualidade, sem supor que os integrantes tenham assumido formalmente funções Scrum.

## 2.2 Modelos de processo e ciclos de vida de software

A Aula 1 distingue as atividades de comunicação, planejamento, modelagem, construção e implantação, com comparação explícita entre cascata, desenvolvimento iterativo e incremental. O ConectaIA deve utilizar evolução iterativo-incremental porque os requisitos de acessibilidade, a compreensão da IA e a estabilidade da integração externa exigem feedback e revisão. Isso constitui escolha metodológica proposta, não reconstrução comprovada da cronologia real da equipe (GUERRA, 2026).

### 2.2.1 Modelos de ciclo de vida: análise estrutural

Ciclo de vida é a sequência organizada de atividades de concepção, requisitos, projeto, implementação, testes, entrega e manutenção. O modelo em cascata privilegia fases ordenadas e controle documental; o modelo iterativo revisita decisões com aprendizado; o incremental entrega subconjuntos funcionais sucessivos. A escolha deve considerar estabilidade dos requisitos, risco, custo de mudança, disponibilidade de usuários e necessidade de feedback.

### 2.2.2 Detalhamento e matriz comparativa

| Modelo | Como funciona | Vantagens | Limitações | Adequação ao ConectaIA |
|---|---|---|---|---|
| Cascata | Etapas sequenciais com marcos de aprovação | Rastreabilidade e previsibilidade em escopo estável | Mudanças tardias custosas | Útil como referência de documentação; insuficiente isoladamente |
| Iterativo | Ciclos de revisão da solução | Incorpora feedback e reduz incertezas | Exige controle de versões e critérios de decisão | Adequado para testar legibilidade, entendimento e navegação |
| Incremental | Entregas funcionais por fatias de valor | Antecipação de valor e testes por incremento | Dependências e integração precisam de gestão | Adequado para liberar Início, trilhas, conversa e recursos de apoio gradualmente |
| Ágil híbrido | Iterações com disciplina documental e fluxo visível | Concilia feedback e rastreabilidade | Exige acordos e comunicação contínua | Proposta recomendada, sujeita à adoção real da equipe |

Para o projeto acadêmico, recomenda-se um processo iterativo-incremental com backlog visível, critérios de aceite por funcionalidade, rastreabilidade entre

requisitos e testes e revisão contínua dos riscos. Trata-se de recomendação metodológica, não declaração de que as etapas já ocorreram na ordem descrita.

## 2.3 Processos de desenvolvimento, verificação, validação e qualidade

### 2.3.1 Oito características da ISO/IEC 25010:2011 e subcaracterísticas

O enunciado pede detalhamento das oito características tradicionais do modelo de qualidade de produto da ISO/IEC 25010:2011. A edição de 2011 foi substituída por revisão de 2023, cujo modelo passou a nove características. Para respeitar a estrutura didática de oito itens, a matriz abaixo utiliza explicitamente o modelo de 2011, sem atribuí-lo incorretamente à edição vigente (ISO/IEC, 2011; ISO/IEC, 2023).

| Característica (2011) | Subcaracterísticas resumidas | Aplicação e avaliação no ConectaIA |
|---|---|---|
| Adequação funcional | Completude, correção e pertinência funcional | RF01–RF10; verificar navegação e resultados esperados |
| Eficiência de desempenho | Tempo de resposta, uso de recursos, capacidade | Medir tempo de carregamento e resposta de recursos locais/API |
| Compatibilidade | Coexistência, interoperabilidade | Testar navegadores, interfaces de voz e integração externa |
| Usabilidade | Reconhecimento, aprendizado, operabilidade, prevenção de erro, estética, acessibilidade | Tarefas T1–T6, teclado, contraste, feedback e leitura |
| Confiabilidade | Maturidade, disponibilidade, tolerância a falhas, recuperabilidade | Validar falha de rede/Gemini e resposta de contingência |
| Segurança | Confidencialidade, integridade, não repúdio, responsabilização, autenticidade | Corrigir chave de API e reduzir envio indevido de dados |
| Manutenibilidade | Modularidade, reutilização, analisabilidade, modificabilidade, testabilidade | Revisão de componentes React e serviço Gemini |
| Portabilidade | Adaptabilidade, instalabilidade, substituibilidade | Compilar em ambiente limpo e validar dispositivos distintos |

### 2.3.2 Verificação versus validação (V&V)

Verificação pergunta se o produto está sendo construído conforme a especificação; inclui inspeção, revisão de requisitos, análise estática, compilação e testes de componentes. Validação pergunta se o produto atende às necessidades reais do público, incluindo tarefas de uso e aceitação. O relatório anterior registra verificação de tipos TypeScript concluída sem erros e inspeção do código; também registra avaliações de usabilidade informadas pela equipe, mas sem acesso aos formulários originais. Uma evidência não deve ser convertida indevidamente na outra.

## 2.4 Governança de artefatos e documentação técnica básica

O modelo institucional de DVCP da Aula 1 requer identificação, problema real, pessoas afetadas, impacto operacional, solução atualmente utilizada, posicionamento do produto, stakeholders, premissas tecnológicas e requisitos para aceitação. A tabela aplicada abaixo condensa essas dimensões para o ConectaIA, sem tratar campos ainda não evidenciados como decisões aprovadas (GUERRA, 2026).

### 2.4.1 Documento de visão e contexto do projeto (DVCP)

| Campo do DVCP | Descrição aplicada ao ConectaIA |
|---|---|
| Problema | Dificuldade de acesso, entendimento e uso seguro de recursos de IA por usuários iniciantes |
| Público e stakeholders | Pessoas idosas, usuários com baixa familiaridade digital, equipe acadêmica, avaliadores e responsáveis pelo tratamento de dados |
| Objetivo de produto | Orientar e apoiar aprendizagem gradativa e interação consciente com IA |
| Escopo inicial | Início, conversa, trilhas, Sobre a IA, Acessibilidade e Ajuda/Socorro |
| Fora do escopo comprovado | Diagnóstico profissional, integração clínica/financeira, persistência de usuários e garantia de funcionamento da API |
| Riscos | Privacidade, exposição de credencial de API, dependência de provedor externo, acessibilidade ainda não certificada |
| Critérios de sucesso | RF/RNF aceitos, navegação validada, testes documentados, artefatos rastreáveis, feedback do público |
| Estado | Documento inicial produzido nesta adequação; aceitação formal do DVCP pela equipe não demonstrada |

A documentação deve ser versionada, possuir autoria e data, permitir revisão por pares e evitar contradições entre README, código e relatório. Recomenda-se manter catálogo de requisitos, diagramas, backlog, decisões arquiteturais, plano de testes, registro de defeitos e manual conciso de operação. A documentação do usuário usa linguagem comum; a técnica descreve estrutura e dependências; a de requisitos define comportamento verificável.

## 2.5 Respostas-modelo das atividades da Etapa 1

As atividades a seguir são respostas-modelo elaboradas a partir das entregas do enunciado Projeto 4 e dos cinco materiais de apoio. Elas não reproduzem um caderno oficial de questões, que não foi fornecido. Modelos e resultados informados pela equipe não equivalem a execução técnica comprovada.

### 2.5.1 Atividade 1.1 — Definir o problema real e a solução de software

**Resposta-modelo**

O problema abordado é a dificuldade de pessoas idosas e de usuários com baixa familiaridade digital em compreender, acessar e utilizar ferramentas de inteligência artificial com autonomia e segurança. O ConectaIA propõe uma interface web de orientação educativa com navegação direta, trilhas de aprendizagem, conversa assistida, explicações sobre IA, opções de acessibilidade e área de ajuda. A proposta procura reduzir barreiras cognitivas, de legibilidade e de formulação de perguntas.

O público prioritário é composto por pessoas idosas e iniciantes em tecnologia. A investigação anterior registrou três entrevistas semiestruturadas (P1, P2 e P3), com necessidades relacionadas a privacidade, dúvidas de uso, preferência por voz e legibilidade. Trata-se de amostra exploratória e não representativa da população.

**Documento de Visão e Contexto do Projeto — DVCP**

| Campo do DVCP | Descrição aplicada ao ConectaIA |
|---|---|
| Problema | Dificuldade de acesso, entendimento e uso seguro de recursos de IA por usuários iniciantes |
| Público e stakeholders | Pessoas idosas, usuários com baixa familiaridade digital, equipe acadêmica, avaliadores e responsáveis pelo tratamento de dados |
| Objetivo de produto | Orientar e apoiar aprendizagem gradativa e interação consciente com IA |
| Escopo inicial | Início, conversa, trilhas, Sobre a IA, Acessibilidade e Ajuda/Socorro |
| Fora do escopo comprovado | Diagnóstico profissional, integração clínica/financeira, persistência de usuários e garantia de funcionamento da API |
| Riscos | Privacidade, exposição de credencial de API, dependência de provedor externo, acessibilidade ainda não certificada |
| Critérios de sucesso | RF/RNF aceitos, navegação validada, testes documentados, artefatos rastreáveis, feedback do público |
| Estado | Documento inicial produzido nesta adequação; aceitação formal do DVCP pela equipe não demonstrada |

Situação: DVCP documentado; aprovação formal da equipe e validação do escopo permanecem sem comprovação.

### 2.5.2 Atividade 1.2 — Comparar modelos de ciclo de vida e justificar a escolha

**Resposta-modelo**

O desenvolvimento em cascata organiza as fases de forma predominantemente sequencial e favorece planejamento com escopo estável, mas dificulta incorporar feedback tardio. O modelo iterativo revisa progressivamente a solução. O incremental entrega subconjuntos funcionais em ciclos sucessivos. Para o ConectaIA, o modelo incremental com inspeções iterativas é a recomendação documental mais compatível com aprendizagem gradual, correções de acessibilidade e feedback dos usuários.

| Modelo | Vantagens | Limitações | Adequação ao ConectaIA |
|---|---|---|---|
| Cascata | Rastreabilidade e previsibilidade em escopo estável | Mudanças tardias custosas | Útil como referência de documentação; insuficiente isoladamente |
| Iterativo | Incorpora feedback e reduz incertezas | Exige controle de versões e critérios de decisão | Adequado para testar legibilidade, entendimento e navegação |
| Incremental | Antecipação de valor e testes por incremento | Dependências e integração precisam de gestão | Adequado para liberar Início, trilhas, conversa e recursos de apoio gradualmente |
| Ágil híbrido | Concilia feedback e rastreabilidade | Exige acordos e comunicação contínua | Proposta recomendada, sujeita à adoção real da equipe |

Situação: escolha fundamentada em documento. Adoção efetiva de ciclos, datas e versões deve ser comprovada por registros do projeto.

### 2.5.3 Atividade 1.3 — Relacionar qualidade, verificação e validação

**Resposta-modelo**

A referência pedagógica da Aula 1 utiliza as oito características do modelo ISO/IEC 25010:2011. No ConectaIA, adequação funcional pode ser verificada pelo atendimento dos requisitos RF; usabilidade por navegação e facilidade de aprendizagem; segurança pela gestão de credenciais e proteção de dados; compatibilidade pelo funcionamento em diferentes navegadores; confiabilidade pelo tratamento de falhas; e manutenibilidade pela modularização do código. O modelo deve orientar critérios de aceite mensuráveis, não apenas afirmações gerais de qualidade.

| Característica (2011) | Aplicação e avaliação no ConectaIA |
|---|---|
| Adequação funcional | RF01–RF10; verificar navegação e resultados esperados |
| Eficiência de desempenho | Medir tempo de carregamento e resposta de recursos locais/API |
| Compatibilidade | Testar navegadores, interfaces de voz e integração externa |
| Usabilidade | Tarefas T1–T6, teclado, contraste, feedback e leitura |
| Confiabilidade | Validar falha de rede/Gemini e resposta de contingência |
| Segurança | Corrigir chave de API e reduzir envio indevido de dados |
| Manutenibilidade | Revisão de componentes React e serviço Gemini |
| Portabilidade | Compilar em ambiente limpo e validar dispositivos distintos |

Verificação significa confrontar artefatos e implementação com especificações (revisão de código, inspeção de requisitos e testes). Validação significa confirmar que a solução atende às necessidades do usuário (tarefas com participantes e critérios de aceitação). A inspeção TypeScript relatada no relatório não equivale à homologação completa do produto.

### 2.5.4 Atividade 1.4 — Organizar a documentação básica do sistema

**Resposta-modelo**

A documentação do ConectaIA deve manter três grupos vinculados: (a) requisitos e negócio — problema, stakeholders, RF/RNF, priorização e critérios de aceitação; (b) técnica e arquitetura — BPMN, UML, serviços React/TypeScript, riscos e decisões; (c) usuário e operação — instruções de navegação, limitações da IA, orientações de privacidade e roteiro de demonstração.

Artefatos presentes na versão acadêmica: DVCP, matriz RF/RNF, diagramas conceituais, backlog proposto, plano de testes e registro de limitações. A documentação deve ser harmonizada com o repositório real, pois a auditoria anterior encontrou divergências entre README interno e código em relação à integração Gemini e aos recursos de voz.

Situação: resposta documental pronta; integração documental ao repositório e histórico de revisões pendentes de prova.

# 3 ETAPA 2 — REQUISITOS E MODELAGEM INICIAL

## 3.1 Visão geral, ambiguidade e alinhamento estrutural

A Engenharia de Requisitos transforma necessidades, restrições e expectativas em descrições verificáveis. Ambiguidades no início geram propagação de erros: uma expressão vaga sobre “acessibilidade” pode ser interpretada como existência de um botão, embora a aceitação dependa de contraste, foco, operação por teclado e legibilidade real. Para reduzir o efeito cascata, cada requisito do ConectaIA recebe identificação, prioridade, critério de aceite, referência a caso de uso e caso de teste.

O material da Aula 2 reforça a passagem de necessidades humanas à especificação e à arquitetura, com ênfase na propagação de ambiguidades desde a elicitação até os testes. Como controle de qualidade, cada necessidade relatada por P1–P3 deve ser associada a um requisito verificável, caso de uso, item de backlog e caso de teste; a matriz já existente nesta etapa constitui a linha de base documental, sujeita a homologação (FESP, 2026, Aula 2).

## 3.2 Elicitação de requisitos e mapeamento de stakeholders

### 3.2.1 Arsenal metodológico: entrevistas, observação e questionários

A base empírica documentada é uma pesquisa exploratória com três entrevistas semiestruturadas: P1 (50–59 anos, uso digital frequente, dificuldade de formular perguntas), P2 (60–69, uso ocasional, preocupação com informações pessoais e preferência por texto/voz) e P3 (60–70, uso raro, dificuldade visual, receio de errar e preferência por microfone). O tamanho da amostra não permite generalizar para todas as pessoas idosas. Questionários estruturados, observação contextual, card sorting e tree testing são possibilidades complementares; não constam como executados na documentação disponível.

| Participante | Perfil registrado | Relação com IA e necessidades |
|---|---|---|
| P1 | 50–59 anos; utiliza serviços digitais frequentemente. | Já utiliza IA em tarefas diárias e trabalho; relata dificuldade para formular perguntas e preocupações com privacidade. |
| P2 | 60–69 anos; utilização ocasional com necessidade pontual de ajuda. | Conhece e utilizou IA; deseja orientação prática, apresenta dúvidas sobre informações pessoais e prefere texto e voz. |
| P3 | 60–70 anos; uso digital raro e possível dependência de | Não compreende claramente o funcionamento da IA; relata |

| Participante | Perfil registrado | Relação com IA e necessidades |
|---|---|---|
| | familiares. | dificuldade para enxergar letras, receio de errar e preferência pelo microfone. |

| Stakeholder | Interesse ou necessidade | Participação sugerida |
|---|---|---|
| Usuário iniciante/idoso | Ler, perguntar, aprender, recuperar erros e sentir segurança | Entrevistas, testes de tarefa e aceitação |
| Equipe do projeto | Entregar MVP coerente, seguro e documentado | Requisitos, backlog, revisão de código e testes |
| Docente/avaliador | Verificar as cinco etapas e seus artefatos | Revisão acadêmica do relatório e demonstração |
| Provedor Gemini | Receber requisições de API conforme integração prevista | Dependência técnica; política de uso e gestão de chave |
| Responsável por privacidade | Evitar uso indevido de dados pessoais | Revisão de aviso e minimização |

### 3.2.2 Requisitos funcionais e não funcionais

Requisitos funcionais (RF) descrevem serviços e comportamentos observáveis; requisitos não funcionais (RNF) estabelecem qualidades e restrições como acessibilidade, desempenho, segurança e compatibilidade. O inventário RF01–RF10 e RNF01–RNF08 foi preservado do relatório anterior, com a situação documental apontada na versão original.

| Código | Requisito | Evidência e ressalva |
|---|---|---|
| RF01 | Exibir identificação inicial por nome. | Modal na captura; validação de entrada depende de teste. |
| RF02 | Permitir acessar o Início. | Menu e tela visíveis. |
| RF03 | Navegar para Conversar com a IA. | Item de menu e tela visíveis. |
| RF04 | Exibir pergunta, sugestões e histórico demonstrativo. | Figuras de conversa documentadas. |
| RF05 | Oferecer acesso às Trilhas de Sucesso. | Tela temática documentada. |
| RF06 | Apresentar conteúdo Sobre a IA. | Tela educativa documentada. |
| RF07 | Disponibilizar ajustes de acessibilidade. | Controles exibidos; efetividade não aferida. |
| RF08 | Disponibilizar Ajuda / Socorro. | Página e escolhas documentadas. |
| RF09 | Informar a natureza de demonstração da experiência. | Requisito de melhoria; verificar exibição durante a interação. |
| RF10 | Permitir navegação e recuperação por caminhos claros. | Estrutura visual identificada; testar fluxo real. |

| Código | Critério verificável | Validação exigida |
|---|---|---|
| RNF01 | Conteúdo legível com ampliação de fonte. | Inspeção técnica e teste com usuários. |
| RNF02 | Operação por teclado e foco perceptível. | Navegação manual e leitor de tela. |
| RNF03 | Contraste adequado de texto e componentes. | Medição de pares de cores por WCAG 2.2. |
| RNF04 | Navegação consistente entre áreas. | Cenários de navegação completos. |
| RNF05 | Layout responsivo e sem perda de conteúdo. | Larguras desktop/tablet/mobile. |
| RNF06 | Nenhum compartilhamento indevido induzido. | Revisão de avisos e roteiros. |
| RNF07 | Erros apresentados em linguagem | Casos de entrada inválida e |

| Código | Critério verificável | Validação exigida |
|---|---|---|
| | clara. | falhas. |
| RNF08 | Aviso explícito sobre limites da simulação. | Inspeção textual no protótipo. |

### 3.2.3 Matriz de desdobramento dos RNF conforme ISO/IEC 25010

| RNF | Qualidade predominante | Critério de avaliação objetivo | Evidência atual |
|---|---|---|---|
| RNF01 | Usabilidade/ acessibilidade | Ampliar fonte sem perda ou sobreposição de conteúdo | Controle observado; teste pendente |
| RNF02 | Usabilidade/ acessibilidade | Operar fluxo com teclado, foco visível e ordem lógica | Não comprovado |
| RNF03 | Usabilidade/ acessibilidade | Medir contrastes aplicáveis conforme WCAG 2.2 | Não medido |
| RNF04 | Usabilidade | Executar ida e volta entre áreas sem estado inconsistente | Fluxos descritos; execução pendente |
| RNF05 | Portabilidade/usabilidade | Teste em desktop, tablet e mobile sem perda de função | Mobile não demonstrado |
| RNF06 | Segurança | Orientar não inserir dados sensíveis e proteger segredos técnicos | Risco de chave exposta identificado |
| RNF07 | Usabilidade/confiabilidade | Mensagens claras para falha de entrada/rede/API | Casos a executar |
| RNF08 | Adequação funcional | Distinguir demonstração, IA externa e fallback | Verificar rótulos na interface |

## 3.3 Análise, priorização e critérios de aceitação

### 3.3.1 Priorização MoSCoW

MoSCoW separa Must have (indispensável), Should have (importante), Could have (desejável) e Won’t have for now (fora do incremento). A classificação a seguir é uma proposta de priorização acadêmica, não uma votação comprovada dos usuários ou decisão formal da equipe. São tratados como imprescindíveis os caminhos principais e as barreiras de segurança que podem impedir a apresentação responsável do protótipo.

| Classe | Itens propostos | Justificativa |
|---|---|---|
| Must | RF02, RF03, RF05, RF06, RF07, RF08; RNF02, RNF06, RNF07 | Fluxo essencial, acessibilidade mínima e prevenção de risco |
| Should | RF01, RF04, RF10; RNF01, RNF03, RNF04 | Reduzir fricção e melhorar usabilidade |
| Could | Voz, personalização de trilhas, evolução de conteúdo | Adiciona valor, mas depende de suporte e validação |
| Won’t now | Diagnósticos profissionais, histórico persistente e coleta ampliada de dados | Não demonstrados e inadequados ao MVP atual |

### 3.3.2 Critérios de aceitação com BDD/Gherkin

Os critérios de aceitação devem ser observáveis e independentes de implementação. O modelo BDD/Gherkin usa Dado/Quando/Então para alinhar pessoa de produto, desenvolvimento e teste. Os exemplos abaixo são especificações a validar, não relatórios de execução.

Para cumprir a orientação didática, o critério de aceitação deve descrever pré-condição, evento e resultado verificável na forma Dado/Quando/Então. Nos exemplos CA-01 a CA-04, toda afirmação de conformidade somente poderá ser marcada “aprovada” depois da execução e do registro do teste associado. A matriz original de decisão MoSCoW da Aula 2 permanece reproduzida no Anexo B como referência metodológica (FESP, 2026, Aula 2).

- CA-01 — Dado que o usuário está no Início, quando seleciona “Trilhas de Sucesso”, então a aplicação apresenta as quatro opções temáticas e permite abrir uma delas.
- CA-02 — Dado que o usuário está na área de acessibilidade, quando altera a escala de fonte, então o conteúdo permanece legível, com controles alcançáveis e sem truncamentos.
- CA-03 — Dado que a API de IA está indisponível, quando o usuário envia uma pergunta, então a interface informa o limite de serviço e utiliza o mecanismo de contingência sem apresentar saída como resposta externa confirmada.
- CA-04 — Dado que a navegação é feita apenas pelo teclado, quando o usuário percorre as ações principais, então cada ação recebe foco perceptível e pode ser ativada sem mouse.

## 3.4 Modelagem de processos de negócio com BPMN 2.0

### 3.4.1 Elementos básicos: eventos, atividades, gateways e pools

BPMN permite representar fluxo de trabalho por eventos de início/fim (círculos), atividades (retângulos com cantos arredondados), gateways de decisão (losangos), fluxos de sequência (setas) e pools/lanes para participantes. O desenho a seguir descreve um fluxo To-Be simplificado e proposto de interação no ConectaIA, e não uma transcrição automática do aplicativo. As interfaces de usuário

e o comportamento técnico precisam ser confrontados em teste executável (OMG, 2014).

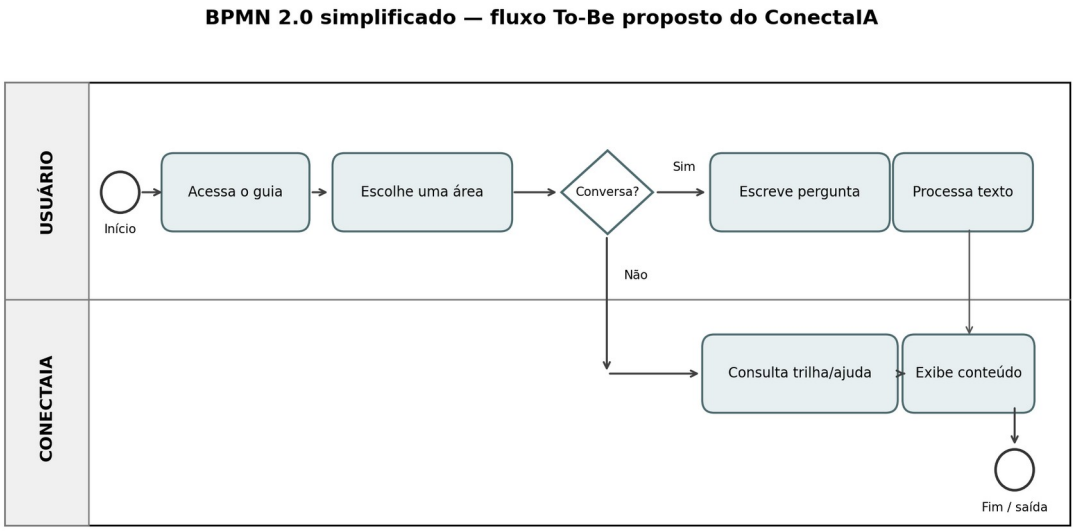



Figura 1 — BPMN 2.0: processo To-Be conceitual de uso do ConectaIA. Fonte: elaboração técnica para o Projeto 4.

### 3.4.2 Processo As-Is e To-Be de atendimento educativo

| Aspecto | As-Is (situação-problema conceitual) | To-Be (solução proposta) |
|---|---|---|
| Início | Usuário procura fontes dispersas e pede ajuda | Acessa página Início e escolhe funcionalidade |
| Esclarecimento | Formula perguntas sem exemplos e sem orientação | Utiliza sugestões de perguntas e conteúdo “Sobre a IA” |
| Aprendizado | Depende de terceiros e consulta conteúdos desconectados | Percorre trilhas temáticas progressivas |
| Dificuldade de leitura | Abandona ou solicita ajuda externa | Ajusta tamanho de fonte, contraste ou recorre a Ajuda/Socorro |
| Riscos de informação | Compartilha dados sem ter clareza sobre consequências | Recebe orientações de privacidade e limites da ferramenta |

### 3.4.3 Catálogo semântico BPMN e exemplo didático FESP

| Elemento BPMN | Significado | Implicação de software |
|---|---|---|
| Evento inicial/final | Condição de início/encerramento do fluxo | Definir disparadores e estado de conclusão |
| Tarefa do usuário | Ação humana observável | Representar entrada, clique, leitura ou escolha |
| Tarefa do sistema | Processamento automatizado | Definir serviço, resposta e tratamento de erro |
| Gateway exclusivo | Escolha condicional | Ramo de decisão com condição explícita |
| Pool/lane | Responsabilidades e fronteiras | Separar ator humano, aplicativo e serviço externo |
| Fluxo de mensagem | Comunicação entre participantes | Representar requisição a fornecedor quando pertinente |

O material oficial da Aula 2 apresenta o processo didático “Triagem e Alocação Automatizada de Chamados”, a pool “Sistema de Atendimento FESP” e o diagrama do módulo acadêmico. Esses exemplos são parte do texto didático e foram preservados no Anexo B, sem atribuí-los indevidamente ao sistema ConectaIA. Para a aplicação concreta ao produto do trabalho, mantém-se em paralelo o processo To-Be de interação educativa representado na Figura 1. A correlação entre o modelo geral de aula e a proposta do ConectaIA está documentada na matriz de rastreabilidade desta etapa (FESP, 2026, Aula 2).

## 3.5 Modelagem básica de sistemas com UML

### 3.5.1 Diagrama e dicionário de casos de uso

UML oferece representações padronizadas da estrutura e do comportamento. Em casos de uso, um ator inicia um serviço e o sistema estabelece a fronteira funcional. O modelo conceitual abaixo deriva das áreas documentadas; não comprova o comportamento de todos os casos em navegador (OMG, 2017).

**UML — casos de uso conceituais (não é diagrama extraído do código)**

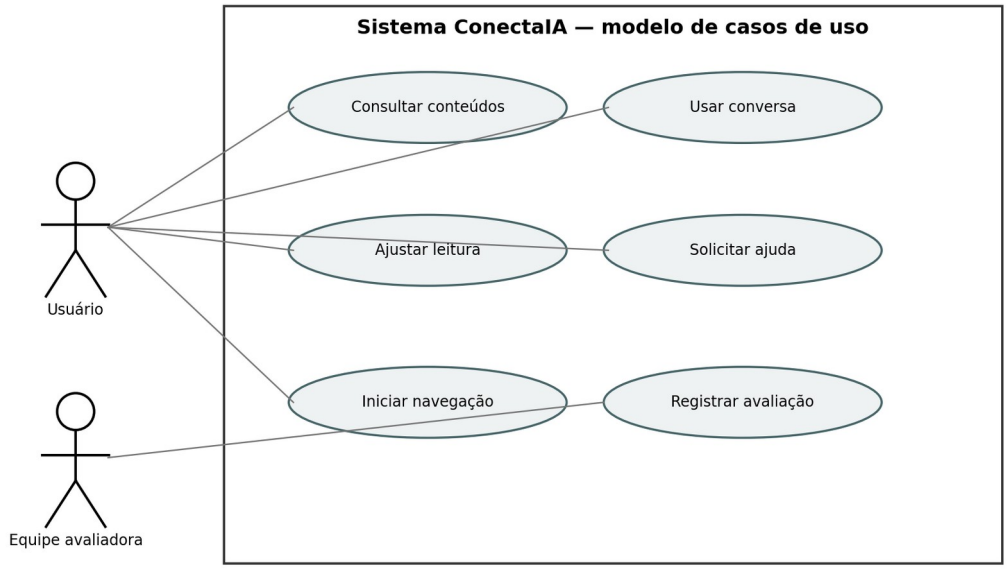



Figura 2 — UML: casos de uso conceituais do ConectaIA. Fonte: elaboração técnica para o Projeto 4.

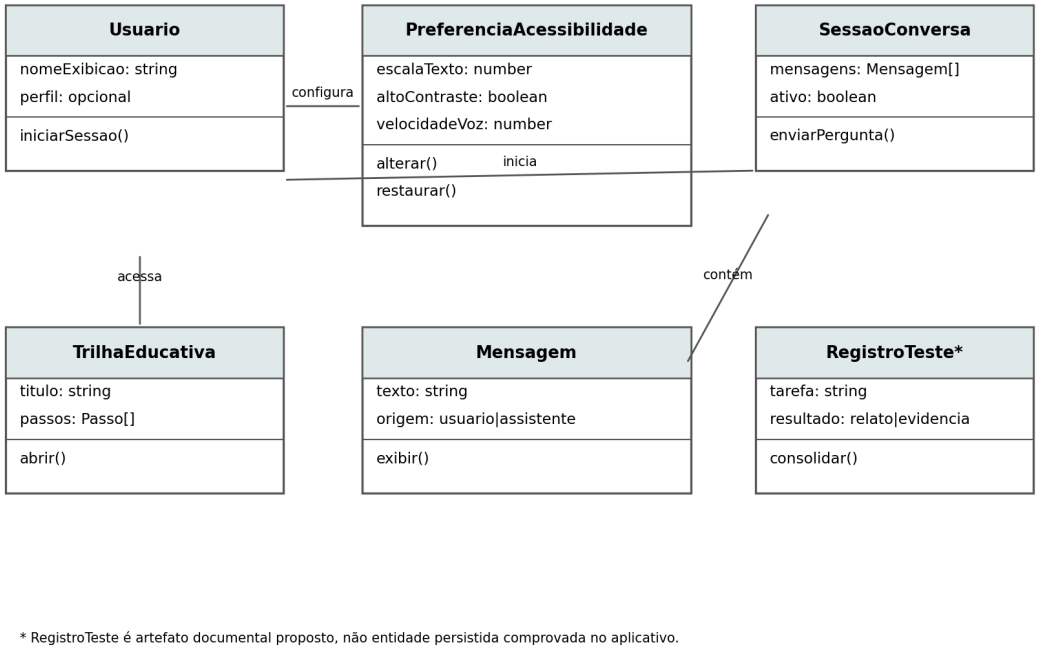
| Caso de uso | Ator | Pré-condição | Fluxo principal | Pós-condição / exceção |
|---|---|---|---|---|
| UC01 Iniciar navegação | Usuário | Página acessível | Informar nome opcional / entrar no Início | Exibir opções; tratar nome inválido conforme decisão futura |
| UC02 Consultar trilhas | Usuário | Início ou menu | Selecionar Trilhas e abrir conteúdo | Exibir passos educativos |
| UC03 Conversar | Usuário | Área de conversa | Digitar pergunta e | Mostrar resposta; |

| Caso de uso | Ator | Pré-condição | Fluxo principal | Pós-condição / exceção |
|---|---|---|---|---|
| com IA | | | enviar | em falha, avisar e aplicar fallback |
| UC04 Configurar leitura | Usuário | Área Acessibilidade | Ajustar escala e contraste | Aplicar mudança sem perda de navegação |
| UC05 Pedir ajuda | Usuário | Menu disponível | Selecionar Ajuda/Socorro | Oferecer caminhos de orientação |
| UC06 Registrar avaliação | Equipe avaliadora | Roteiro e consentimento | Executar tarefas e registrar resultados | Produzir ficha rastreável (processo externo proposto) |

### 3.5.2 Diagrama de classes simplificado da camada de domínio

O diagrama de classes representa conceitos do domínio e associações, não necessariamente classes já presentes no TypeScript. Usuário, preferência de leitura, conversa, mensagens e trilhas permitem discutir responsabilidades e relações. RegistroTeste é identificado como artefato documental externo e não deve ser interpretado como tabela persistida no aplicativo.

**UML — classes de domínio conceituais do ConectaIA**




* RegistroTeste é artefato documental proposto, não entidade persistida comprovada no aplicativo.

Figura 3 — UML: classes de domínio conceituais. Fonte: elaboração técnica para o Projeto 4.

| Entidade conceitual | Dados e responsabilidades | Estado de implementação |
|---|---|---|
| Usuario | Nome de apresentação e início da sessão | Nome e modal mencionados na auditoria do código; persistência não comprovada |
| PreferenciaAcessibilidade | Tamanho, contraste e velocidade de voz | Controles apontados no código; testes visuais pendentes |
| SessaoConversa / Mensagem | Perguntas, respostas, origem e | Histórico e envio identificados em |

| Entidade conceitual | Dados e responsabilidades | Estado de implementação |
|---|---|---|
| | histórico local | src/App.tsx |
| TrilhaEducativa / Passo | Título e sequência de conteúdo didático | Quatro trilhas descritas na auditoria |
| RegistroTeste | Tarefa, execução, erro, tempo e evidência | Modelo de documentação; não é funcionalidade do protótipo |

O tópico do material didático “Diagrama de Casos de Uso do Módulo Acadêmico” refere-se a outro exemplo curricular. Sem a figura original, é possível apenas descrever sua lógica usual de atores (aluno, docente, coordenação), operações (matrícula, consulta, lançamento de notas) e regras de autorização, não apresentar o exemplo exato como conteúdo fornecido.

### 3.5.3 Rastreabilidade bidirecional e integração entre cursos

A rastreabilidade bidirecional conecta necessidade → requisito → caso de uso/modelagem → história de usuário → teste → evidência, e permite percorrer o caminho inverso para justificar cada implementação. A mesma matriz favorece integração entre pesquisa com usuários, UX, Engenharia de Software, QA e governança de privacidade; evita funcionalidades sem motivação e requisitos sem critério de verificação.

A Aula 2 denomina “fio de ouro” o vínculo bidirecional entre demanda, requisito, caso de uso ou processo, estrutura, código e teste. A matriz do ConectaIA documenta parte desses vínculos; sua auditoria requer identificadores consistentes no repositório e evidências de execução. O exemplo “Módulo Acadêmico” constante do material original é apenas demonstração didática e foi preservado no Anexo B (FESP, 2026, Aula 2).

| Necessidade | Requisito | Caso de uso / Processo | História | Teste planejado | Situação |
|---|---|---|---|---|---|
| Aprender passo a passo | RF05 | UC02 / BPMN trilhas | US02 | CT02 | Tarefa T3 relatada; reteste técnico pendente |
| Fazer perguntas | RF03, RF04 | UC03 / BPMN conversa | US03 | CT03/CT04 | Código inspecionado; API externa não homologada |
| Ajustar leitura | RF07, RNF01, RNF03 | UC04 | US04 | CT05/CT06 | Controle em código; WCAG não medida |
| Encontrar ajuda | RF08, RF10 | UC05 | US05 | CT07 | Tarefa T6 relatada; navegação não comprovada |
| Privacidade e segurança | RNF06, RF09 | UC03 | US09 | CT09/CT10 | Chave de API exposta na inspeção prévia |

## 3.6 Respostas-modelo das atividades da Etapa 2

As atividades a seguir são respostas-modelo elaboradas a partir das entregas do enunciado Projeto 4 e dos cinco materiais de apoio. Elas não reproduzem um caderno oficial de questões, que não foi fornecido. Modelos e resultados informados pela equipe não equivalem a execução técnica comprovada.

### 3.6.1 Atividade 2.1 — Levantar requisitos e mapear stakeholders

**Resposta-modelo**

Foram relatadas três entrevistas semiestruturadas com participantes identificados como P1, P2 e P3. Os relatos apontam dificuldades para formular perguntas, preocupação com privacidade, necessidade de orientação, texto ampliável e possibilidade de interação por voz. Essas observações justificam requisitos de navegação, exemplos de perguntas, acessibilidade e ajuda. Questionários quantitativos e observação contextual podem ampliar o estudo, mas não devem ser descritos como procedimentos já realizados.

| Participante | Perfil registrado | Relação com IA e necessidades |
|---|---|---|
| P1 | 50–59 anos; utiliza serviços digitais frequentemente. | Já utiliza IA em tarefas diárias e trabalho; relata dificuldade para formular perguntas e preocupações com privacidade. |
| P2 | 60–69 anos; utilização ocasional com necessidade pontual de ajuda. | Conhece e utilizou IA; deseja orientação prática, apresenta dúvidas sobre informações pessoais e prefere texto e voz. |
| P3 | 60–70 anos; uso digital raro e possível dependência de familiares. | Não compreende claramente o funcionamento da IA; relata dificuldade para enxergar letras, receio de errar e preferência pelo microfone. |

| Stakeholder | Interesse ou necessidade |
|---|---|
| Usuário iniciante/idoso | Ler, perguntar, aprender, recuperar erros e sentir segurança |
| Equipe do projeto | Entregar MVP coerente, seguro e documentado |
| Docente/avaliador | Verificar as cinco etapas e seus artefatos |
| Provedor Gemini | Receber requisições de API conforme integração prevista |
| Responsável por privacidade | Evitar uso indevido de dados pessoais |

Situação: entrevistas relatadas no projeto original; registros primários e validação da síntese precisam ser anexados caso existam.

### 3.6.2 Atividade 2.2 — Especificar e priorizar requisitos com critérios de aceitação

**Resposta-modelo**

Os requisitos funcionais descrevem os serviços oferecidos, enquanto os requisitos não funcionais definem qualidades de uso, operação e proteção. O ConectaIA possui dez RF e oito RNF documentados na entrega revisada. A classificação MoSCoW abaixo é proposta e depende de aprovação pelos responsáveis do produto.

| Código | Requisito |
|---|---|
| RF01 | Exibir identificação inicial por nome. |
| RF02 | Permitir acessar o Início. |
| RF03 | Navegar para Conversar com a IA. |
| RF04 | Exibir pergunta, sugestões e histórico demonstrativo. |
| RF05 | Oferecer acesso às Trilhas de Sucesso. |
| RF06 | Apresentar conteúdo Sobre a IA. |
| RF07 | Disponibilizar ajustes de acessibilidade. |
| RF08 | Disponibilizar Ajuda / Socorro. |
| RF09 | Informar a natureza de demonstração da experiência. |
| RF10 | Permitir navegação e recuperação por caminhos claros. |

| Código | Critério verificável |
|---|---|
| RNF01 | Conteúdo legível com ampliação de fonte. |
| RNF02 | Operação por teclado e foco perceptível. |
| RNF03 | Contraste adequado de texto e componentes. |
| RNF04 | Navegação consistente entre áreas. |
| RNF05 | Layout responsivo e sem perda de conteúdo. |
| RNF06 | Nenhum compartilhamento indevido induzido. |
| RNF07 | Erros apresentados em linguagem clara. |
| RNF08 | Aviso explícito sobre limites da simulação. |

| Classe | Itens propostos | Justificativa |
|---|---|---|
| Must | RF02, RF03, RF05, RF06, RF07, RF08; RNF02, RNF06, RNF07 | Fluxo essencial, acessibilidade mínima e prevenção de risco |
| Should | RF01, RF04, RF10; RNF01, RNF03, RNF04 | Reduzir fricção e melhorar usabilidade |
| Could | Voz, personalização de trilhas, evolução de conteúdo | Adiciona valor, mas depende de suporte e validação |
| Won’t now | Diagnósticos profissionais, histórico persistente e coleta ampliada de dados | Não demonstrados e inadequados ao MVP atual |

**Critérios de aceitação — exemplos BDD/Gherkin**

- Dado que o usuário está no Início, quando seleciona “Trilhas de Sucesso”, então a aplicação deve apresentar as quatro opções temáticas.
- Dado que o usuário abriu Acessibilidade, quando altera o tamanho da fonte, então o texto deve mudar sem sobreposição e sem perda de conteúdo.
- Dado que a rede ou API esteja indisponível, quando o usuário enviar uma pergunta, então deverá visualizar aviso claro e resposta de contingência identificada.
- Dado que o usuário navega apenas pelo teclado, quando percorre os elementos de interface, então todos os controles principais devem receber foco visível e operar corretamente.

Situação: requisitos e aceites documentados; a aprovação e a execução dos testes de aceitação continuam pendentes.

### 3.6.3 Atividade 2.3 — Modelar os processos As-Is e To-Be em BPMN

**Resposta-modelo**

As-Is (situação-problema conceitual): o usuário procura orientações dispersas sobre IA, encontra linguagem complexa, depende de terceiros para formular perguntas e pode abandonar a atividade devido a dificuldades de leitura. To-Be (proposta): acessar o ConectaIA, escolher uma seção, selecionar trilha educativa ou conversa, receber orientação, ajustar acessibilidade quando necessário e concluir ou retornar ao Início. Gateway representa a escolha do caminho; atividades representam ações do usuário e respostas da interface.

| Aspecto | As-Is (situação-problema conceitual) | To-Be (solução proposta) |
|---|---|---|
| Início | Usuário procura fontes dispersas e pede ajuda | Acessa página Início e escolhe funcionalidade |

| Aspecto | As-Is (situação-problema conceitual) | To-Be (solução proposta) |
|---|---|---|
| Esclarecimento | Formula perguntas sem exemplos e sem orientação | Utiliza sugestões de perguntas e conteúdo “Sobre a IA” |
| Aprendizado | Depende de terceiros e consulta conteúdos desconectados | Percorre trilhas temáticas progressivas |
| Dificuldade de leitura | Abandona ou solicita ajuda externa | Ajusta tamanho de fonte, contraste ou recorre a Ajuda/Socorro |

**BPMN 2.0 simplificado — fluxo To-Be proposto do ConectaIA**

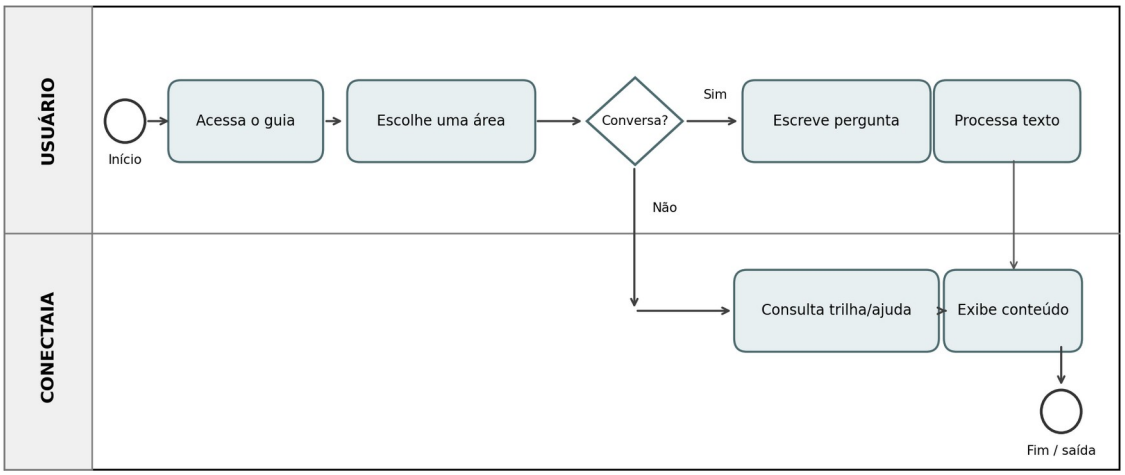



Figura RA1 — Fluxo BPMN conceitual para utilização do ConectaIA; não é evidência de execução.

Situação: modelo conceitual preparado; comparar com as interações reais da aplicação antes de afirmar conformidade executável.

### 3.6.4 Atividade 2.4 — Criar diagramas UML e matriz de rastreabilidade

**Resposta-modelo**

A modelagem de casos de uso representa as ações do usuário nas áreas da aplicação. A modelagem de classes descreve entidades conceituais, como preferências de acessibilidade, sessão de conversa, mensagem e trilha educativa. Esses modelos são conceituais e não devem ser tratados como reprodução literal de classes TypeScript existentes.

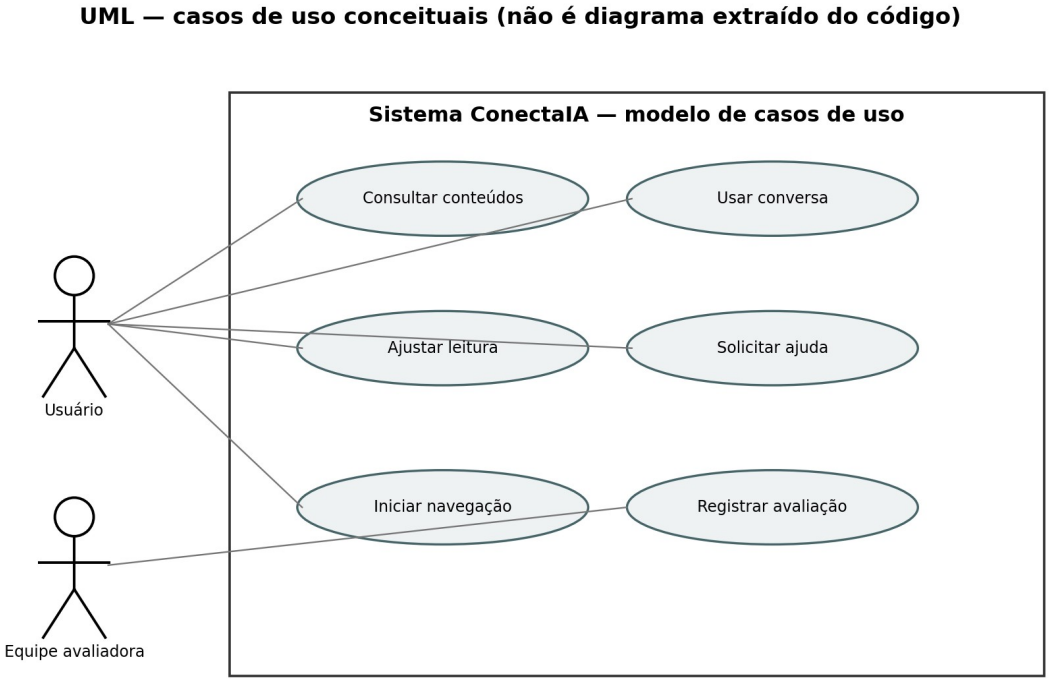



Figura RA2 — Casos de uso conceituais do ConectaIA.

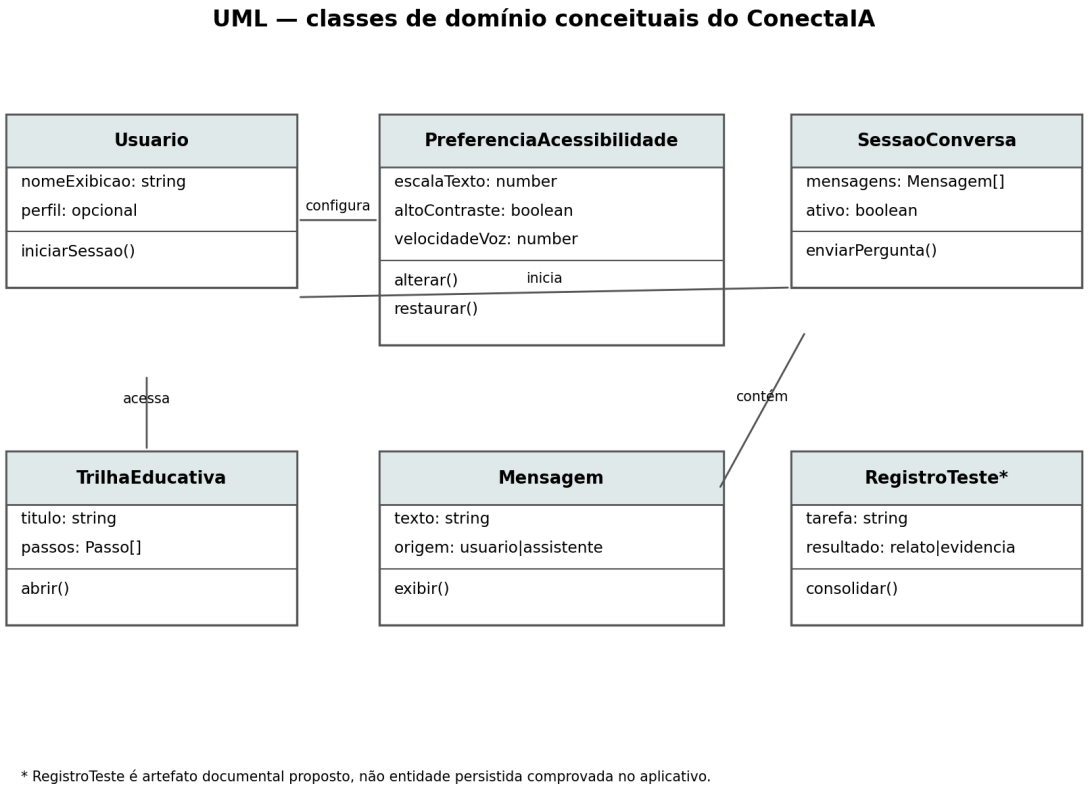



Figura RA3 — Classes de domínio conceituais do ConectaIA.

**Rastreabilidade — vínculo do problema ao teste**

| Necessidade | Requisito | Caso de uso / Processo | História | Teste planejado |
|---|---|---|---|---|
| Aprender passo a passo | RF05 | UC02 / BPMN trilhas | US02 | CT02 |
| Fazer perguntas | RF03, RF04 | UC03 / BPMN conversa | US03 | CT03/CT04 |
| Ajustar leitura | RF07, RNF01, RNF03 | UC04 | US04 | CT05/CT06 |
| Encontrar ajuda | RF08, RF10 | UC05 | US05 | CT07 |
| Privacidade e segurança | RNF06, RF09 | UC03 | US09 | CT09/CT10 |

Situação: diagramas e matriz produzidos; ainda é necessária a revisão de consistência entre BPMN, UML, requisitos, código e testes executados.

# 4 ETAPA 3 — METODOLOGIAS ÁGEIS E PLANEJAMENTO

## 4.1 Paradigma ágil, Manifesto Ágil e complexidade

O Playbook institucional da Aula 3 articula três planos: mentalidade ágil, sistema de trabalho (Scrum/Kanban) e execução (backlog, decomposição, estimativas e critérios INVEST). No ConectaIA, a mentalidade orienta descobertas com P1–P3; a gestão de fluxo organiza histórias e revisões; e a execução depende de critérios de aceitação e software demonstrável. A descrição não implica que cerimônias ou reuniões específicas já tenham ocorrido (FESP, 2026, Aula 3).

### 4.1.1 Valores do Manifesto Ágil — comparação

O Manifesto para Desenvolvimento Ágil de Software (2001) privilegia adaptação, entrega útil, colaboração e feedback, sem abolir processos, documentação, contratos e planejamento. O desafio é ajustar o grau de formalismo ao risco do trabalho. Para o ConectaIA, as práticas ágeis servem ao refinamento contínuo das funções acessíveis, desde que acompanhadas de requisitos verificáveis e controle dos riscos de integração externa.

| Valor enfatizado | Elemento complementar que mantém valor | Aplicação proposta no projeto |
|---|---|---|
| Indivíduos e interações | Processos e ferramentas | Reuniões curtas para revisar linguagem, fluxo e riscos |
| Software funcionando | Documentação abrangente | Demonstrar incrementos sem perder rastreabilidade e instruções |
| Colaboração com cliente | Negociação contratual | Obter feedback de usuários e alinhamento com docentes |
| Responder a mudanças | Seguir um plano | Ajustar backlog conforme testes de usabilidade |

### 4.1.2 Os doze princípios operacionais da agilidade

- Princípio 1: Satisfazer o usuário por entrega contínua de valor. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 2: Aceitar mudanças relevantes mesmo tardiamente. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 3: Entregar versões funcionais com frequência. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 4: Promover colaboração cotidiana entre envolvidos. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.

- Princípio 5: Confiar em pessoas motivadas e prover apoio. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 6: Preferir comunicação direta e compreensão compartilhada. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 7: Usar software em funcionamento como medida central de progresso. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 8: Manter ritmo de execução sustentável. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 9: Investir em qualidade técnica e bom design. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 10: Praticar simplicidade e reduzir trabalho sem valor. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 11: Estimular organização e responsabilidade coletiva. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.
- Princípio 12: Inspecionar resultados e ajustar processo regularmente. No ConectaIA, aplicar conforme necessidade e viabilidade, sem declarar cumprimento histórico.

### 4.1.3 Cynefin e domínios de decisão

Cynefin organiza contextos conforme a relação entre causa e efeito e auxilia a selecionar estratégias de trabalho. Uma necessidade de trocar o texto de um botão pode ser clara; já prever efeitos de ajustes de acessibilidade em diferentes perfis exige investigação. Dependências de API, voz e privacidade introduzem incertezas que recomendam experimentação controlada.

| Domínio | Natureza | Resposta de gestão | Exemplo aplicado |
|---|---|---|---|
| Claro | Causa e efeito conhecidos | Identificar → categorizar → responder | Corrigir rótulo documental inconsistente |
| Complicado | Exige análise especializada | Identificar → analisar → responder | Investigar falha de build e dependência Rollup |
| Complexo | Padrões surgem com experimentação | Experimentar → perceber → responder | Testar interação com público de baixa familiaridade |
| Caótico | Sem relação estável aparente | Agir → perceber → responder | Contenção imediata de chave de API exposta |
| Confuso/aporia | Domínio ainda não identificado | Separar problemas e recolher evidências | Distinguir defeito real de hipótese de teste |

## 4.2 Scrum: papéis, eventos e artefatos

### 4.2.1 Controle empírico e responsabilidades

Scrum emprega transparência, inspeção e adaptação. A Scrum Guide estabelece as accountabilities Product Owner, Scrum Master e Developers, com eventos Sprint, Sprint Planning, Daily Scrum, Sprint Review e Sprint Retrospective. O projeto pode aplicar esse modelo, mas os arquivos fornecidos não evidenciam nomeação formal de papéis ou sprints já executadas. As responsabilidades descritas a seguir constituem arranjo sugerido (SCHWABER; SUTHERLAND, 2020).

| Papel | Responsabilidade principal | Aplicação sugerida ao ConectaIA |
|---|---|---|
| Product Owner | Maximizar valor e ordenar backlog | Validar prioridades com usuário/docente |
| Scrum Master | Promover efetividade e remover impedimentos | Facilitar eventos, visibilidade de riscos e cooperação |
| Developers | Criar incremento conforme Definition of Done | Implementar, revisar, testar e documentar |
| Apoiadores (não papéis Scrum) | Contribuição especializada | UX, acessibilidade, privacidade e avaliação |

### 4.2.2 Ciclo operacional e eventos formais

A Sprint é um período fixo de planejamento e produção de incremento. Na Planning, define-se objetivo e itens selecionados; a Daily Scrum inspeciona progresso e adapta o plano; na Review, demonstra-se o incremento e coleta-se feedback; na Retrospective, ajusta-se o método de trabalho. Proposta de duas sprints acadêmicas de duração a confirmar pela equipe: a primeira foca navegação, trilhas e critérios de leitura; a segunda foca conversa, contingência, privacidade e evidências de teste. Não se atribuem datas realizadas ou resultados sem comprovação.

| Sprint proposta | Objetivo de incremento | Atividades e verificações | Saída esperada |
|---|---|---|---|
| Sprint A — base | Navegação e aprendizagem utilizáveis | RF02, RF05, RF06, RF08; critérios de leitura; documentação | Fluxos demonstráveis e registros de teste |
| Sprint B — interação | Conversa segura e validação do MVP | RF03, RF04, RF07, RF09; correção de segredo; testes | Versão demonstrável e evidências anexadas |

### 4.2.3 Artefatos Scrum e seus compromissos

| Artefato | Compromisso | Definição para o projeto |
|---|---|---|
| Product Backlog | Product Goal | Apoiar aprendizagem segura e acessível sobre IA |
| Sprint Backlog | Sprint Goal | Objetivo mensurável do |

| Artefato | Compromisso | Definição para o projeto |
|---|---|---|
| | | incremento de cada ciclo |
| Increment | Definition of Done | Critérios de aceite, testes e rastreabilidade satisfeitos |

## 4.3 Kanban e engenharia de fluxo

A Aula 3 relaciona limitação de trabalho em progresso (WIP), lead time, cycle time e vazão de entrega. Pela Lei de Little, em regime estável, o trabalho em progresso médio é igual à taxa média de saída multiplicada pelo tempo médio no sistema ($L = \lambda W$). Sem série histórica de cartões ou data de conclusão das tarefas, o ConectaIA não dispõe de medições empíricas para essa equação; os valores e gráficos apresentados são conceituais (FESP, 2026, Aula 3; LITTLE, 1961).

### 4.3.1 Seis práticas fundamentais e gestão visual

As seis práticas associadas ao Kanban Method são visualizar o trabalho, limitar WIP, gerenciar o fluxo, explicitar políticas, implementar ciclos de feedback e melhorar colaborativamente por experimentos. Para o ConectaIA, um quadro com estados Backlog → A fazer → Em andamento → Validação → Concluído reduz dispersão e permite evidenciar bloqueios. Os limites de WIP mostrados são somente uma proposta pedagógica, sujeita à capacidade real da equipe.

**Quadro Kanban ilustrativo — proposta de organização, não status real da equipe**

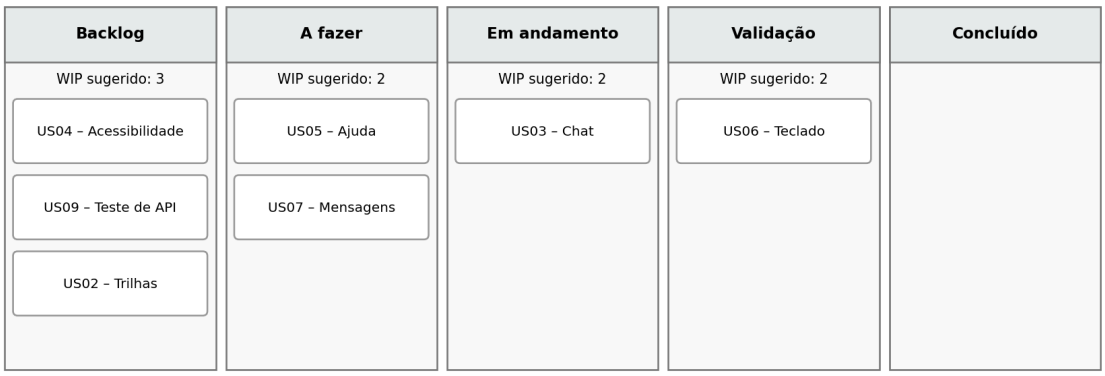



Figura 4 — Quadro Kanban ilustrativo, sem correspondência comprovada com o estado real das tarefas. Fonte: elaboração técnica.

### 4.3.2 Métricas de fluxo, Lei de Little e gráfico de WIP

Lead time é o intervalo entre solicitação e entrega; cycle time é o intervalo entre início efetivo e conclusão; throughput representa itens concluídos por unidade de tempo; WIP é o total de itens iniciados e não terminados. Em sistema estável, a

Lei de Little relaciona WIP médio, taxa média de saída e tempo médio no sistema: WIP = throughput × lead time. Essas medidas exigem carimbos de tempo de itens; como não foram fornecidos, não se apresentam indicadores reais do ConectaIA.

Exemplo puramente didático: com saída média hipotética de 2 itens por semana e lead time médio hipotético de 1,5 semana, WIP médio seria de 3 itens. O gráfico a seguir representa uma tendência conceitual de produtividade frente ao crescimento do WIP, não medições do projeto nem uma relação universal válida para todos os sistemas.

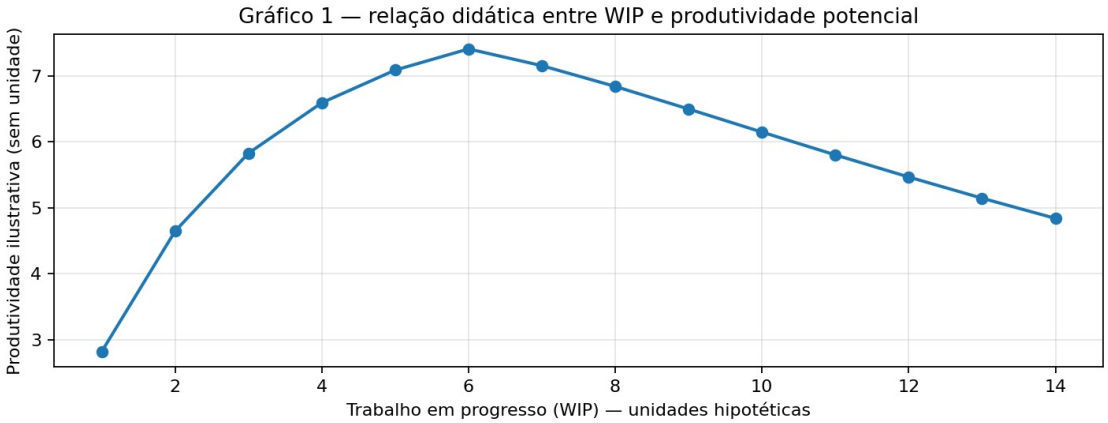



Gráfico 1 — Ilustração conceitual da relação entre WIP e produtividade, sem dados empíricos. Fonte: elaboração técnica.

### 4.3.3 Scrum versus Kanban

| Critério | Scrum | Kanban | Recomendação para ConectaIA |
|---|---|---|---|
| Cadência | Sprints fixas | Fluxo contínuo | Sprints curtas com quadro de fluxo |
| Papéis | Accountabilities prescritas | Sem papéis novos obrigatórios | Definir responsabilidades sem atribuições fictícias |
| Mudanças | Replanejamento pelo objetivo da Sprint | Reposição contínua | Revisar prioridades no refinamento |
| Métricas | Objetivo de Sprint e incremento | Lead time, cycle time, throughput e WIP | Acompanhar entregas e gargalos |
| Visualização | Sprint Backlog | Quadro e políticas explícitas | Unificar gestão documental e desenvolvimento |

## 4.4 Engenharia de backlog, histórias de usuário e estimativas

A Aula 3 exige que as histórias observem o acrônimo INVEST: independentes, negociáveis, valiosas, estimáveis, pequenas e testáveis. Para o planejamento da turma, recomenda-se estimar com Planning Poker apenas após refinamento coletivo, registrar divergências e relacionar cada item a um critério de

pronto (Definition of Done). O backlog do ConectaIA é uma proposta inicial de engenharia, não medição de velocidade da equipe (FESP, 2026, Aula 3).

### 4.4.1 Product backlog proposto para o MVP

| ID | História de usuário (formato proposto) | MoSCoW | SP* | Rastreio | Teste |
|---|---|---|---|---|---|
| US01 | Como usuário, quero chegar facilmente ao Início para escolher por onde começar. | Must | 3 | RF01/RF02 | CT01 |
| US02 | Como usuário, quero abrir trilhas simples para aprender progressivamente. | Must | 5 | RF05 | CT02 |
| US03 | Como usuário, quero digitar uma pergunta e ver a resposta para praticar IA. | Must | 8 | RF03/RF04 | CT03/04 |
| US04 | Como usuário com dificuldade de leitura, quero ajustar tamanho e contraste. | Must | 5 | RF07/RNF01/RNF03 | CT05/06 |
| US05 | Como usuário iniciante, quero encontrar Ajuda/Socorro para recuperar orientação. | Must | 3 | RF08/RF10 | CT07 |
| US06 | Como usuário, quero navegar por teclado para utilizar o sistema sem mouse. | Must | 5 | RNF02 | CT08 |
| US07 | Como usuário, quero mensagens claras quando uma requisição falhar. | Must | 5 | RNF07 | CT04 |
| US08 | Como usuário, quero entender limitações da IA e cuidados com dados. | Should | 3 | RF06/RF09/RNF08 | CT09 |
| US09 | Como mantenedor, quero manter a chave de API fora do navegador para evitar exposição. | Must | 8 | RNF06 | CT10 |
| US10 | Como equipe, quero disponibilizar instruções de uso e arquitetura versionadas. | Should | 3 | DVCP/Docs | Revisão |

| ID | História de usuário (formato proposto) | MoSCoW | SP* | Rastreio | Teste |
|---|---|---|---|---|---|
| US11 | Como usuário, quero acionar microfone quando suportado para interagir por voz. | Could | 8 | Voz | Teste de browser |
| US12 | Como equipe, quero registros auditáveis de testes para validar qualidade. | Must | 5 | QA | Evidências |

* Story points (SP) são estimativas preliminares ilustrativas, não resultados de Planning Poker realizado. Priorização e esforço devem ser homologados pelo time.

### 4.4.2 Critério INVEST e auditoria das histórias

INVEST avalia se a história é Independente, Negociável, Valiosa, Estimável, Pequena e Testável. Uma história de acessibilidade genérica tende a ser ampla e pouco testável; separar aumento de fonte, contraste e navegação por teclado permite estabelecer critérios de aceitação objetivos. A matriz propõe ajustes de qualidade para o backlog, sem afirmar que houve cerimônia formal de refatoração.

| História | Risco INVEST | Ajuste proposto | Aceite |
|---|---|---|---|
| US03 – Conversar | Integração externa adiciona dependência | Separar UI, serviço, contingência e tratamento de erro | CT03 e CT04 |
| US04 – Acessibilidade | Grande escopo e difícil estimativa | Separar fonte e contraste por critérios medíveis | CT05 e CT06 |
| US09 – Proteger segredo | Depende de decisão arquitetural | Definir proxy, env no servidor e rotação de chave | CT10 |
| US12 – Evidências | Critério de pronto subjetivo | Exigir resultado, ambiente, autor, data e anexos | Checklist de entrega |

### 4.4.3 Dimensionamento relativo e Planning Poker

No Planning Poker, participantes estimam complexidade relativa de histórias, discutem divergências e convergem com base em critérios compartilhados. Pode-se adotar escala 1, 2, 3, 5, 8, 13. Os valores da tabela são apenas hipóteses iniciais elaboradas para demonstrar planejamento, pois não foi disponibilizado histórico de votação, capacidade da equipe ou velocidade. Antes do compromisso de Sprint, dividir itens maiores e reestimar com os responsáveis.

Síntese operacional: o Product Goal define o propósito; o Product Backlog descreve prioridades; a Sprint seleciona objetivos e itens; o Kanban torna gargalos visíveis; critérios de aceite e Definition of Done protegem qualidade; os testes

devolvem informações para ajustes. Esse é o playbook proposto para o Projeto Integrador, a ser validado na prática.

## 4.5 Respostas-modelo das atividades da Etapa 3

As atividades a seguir são respostas-modelo elaboradas a partir das entregas do enunciado Projeto 4 e dos cinco materiais de apoio. Elas não reproduzem um caderno oficial de questões, que não foi fornecido. Modelos e resultados informados pela equipe não equivalem a execução técnica comprovada.

### 4.5.1 Atividade 3.1 — Aplicar o Manifesto Ágil ao desenvolvimento

**Resposta-modelo**

Os quatro valores do Manifesto Ágil priorizam indivíduos e interações, software funcionando, colaboração com o cliente e resposta a mudanças, sem eliminar a importância de ferramentas, documentação, contratos e planejamento. No ConectaIA, a consequência prática é demonstrar incrementos utilizáveis, discutir a interface com usuários e revisar o backlog diante de problemas de compreensão ou acessibilidade.

| Valor enfatizado | Elemento complementar que mantém valor | Aplicação proposta no projeto |
|---|---|---|
| Indivíduos e interações | Processos e ferramentas | Reuniões curtas para revisar linguagem, fluxo e riscos |
| Software funcionando | Documentação abrangente | Demonstrar incrementos sem perder rastreabilidade e instruções |
| Colaboração com cliente | Negociação contratual | Obter feedback de usuários e alinhamento com docentes |
| Responder a mudanças | Seguir um plano | Ajustar backlog conforme testes de usabilidade |

O modelo Cynefin ajuda a classificar decisões: uma correção simples de rótulo pode ser tratada de forma direta, enquanto padrões de uso de pessoas com pouca familiaridade tecnológica exigem experimentação e feedback. Trata-se de interpretação metodológica, não evidência de cerimônias executadas.

### 4.5.2 Atividade 3.2 — Definir Scrum: papéis, eventos e artefatos

**Resposta-modelo**

A equipe pode organizar o trabalho por Sprints curtas, adotando Product Owner para ordenar o backlog, Scrum Master para facilitar o processo e Developers

para construir o incremento. Sprint Planning define o objetivo, Daily Scrum sincroniza o trabalho, Sprint Review verifica o incremento e Sprint Retrospective orienta melhorias. Esses papéis não foram formalmente atribuídos a integrantes específicos na documentação fornecida.

| Papel | Responsabilidade principal | Aplicação sugerida ao ConectaIA |
|---|---|---|
| Product Owner | Maximizar valor e ordenar backlog | Validar prioridades com usuário/docente |
| Scrum Master | Promover efetividade e remover impedimentos | Facilitar eventos, visibilidade de riscos e cooperação |
| Developers | Criar incremento conforme Definition of Done | Implementar, revisar, testar e documentar |
| Apoiadores (não papéis Scrum) | Contribuição especializada | UX, acessibilidade, privacidade e avaliação |

| Sprint proposta | Objetivo de incremento | Atividades e verificações | Saída esperada |
|---|---|---|---|
| Sprint A — base | Navegação e aprendizagem utilizáveis | RF02, RF05, RF06, RF08; critérios de leitura; documentação | Fluxos demonstráveis e registros de teste |
| Sprint B — interação | Conversa segura e validação do MVP | RF03, RF04, RF07, RF09; correção de segredo; testes | Versão demonstrável e evidências anexadas |

| Artefato | Compromisso | Definição para o projeto |
|---|---|---|
| Product Backlog | Product Goal | Apoiar aprendizagem segura e acessível sobre IA |
| Sprint Backlog | Sprint Goal | Objetivo mensurável do incremento de cada ciclo |
| Increment | Definition of Done | Critérios de aceite, testes e rastreabilidade satisfeitos |

Situação: planejamento de Scrum proposto; faltam atas, datas e evidências de Sprints efetivamente realizadas.

### 4.5.3 Atividade 3.3 — Criar quadro Kanban e definir métricas de fluxo

**Resposta-modelo**

Para visualizar as atividades, utilizar colunas “Backlog”, “A fazer”, “Em andamento”, “Em revisão”, “Em testes” e “Concluído”; configurar limites de WIP e mover os cartões de acordo com o trabalho real. As métricas recomendadas são lead time (solicitação até entrega), cycle time (início do trabalho até conclusão), throughput (quantidade concluída no período) e WIP (itens simultaneamente

iniciados). O quadro da figura é ilustrativo; seus estados não comprovam avanço real.

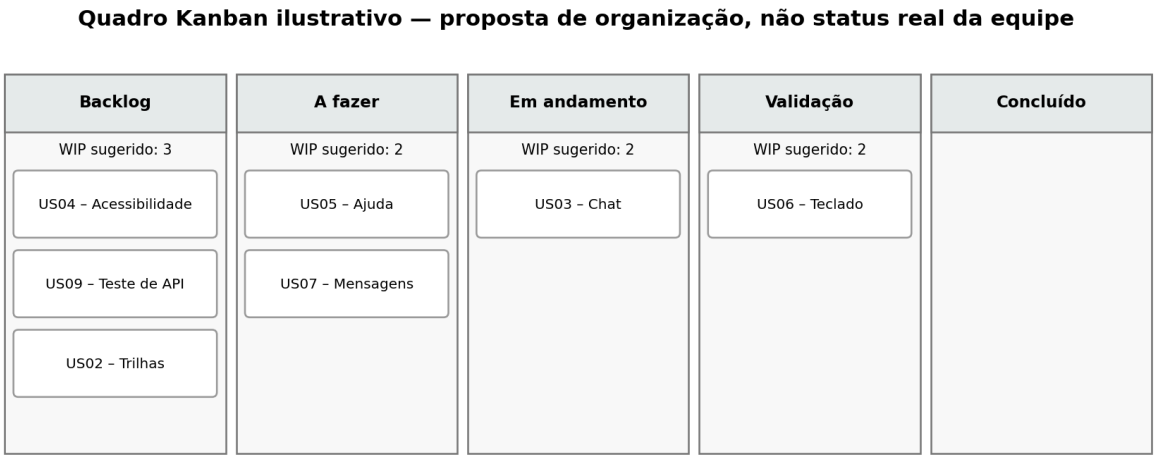



Figura RA4 — Quadro Kanban ilustrativo com limite de WIP; não representa execução histórica.

A Lei de Little ($L = \lambda \times W$) relaciona quantidade média de itens no sistema, vazão e tempo médio de permanência em condições estáveis. Somente usar métricas reais se houver registro de datas e quantidades; nenhuma medição de fluxo foi fornecida.

### 4.5.4 Atividade 3.4 — Construir backlog e planejar entregas

**Resposta-modelo**

O backlog foi organizado em histórias com valor para o usuário, priorização MoSCoW, estimativas preliminares e vínculo a requisitos/testes. O critério INVEST orienta histórias independentes, negociáveis, valiosas, estimáveis, pequenas e testáveis. Os story points apresentados são estimativas ilustrativas, não resultado de Planning Poker executado.

| ID | História de usuário (formato proposto) | MoSCoW | SP* | Rastreio |
|---|---|---|---|---|
| US01 | Como usuário, quero chegar facilmente ao Início para escolher por onde começar. | Must | 3 | RF01/RF02 |
| US02 | Como usuário, quero abrir trilhas simples para aprender progressivamente. | Must | 5 | RF05 |
| US03 | Como usuário, quero digitar uma pergunta e ver a resposta para praticar IA. | Must | 8 | RF03/RF04 |
| US04 | Como usuário com dificuldade de leitura, quero ajustar tamanho e contraste. | Must | 5 | RF07/RNF01/RNF03 |
| US05 | Como usuário iniciante, quero encontrar Ajuda/Socorro para recuperar orientação. | Must | 3 | RF08/RF10 |

| ID | História de usuário (formato proposto) | MoSCoW | SP* | Rastreio |
|---|---|---|---|---|
| US06 | Como usuário, quero navegar por teclado para utilizar o sistema sem mouse. | Must | 5 | RNF02 |
| US07 | Como usuário, quero mensagens claras quando uma requisição falhar. | Must | 5 | RNF07 |
| US08 | Como usuário, quero entender limitações da IA e cuidados com dados. | Should | 3 | RF06/RF09/RNF08 |
| US09 | Como mantenedor, quero manter a chave de API fora do navegador para evitar exposição. | Must | 8 | RNF06 |
| US10 | Como equipe, quero disponibilizar instruções de uso e arquitetura versionadas. | Should | 3 | DVCP/Docs |
| US11 | Como usuário, quero acionar microfone quando suportado para interagir por voz. | Could | 8 | Voz |
| US12 | Como equipe, quero registros auditáveis de testes para validar qualidade. | Must | 5 | QA |

Planejamento proposto: Sprint A foca navegação, aprendizado e orientação; Sprint B foca conversa, acessibilidade, correção de segurança e testes. Antes da execução, registrar responsáveis, datas, critérios de pronto e evidências anexadas ao backlog.

# 5 ETAPA 4 — DESENVOLVIMENTO COLABORATIVO E PROTÓTIPOS

## 5.1 Cooperação técnica em equipes multidisciplinares

A colaboração integra requisitos, UX, desenvolvimento, segurança, QA e comunicação. Para prevenir decisões dispersas, cada mudança deve vincular problema, requisito, responsável, implementação, revisão e teste. Ferramentas digitais podem organizar demandas, discussões e arquivos; não há evidência fornecida de qual plataforma de comunicação ou gestão foi efetivamente adotada. São sugeridos acordos de linguagem, definição de pronto, atas curtas, revisões de mudança e divisão explícita das responsabilidades.

| Atividade | Responsável sugerido (sem nome atribuído) | Evidência exigida |
|---|---|---|
| Refinar requisitos | Produto + UX | Requisito e aceite versionados |
| Implementar funcionalidade | Desenvolvimento | Commit e execução local |
| Revisar código e segurança | Segundo revisor técnico | Pull request e checklist |
| Testar acessibilidade | QA + usuário representativo | Roteiro e resultado de teste |
| Atualizar documentação | Autor do incremento | Arquivo revisado e referência cruzada |
| Priorizar produto | Product Owner sugerido | Backlog ordenado e decisão registrada |

## 5.2 Controle de versão distribuído com Git e GitHub

O material da Aula 4 descreve o ciclo Working Directory → Staging Area → repositório local → repositório remoto, além de estratégias Gitflow e Trunk-Based e revisão por Pull Request. Recomenda-se utilizar branches pequenas e commits semânticos (feat, fix, refactor, test), assegurando revisão e execução de verificações antes da integração. Como o histórico Git não foi fornecido, nenhum PR ou commit é declarado como realizado (FESP, 2026, Aula 4).

### 5.2.1 Estratégias de branching: Gitflow e Trunk-Based

Git permite rastrear alterações por commits, branches e merges; GitHub pode facilitar pull requests, code review e histórico de decisões. Gitflow separa ramos de desenvolvimento, funcionalidades, release e correção; Trunk-Based Development integra mudanças pequenas com frequência à linha principal, frequentemente com testes automáticos e feature flags. Para um protótipo acadêmico reduzido, recomenda-se branches curtas por história e integração frequente, sem declarar que

a equipe já adota essa política. O pacote anteriormente auditado não equivale a histórico Git completo.

### 5.2.2 Anatomia de Pull Request e Code Review

Uma revisão por pares deve informar objetivo, requisitos afetados, screenshots quando houver alteração visual, passos de teste, riscos e aprovação de outro integrante. Commits devem ter mensagens objetivas e não incluir segredos; arquivos .env e credenciais não podem aparecer no repositório público. Sugestão de política: proteger o ramo principal, exigir revisão, executar lint/typecheck/build em ambiente limpo e registrar evidências do critério de pronto. Nenhuma revisão ou workflow CI é apresentada como concluída sem log correspondente.

| Elemento da revisão | Critério proposto |
|---|---|
| Título do PR | Referência à US e descrição funcional |
| Contexto | Problema, requisito e tela afetada |
| Mudança | Componentes/serviços modificados e justificativa |
| Teste | Passos reproduzíveis e resultado anexado |
| Segurança | Nenhum segredo exposto; dependências revisadas |
| Aprovação | Revisor distinto do autor e registro de decisão |

## 5.3 Prototipação incremental e Produto Mínimo Viável (MVP)

Protótipo é uma representação verificável da solução com diferentes níveis de fidelidade; MVP delimita o menor conjunto capaz de entregar valor e gerar aprendizado. As capturas do ConectaIA apresentam Início, conversa, trilhas, explicação sobre IA, acessibilidade, ajuda e modal de identificação. O código auditado contempla seis áreas principais e integração de IA programada; isso sustenta a existência de implementação, mas não comprova todos os fluxos em produção. A fronteira do MVP proposta prioriza navegação acessível, conteúdo educativo, conversa com contingência, avisos de privacidade e registro de validação.

| Incremento | Funcionalidade | Status documental | Critério pendente |
|---|---|---|---|
| Base | Início, menu e identificação | Telas e código descritos | Executar teste completo em navegador |
| Aprendizagem | Trilhas e Sobre a IA | Capturas e quatro trilhas em código | Confirmar todos os caminhos e conteúdos |
| Interação | Conversa, sugestões e fallback | Código de envio e serviço Gemini inspecionados | Testar API real sem expor chave |
| Inclusão | Fonte, contraste, voz, Ajuda/Socorro | Controles e Web Speech API no código | Testar teclado, WCAG, navegadores e acessibilidade |
| Garantia | Testes e critérios de aceite | Relatos SUS/tarefas, roteiros e planos | Anexar originais e retestes técnicos |

## 5.4 Protótipo funcional e arquitetura de camadas

A Aula 4 distingue baixa fidelidade, média fidelidade, alta fidelidade e protótipo executável; sua arquitetura de referência separa apresentação, controle/APIs, negócio/serviços e acesso a dados/persistência. O ConectaIA apresenta UI em React/TypeScript e integração externa planejada, mas não há comprovação de serviço backend seguro nem persistência própria. Por isso, a arquitetura de quatro camadas da aula é uma recomendação de evolução, e não um retrato da implementação existente (FESP, 2026, Aula 4).

### 5.4.1 Arquitetura identificada e proposta de evolução

A auditoria estática do pacote ENTREGA CONECTA.IA.zip realizada na versão anterior identificou interface React/TypeScript em src/App.tsx, serviço de integração a IA em src/services/gemini.ts, estilos em src/styles.css e recursos de voz via Web Speech API dependentes do navegador. A estrutura documentada permite separar camada de apresentação (componentes e estado), serviço de IA/contingência (requisições e fallback) e infraestrutura externa (Google Gemini/navegador). Não foi verificado backend privado, banco de dados persistente ou autenticação de usuários.

| Camada | Componente/tecnologia identificado | Função e limitação |
|---|---|---|
| Apresentação | React / TypeScript, src/App.tsx | Estado das seis áreas, formulário, chat, trilhas e ajustes |
| Serviço de IA | src/services/gemini.ts | Streaming de texto e fallback; chamada real não homologada |
| Estilo e acesso | src/styles.css / navegador | Escala de fonte, contraste; testes WCAG pendentes |
| Recursos de voz | Web Speech API | Microfone e síntese de fala dependentes de permissão/suporte |
| Infraestrutura | Provedor externo Gemini | Depende de rede, credencial protegida e condições de uso |
| Serviço backend seguro | Proxy de API recomendado | Não consta implementado no pacote auditado |

### 5.4.2 Riscos técnicos e requisitos de correção

A inspeção anterior identificou chave de integração fixa no código-fonte e arquivo .env incluído no pacote. Uma variável com prefixo VITE_ destinada ao frontend não protege segredo, pois pode ser exposta no código compilado. Antes de publicar ou demonstrar a conexão real, deve-se revogar/rotacionar a chave, impedir sua distribuição, planejar backend/proxy seguro com autenticação e limites de uso e

revisar o envio de mensagens ao provedor. A verificação TypeScript tsc -b foi relatada como concluída sem erros; nova compilação Vite falhou em um ambiente de análise por ausência de dependência nativa Linux do Rollup, sem que isso demonstre defeito universal do produto.

### 5.4.3 Registro do protótipo e telas disponíveis

O Apêndice A preserva as capturas relevantes do relatório anterior. A organização visual observada utiliza navegação lateral, cartões e linguagem simples, com seis áreas principais e identificação inicial. Essas imagens são evidência de apresentação visual, não teste de interação. Os wireframes reconstruídos nas etapas anteriores de documentação também são preservados, mas não podem ser tratados como prova histórica de que precederam o desenvolvimento.

## 5.5 Respostas-modelo das atividades da Etapa 4

As atividades a seguir são respostas-modelo elaboradas a partir das entregas do enunciado Projeto 4 e dos cinco materiais de apoio. Elas não reproduzem um caderno oficial de questões, que não foi fornecido. Modelos e resultados informados pela equipe não equivalem a execução técnica comprovada.

### 5.5.1 Atividade 4.1 — Definir a organização colaborativa da equipe

**Resposta-modelo**

Uma divisão sugerida de responsabilidades contempla refinamento de requisitos, implementação, revisão de código, validação de acessibilidade e atualização dos documentos. Decisões relevantes devem ser registradas em issues, reuniões documentadas ou revisões; não é correto atribuir uma função a integrantes específicos sem confirmação da equipe.

| Atividade | Responsável sugerido (sem nome atribuído) | Evidência exigida |
|---|---|---|
| Refinar requisitos | Produto + UX | Requisito e aceite versionados |
| Implementar funcionalidade | Desenvolvimento | Commit e execução local |
| Revisar código e segurança | Segundo revisor técnico | Pull request e checklist |
| Testar acessibilidade | QA + usuário representativo | Roteiro e resultado de teste |
| Atualizar documentação | Autor do incremento | Arquivo revisado e referência cruzada |
| Priorizar produto | Product Owner sugerido | Backlog ordenado e decisão registrada |

Situação: modelo de colaboração documentado; não há atas ou registro de distribuição real das tarefas.

### 5.5.2 Atividade 4.2 — Descrever o fluxo de versionamento Git/GitHub

**Resposta-modelo**

Uma estratégia viável para o grupo é Trunk-Based Development com mudanças pequenas e pull requests curtos; Gitflow também é válido se houver necessidade de branches permanentes de desenvolvimento e preparação de releases. O fluxo recomendado é: criar issue associada a US, abrir branch de funcionalidade, implementar alteração pequena, executar verificações, gerar commit semântico, abrir pull request, obter revisão e mesclar somente após aceitação.

Exemplos de títulos de commits (somente modelos, não registros históricos): “feat: adicionar aviso de privacidade no chat”; “fix: corrigir foco no menu de navegação”; “test: validar tratamento de erro na resposta da IA”.

| Elemento da revisão | Critério proposto |
|---|---|
| Título do PR | Referência à US e descrição funcional |
| Contexto | Problema, requisito e tela afetada |
| Mudança | Componentes/serviços modificados e justificativa |
| Teste | Passos reproduzíveis e resultado anexado |
| Segurança | Nenhum segredo exposto; dependências revisadas |
| Aprovação | Revisor distinto do autor e registro de decisão |

Evidências necessárias: URL ou exportação do repositório; histórico real de commits; telas de pull requests; revisões e aprovações; correspondência com itens do backlog. Nada disso pode ser inferido apenas de uma cópia ZIP do projeto.

### 5.5.3 Atividade 4.3 — Delimitar o Produto Mínimo Viável (MVP)

**Resposta-modelo**

O MVP do ConectaIA é a menor versão que permite ao usuário iniciar uma sessão, compreender o objetivo do produto, acessar trilhas, formular perguntas e obter apoio básico, com mensagens claras sobre limites e privacidade. Recursos avançados devem ficar fora do primeiro incremento quando dependem de validação adicional ou ampliam risco e complexidade. A presença das telas no código não substitui teste funcional em navegador.

| Incremento | Funcionalidade | Status documental | Critério pendente |
|---|---|---|---|
| Base | Início, menu e identificação | Telas e código descritos | Executar teste completo em navegador |
| Aprendizagem | Trilhas e Sobre a IA | Capturas e quatro trilhas em código | Confirmar todos os caminhos e conteúdos |
| Interação | Conversa, sugestões e fallback | Código de envio e serviço Gemini inspecionados | Testar API real sem expor chave |
| Inclusão | Fonte, contraste, voz, Ajuda/Socorro | Controles e Web Speech API no código | Testar teclado, WCAG, navegadores e acessibilidade |
| Garantia | Testes e critérios de aceite | Relatos SUS/tarefas, roteiros e planos | Anexar originais e retestes técnicos |

### 5.5.4 Atividade 4.4 — Descrever arquitetura e protótipo funcional

**Resposta-modelo**

O código anteriormente analisado foi descrito como aplicação React/TypeScript, com lógica de navegação e interface em src/App.tsx, integração com Google Gemini em src/services/gemini.ts, estilos em src/styles.css e recursos de voz associados à Web Speech API. A auditoria documental relatou verificação TypeScript bem-sucedida, mas não homologação completa no navegador ou na API externa.

| Camada | Componente/tecnologia identificado | Função e limitação |
|---|---|---|
| Apresentação | React / TypeScript, src/App.tsx | Estado das seis áreas, formulário, chat, trilhas e ajustes |
| Serviço de IA | src/services/gemini.ts | Streaming de texto e fallback; chamada real não homologada |
| Estilo e acesso | src/styles.css / navegador | Escala de fonte, contraste; testes WCAG pendentes |
| Recursos de voz | Web Speech API | Microfone e síntese de fala dependentes de permissão/suporte |
| Infraestrutura | Provedor externo Gemini | Depende de rede, credencial protegida e condições de uso |
| Serviço backend seguro | Proxy de API recomendado | Não consta implementado no pacote auditado |

Foi identificado risco de segredo de API no frontend e arquivo .env no pacote anterior. A ação técnica indicada é rotacionar a chave, removê-la do código distribuído e usar mediação segura no backend antes de uma demonstração conectada. A implementação dessa correção não foi comprovada.

Situação: arquitetura e protótipo documentados; execução completa, correção de credencial e integração real pendentes de evidência.

# 6 ETAPA 5 — VALIDAÇÃO, QUALIDADE E APRESENTAÇÃO FINAL

## 6.1 Fundamentos de testes de software e garantia da qualidade

A pirâmide de testes da Aula 5 prioriza casos unitários na base, testes de integração em número intermediário e testes de interface ponta a ponta em menor quantidade. Para ConectaIA, incluem-se: validação das regras de navegação e conteúdo, comportamento da integração Gemini em caso de falha, teste de acessibilidade por teclado e casos exploratórios de uso com pessoas representativas. A existência de planos não demonstra sua execução (FESP, 2026, Aula 5).

### 6.1.1 Tipos de teste, caixa-preta e caixa-branca

Testes unitários verificam componentes isolados; testes funcionais exercitam comportamentos especificados; testes de aceitação verificam critérios de valor para usuários/stakeholders. Caixa-preta deriva casos de entradas e saídas sem depender de implementação; caixa-branca examina caminhos, ramificações, funções e tratamento de exceções. O controle de qualidade combina revisão estática, automação quando viável, testes manuais, acessibilidade e validação com pessoas representativas. A existência de plano não deve ser confundida com execução.

### 6.1.2 Plano de testes por requisito e registro de defeitos

| Caso | Requisito | Cenário | Procedimento | Resultado esperado | Estado |
|---|---|---|---|---|---|
| CT01 | RF01/RF02 | Identificação e retorno ao Início | Abrir, prosseguir e retornar | Início exibido sem erro | Pendente |
| CT02 | RF05 | Abrir as quatro trilhas | Selecionar cada trilha | Conteúdo certo e retorno | Pendente |
| CT03 | RF03/RF04 | Conversa por texto | Enviar entrada válida | Resposta exibida e identificada | Pendente |
| CT04 | RNF07 | Falha de API/rede | Indisponibilizar rede controladamente | Aviso claro e fallback inequívoco | Pendente |
| CT05 | RNF01 | Ampliação de fonte | Alterar escala e navegar | Sem sobreposição ou perda | Pendente |
| CT06 | RNF03 | Contraste | Inspecionar combinações | Conformidade com WCAG aplicável | Pendente |
| CT07 | RF08/RF10 | Ajuda/Socorro | Seguir caminho de recuperação | Orientação sem beco sem saída | Pendente |
| CT08 | RNF02 | Teclado e foco | Tab, Shift+Tab, Enter, Space | Alcance e foco visível | Pendente |
| CT09 | RF09/RNF08 | Transparência e privacidade | Inspecionar avisos e | Limites comunicados, | Pendente |

| Caso | Requisito | Cenário | Procedimento | Resultado esperado | Estado |
|---|---|---|---|---|---|
| | | | entradas | sem indução | |
| CT10 | RNF06 | Segredo de API | Inspecionar bundle e fonte | Credencial não exposta ao cliente | Falha identificada na revisão anterior |

O ciclo de vida de um bug compreende identificação, registro com evidências e reprodução, triagem, classificação de prioridade/severidade, atribuição, correção, revisão, reteste e encerramento (ou reabertura). Deve-se distinguir defeito confirmado de suspeita, bloqueio de ambiente e requisito não validado. Exemplo real de risco técnico identificado: segredo de API no código, que exige correção e reteste antes de ser marcado como resolvido.

| Campo do defeito | Preenchimento proposto |
|---|---|
| ID e título | BUG-SEC-01 — Chave de API incluída no frontend |
| Origem | Auditoria documental do pacote anterior, Apêndice H |
| Severidade | Alta (risco de exposição e uso indevido) |
| Ação | Rotacionar credencial, retirar do bundle e criar mediação segura |
| Responsável e prazo | Não atribuídos / a definir pela equipe |
| Status | Correção não comprovada no material atual |
| Reteste e evidência | Inspeção de bundle, segredo ausente no client e requisição funcionando via backend — pendente |

## 6.2 Validação da solução com usuários e avaliação de usabilidade

O relatório de origem informa três entrevistas qualitativas iniciais e, posteriormente, avaliação de usabilidade com os mesmos participantes P1, P2 e P3. A equipe comunicou conclusão de seis tarefas por pessoa (18/18), escore SUS final 100/100 para cada participante, 90 segundos por tarefa e zero erros observados. Não foram anexados questionários SUS com as dez respostas individuais, fichas cronometradas, solicitações de ajuda, gravações nem resultados de auditoria independente; por isso, os valores permanecem identificados como relatos da equipe e não como resultados tecnicamente verificados. Com n = 3 não se infere representatividade populacional.

| Tarefa | Resultado comunicado pela equipe | Situação documental |
|---|---|---|
| T1 — Acesso inicial | Os usuários conseguiram entrar e acessar a página principal sem dificuldade. | Concluída, segundo relato coletivo. |
| T2 — Conversar com a IA | Os usuários relataram conseguir formular e enviar perguntas e interagir na área de conversa sem problemas. | Concluída; integração de IA externa não verificada. |
| T3 — Trilhas de Sucesso | Os usuários acessaram a seção Trilhas de Sucesso. | Concluída, segundo relato coletivo. |
| T4 — Sobre a IA | Os usuários localizaram a seção | Concluída, segundo relato |

| Tarefa | Resultado comunicado pela equipe | Situação documental |
|---|---|---|
| | Sobre a IA sem dificuldade, atribuindo a facilidade ao direcionamento da navegação. | coletivo. |
| T5 — Acessibilidade | Os ajustes de tamanho de fonte e contraste foram descritos como simples e sem complicações. | Concluída; conformidade WCAG não comprovada. |
| T6 — Ajuda/Socorro | A localização e o uso de Ajuda/Socorro foram relatados como funcionando adequadamente. | Concluída; funcionamento técnico não auditado. |

| Indicador | Resultado disponível | Natureza da evidência | Limitação |
|---|---|---|---|
| SUS | P1 = 100; P2 = 100; P3 = 100; média 100/100 | Informação comunicada pela equipe | Itens individuais não anexados |
| Conclusão de tarefas | 18/18 execuções relatadas = 100% | Síntese agregada da equipe | Fichas individuais não anexadas |
| Tempo | 90 segundos por tarefa, segundo relato | Informação declarada | Sem cronometragem original |
| Erros | 0 em 18 execuções, segundo relato | Informação declarada | Sem observação auditável |
| Pedidos de ajuda | Não quantificados | Sem dado de origem | Não considerar zero |
| Conformidade WCAG 2.2 | Não comprovada | Checklist e recomendações | Exige execução formal |

O SUS utiliza dez itens de 1 a 5; para itens ímpares subtrai-se 1 da resposta, e para pares subtrai-se a resposta de 5; a soma das contribuições é multiplicada por 2,5. O resultado deve ser reconstituído a partir dos formulários, quando disponíveis. A validação futura deve especificar dispositivo, navegador, roteiro, consentimento, tarefa, condição de sucesso, tempo, erros, ajuda e observações, mantendo registros anonimizados.

### 6.2.1 Critérios de aceitação revisados e adaptação por feedback

Feedback qualitativo é insumo para o backlog e não comprovação de requisito atendido. Os participantes relataram valorização da facilidade de acesso, da utilidade cotidiana e da aprendizagem. P2 comunicou redução de dificuldades e dúvidas anteriores. Recomenda-se revisar critérios ligados à formulação de perguntas, ajuste visual e aviso de segurança, conduzir rodada documentada após mudanças e confrontar resultados com os critérios da versão anterior, sem fabricar diferenças quantitativas.

## 6.3 Retrospectiva, Kaizen e melhoria contínua

Kaizen corresponde à melhoria contínua com alterações pequenas, observáveis e incorporadas ao processo. Uma retrospectiva deve discutir o que favoreceu o progresso, onde ocorreram bloqueios e que ações mensuráveis serão

adotadas. Como não há ata de retrospectiva fornecida, apresenta-se o roteiro proposto, não lições aprendidas atribuídas à equipe.

A dinâmica Starfish da Aula 5 propõe cinco dimensões para a retrospectiva: começar a fazer, parar de fazer, fazer mais, fazer menos e continuar fazendo. O roteiro do ConectaIA deve registrar decisões e responsáveis, distinguindo sugestões metodológicas de atas reais. Não há registro recebido que permita afirmar a execução dessa dinâmica pela equipe (FESP, 2026, Aula 5).

| Pergunta de retrospectiva | Evidência a coletar | Ação proposta (não executada) |
|---|---|---|
| O que funcionou? | Feedback, teste reprodutível e entregável aceito | Padronizar boas práticas verificadas |
| O que prejudicou? | Impedimentos e inconsistências documentais | Alinhar README, relatório e implementação |
| Que risco precisa ser removido? | Credenciais, permissões e privacidade | Proteger segredo e reduzir dependência insegura |
| Que melhoria trará valor? | Dados de participantes e testes WCAG | Refinar navegação e acessibilidade por evidência |
| Como medir? | Tarefas, erros, duração, solicitações de ajuda | Repetir roteiro e comparar versões de forma controlada |

## 6.4 Apresentação técnica, defesa do projeto e entrega final

A apresentação técnica deverá responder: qual problema real está sendo resolvido; quem são os usuários; quais requisitos e riscos orientaram a proposta; como o processo de desenvolvimento foi planejado; quais diagramas representam o sistema; como backlog e fluxo serão geridos; quais funcionalidades são demonstráveis; quais testes foram de fato executados; e o que permanece pendente. Sugere-se uma demonstração curta e controlada: início → trilha → conversa (com serviço externo somente após correção do segredo) → acessibilidade → Ajuda/Socorro → resultados/limitações.

### 6.4.1 Matriz de rastreabilidade e auditoria da entrega

| Entregável obrigatório | Localização nesta versão | Cobertura documental | Comprovação factual |
|---|---|---|---|
| Definição do problema e contexto | Introdução / DVCP | Atendido | Entrevistas relatadas |
| Requisitos e modelagem BPMN/UML | Etapa 2, tabelas e Figuras 1–3 | Atendido como modelagem proposta | Homologação dos diagramas pendente |
| Backlog e planejamento Scrum/Kanban | Etapa 3, Figuras 4 e Gráfico 1 | Atendido como proposta | Sprints e métricas reais não demonstradas |
| Protótipo funcional | Etapa 4 / Apêndice A | Código e capturas descritos | Execução e API real não homologadas |
| Registro de testes/validações | Etapa 5 / Apêndice B | Relatos e plano disponíveis | Formulários e testes técnicos pendentes |
| Melhoria contínua e colaboração | Etapas 4 e 5 | Método e modelos definidos | Registros de reuniões/retrospectiva ausentes |
| Textos didáticos e | Etapas 1–5 (itens | Cinco aulas integrais (54 | Caderno oficial de |

| Entregável obrigatório | Localização nesta versão | Cobertura documental | Comprovação factual |
|---|---|---|---|
| exercícios integrais | 2.5/3.6/4.5/5.5/6.5) / Anexos A–E | páginas) e 20 respostas-modelo integradas | exercícios não fornecido; correspondência literal não verificável |

### 6.4.2 Rubrica integrada e limites de avaliação

A Aula 5 fornece uma rubrica de avaliação integrada da FESP, reproduzida integralmente no Anexo E e apresentada em quadro sintético abaixo. As quatro dimensões da rubrica são: domínio do problema e requisitos; modelagem e arquitetura; execução ágil e versionamento; qualidade técnica e protótipo. As faixas são Insuficiente (0,0–4,9), Regular (5,0–6,9), Bom (7,0–8,9) e Excelente (9,0–10,0). A classificação “Excelente” exige evidências, e não apenas formulação documental. Não se atribui nota ao ConectaIA sem comprovação das atividades práticas (FESP, 2026, Aula 5).

A Figura abaixo reproduz o quadro do professor sem alterações, incluindo todas as quatro faixas de pontuação. O quadro editável imediatamente a seguir resume apenas o patamar “Excelente” e sua correspondência com as evidências atualmente disponíveis.

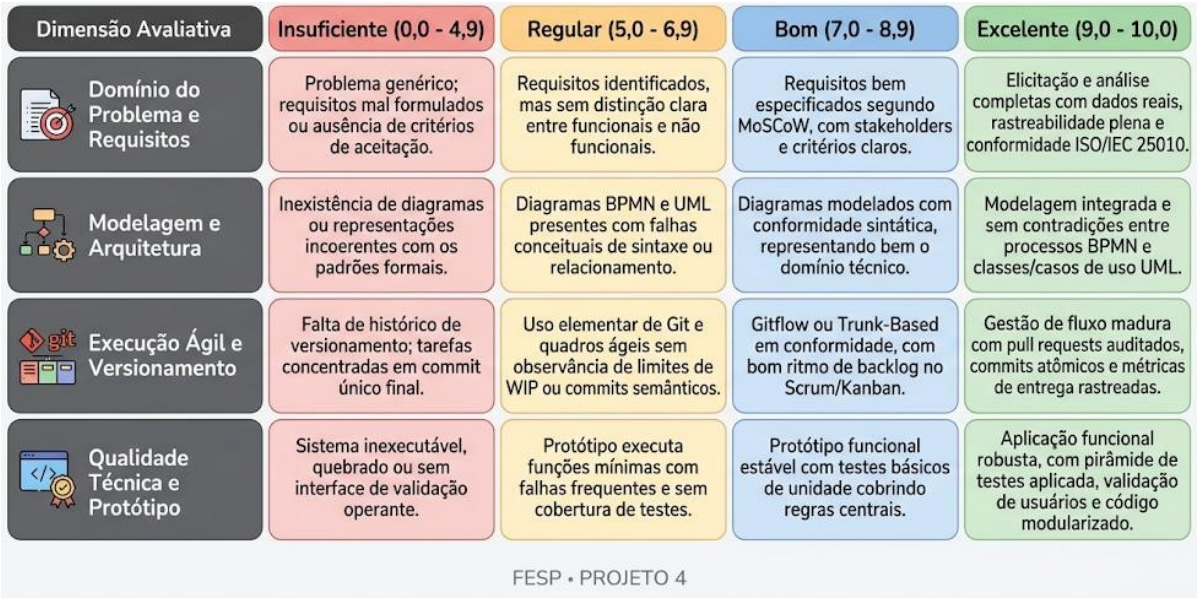



Figura — Rubrica de avaliação integrada da Aula 5. Fonte: FESP (2026), Aula 5, p. 4; reprodução da fonte original.

| Dimensão | Condição “Excelente” da Aula 5 | Condição comprovável / lacuna do ConectaIA |
|---|---|---|
| Domínio do problema e requisitos | Elicitação e análise completas com dados reais, rastreabilidade plena e conformidade ISO/IEC 25010. | Requisitos e relatos de três usuários documentados; comprovar homologação e rastreabilidade ponta a ponta. |
| Modelagem e arquitetura | Modelagem integrada e sem contradições entre processos BPMN e classes/casos de | Diagramas conceituais disponíveis; exigir revisão técnica e coerência com a |

| Dimensão | Condição “Excelente” da Aula 5 | Condição comprovável / lacuna do ConectaIA |
|---|---|---|
| | uso UML. | implementação. |
| Execução ágil e versionamento | Gestão de fluxo madura, com pull requests auditados, commits atômicos e métricas de entrega rastreadas. | Backlog e Kanban são propostas; histórico Git, PRs e métricas efetivas não foram enviados. |
| Qualidade técnica e protótipo | Aplicação funcional robusta, com pirâmide de testes aplicada, validação de usuários e código modularizado. | Inspeção estática e relatos SUS disponíveis; faltam testes técnicos comprovados e execução em navegador. |

## 6.5 Respostas-modelo das atividades da Etapa 5

As atividades a seguir são respostas-modelo elaboradas a partir das entregas do enunciado Projeto 4 e dos cinco materiais de apoio. Elas não reproduzem um caderno oficial de questões, que não foi fornecido. Modelos e resultados informados pela equipe não equivalem a execução técnica comprovada.

### 6.5.1 Atividade 5.1 — Elaborar e executar um plano de testes

**Resposta-modelo**

A pirâmide de testes da Aula 5 distingue testes unitários (comportamentos isolados), testes de integração (interação entre módulos/serviços) e testes de interface ou ponta a ponta (fluxo real do usuário). A técnica caixa-preta confronta entrada e saída com critérios de aceite; a técnica caixa-branca investiga caminhos e estruturas internas. Para cada execução devem constar data, versão, ambiente, passos, resultado esperado e resultado observado.

| Caso | Requisito | Cenário | Resultado esperado | Estado |
|---|---|---|---|---|
| CT01 | RF01/ RF02 | Identificação e retorno ao Início | Início exibido sem erro | Pendente |
| CT02 | RF05 | Abrir as quatro trilhas | Conteúdo certo e retorno | Pendente |
| CT03 | RF03/ RF04 | Conversa por texto | Resposta exibida e identificada | Pendente |
| CT04 | RNF07 | Falha de API/rede | Aviso claro e fallback inequívoco | Pendente |
| CT05 | RNF01 | Ampliação de fonte | Sem sobreposição ou perda | Pendente |
| CT06 | RNF03 | Contraste | Conformidade com WCAG aplicável | Pendente |
| CT07 | RF08/ RF10 | Ajuda/Socorro | Orientação sem beco sem saída | Pendente |
| CT08 | RNF02 | Teclado e foco | Alcance e foco visível | Pendente |
| CT09 | RF09/ RNF08 | Transparência e privacidade | Limites comunicados, sem indução | Pendente |
| CT10 | RNF06 | Segredo de API | Credencial não exposta ao cliente | Falha identificada na revisão anterior |

Situação: plano de testes existente. CT01–CT09 permanecem pendentes. CT10 registra uma falha observada em inspeção anterior, mas a execução de reteste e correção não está comprovada. Não preencher casos pendentes como aprovados sem execução.

### 6.5.2 Atividade 5.2 — Registrar validação com usuários e interpretar o SUS

**Resposta-modelo**

A avaliação relatada pela equipe envolveu três participantes (P1, P2 e P3), que teriam concluído seis tarefas cada, somando 18/18 execuções. A equipe informou escore SUS de 100/100 por participante e média amostral 100/100. Esses valores são dados comunicados pelo grupo e não foram comprovados por formulários SUS originais nem fichas individuais. Não permitem generalização populacional nem substituem testes técnicos.

| Indicador | Resultado disponível | Natureza da evidência |
|---|---|---|
| SUS | P1 = 100; P2 = 100; P3 = 100; média 100/100 | Informação comunicada pela equipe |
| Conclusão de tarefas | 18/18 execuções relatadas = 100% | Síntese agregada da equipe |
| Tempo | 90 segundos por tarefa, segundo relato | Informação declarada |
| Erros | 0 em 18 execuções, segundo relato | Informação declarada |
| Pedidos de ajuda | Não quantificados | Sem dado de origem |
| Conformidade WCAG 2.2 | Não comprovada | Checklist e recomendações |

Para tornar a resposta auditável, reunir os formulários SUS das dez questões, as fichas de execução por tarefa e participante, registros de tempo/erros/pedidos de ajuda e eventuais comentários. Quando indisponíveis, identificar a limitação e refazer uma avaliação controlada com consentimento.

### 6.5.3 Atividade 5.3 — Realizar retrospectiva e propor melhoria contínua

**Resposta-modelo — roteiro a aplicar**

A retrospectiva deve distinguir inspeção do processo de avaliação do produto. O método Starfish utilizado na Aula 5 permite classificar decisões em Começar, Parar, Fazer Mais, Fazer Menos e Continuar. A tabela a seguir oferece ações sugeridas ao ConectaIA, que NÃO representam retrospectiva já realizada:

| Categoria Starfish | Ação proposta para a próxima iteração | Evidência esperada |
|---|---|---|
| Começar | Registrar testes unitários, de integração e E2E | Relatório de execução e commits |
| Parar | Distribuir arquivos com chave de API embutida | Diff de correção e validação do bundle |
| Fazer mais | Validar leitura, navegação e linguagem com usuários | Fichas de testes e backlog refinado |
| Fazer menos | Aceitar afirmações de funcionamento baseadas apenas em capturas | Critérios objetivos de aceite |
| Continuar | Manter linguagem simples e trilhas progressivas | Feedback documentado e novas versões |

Situação: plano de retrospectiva pronto. Para considerar a atividade executada, adicionar data, participantes, comentários reais e decisão aprovada.

### 6.5.4 Atividade 5.4 — Estruturar a defesa e auditar a entrega

**Resposta-modelo**

A apresentação do Projeto 4 deve seguir uma sequência técnica que conecte o problema ao resultado: 1) problema real e público; 2) pesquisa e requisitos; 3) BPMN/UML e arquitetura; 4) backlog, Scrum/Kanban e colaboração; 5) demonstração do protótipo; 6) testes e evidências; 7) limitações, segurança, retrospectiva e próximos incrementos.

| Entregável obrigatório | Cobertura documental | Comprovação factual |
|---|---|---|
| Definição do problema e contexto | Atendido | Entrevistas relatadas |
| Requisitos e modelagem BPMN/UML | Atendido como modelagem proposta | Homologação dos diagramas pendente |
| Backlog e planejamento Scrum/Kanban | Atendido como proposta | Sprints e métricas reais não demonstradas |
| Protótipo funcional | Código e capturas descritos | Execução e API real não homologadas |
| Registro de testes/validações | Relatos e plano disponíveis | Formulários e testes técnicos pendentes |
| Melhoria contínua e colaboração | Método e modelos definidos | Registros de reuniões/retrospectiva ausentes |
| Textos didáticos e exercícios integrais | Cinco aulas completas preservadas como fac-símiles (54 páginas) | Respostas originais dos exercícios não fornecidas |

A rubrica da Aula 5 define a faixa Excelente (9,0–10,0) por requisitos completos e rastreáveis, BPMN/UML coerentes, Git/PR e métricas reais, produto

robusto com pirâmide de testes aplicada e validação com usuários. A documentação, isoladamente, não é suficiente para comprovar esse nível.

## 7 MATRIZ DE ADERÊNCIA E PENDÊNCIAS

Esta versão integra cinco etapas do Projeto 4, relacionando pesquisa e protótipo ConectaIA com requisitos, diagramas, planejamento ágil, testes e 20 atividades-modelo explicitamente respondidas e organizadas em cada etapa. Os cinco Textos de Apoio Didáticos permanecem preservados integralmente nas 54 páginas fac-similares dos Anexos A–E. Não foi enviado o arquivo original com enunciados ou respostas de exercícios; portanto, a correspondência literal com esse material ainda não pode ser atestada. A conformidade prática continua condicionada a histórico Git/GitHub, demonstração executável em navegador, testes, fichas de validação e correção do risco de credencial de API.

| Requisito/ação | Condição nesta versão | Pendência para entrega verificável |
|---|---|---|
| Capa, folha de rosto, sumário, introdução, conclusão, referências | Organizados | Confirmar dados formais da disciplina e identificação da atividade |
| Etapa 1 — fundamentos, ciclos, ISO, V&V, DVCP | Conteúdo e matriz produzidos | Texto didático completo reproduzido no Anexo A; comprovar escolhas da equipe |
| Etapa 2 — requisitos, priorização, BPMN, UML, rastreabilidade | Artefatos técnicos propostos | Homologar com equipe e revisar casos de uso |
| Etapa 3 — Agile, Scrum, Kanban, Cynefin, backlog | Conteúdos, quadro e proposta gerados | Registrar execução real e acordos |
| Etapa 4 — Git, colaboração, MVP e arquitetura | Fundamentação + auditoria de código preservada | Obter histórico Git e validar sistema em navegador |
| Etapa 5 — testes, validação e retrospectiva | Plano + relatos disponíveis | Anexar fichas, retestes e ata de retrospectiva |
| Segurança e credenciais | Risco identificado e ação especificada | Rotacionar/retirar chave do frontend, revalidar |
| Textos das cinco aulas e respostas dos exercícios | Cinco aulas integrais (54 páginas) e 20 respostas-modelo nas etapas | Cotejar com exercícios originais, se fornecidos, e comprovar a execução prática |

## 8 CONSIDERAÇÕES FINAIS

A adequação do ConectaIA ao Projeto 4 converteu a documentação preexistente de IHC/UX em uma proposta organizada de Engenharia de Software, articulando cinco etapas: fundamentos e qualidade; requisitos e modelagem; metodologias ágeis e planejamento; desenvolvimento colaborativo e prototipação; e validação e melhoria contínua. A proposta preserva o problema de inclusão digital, os requisitos identificados, as características do protótipo e os dados comunicados pela equipe, acrescentando modelos BPMN/UML, backlog, matrizes de qualidade, critérios de aceitação e plano de evolução.

A distinção entre modelos propostos e evidências efetivamente obtidas evita apresentar como concluídas atividades que não foram demonstradas. A disponibilização dos cinco materiais didáticos permitiu consolidar referências e preservar os textos originais. As vinte respostas-modelo das atividades estão incorporadas ao corpo das cinco etapas, com correspondência aos assuntos do prompt. Caso exista um caderno oficial de exercícios, deverá ser cotejado com estas respostas. Permanecem necessárias comprovações experimentais e colaborativas: navegação e integração da aplicação, segurança da chave de API, testes executados, dados SUS originais, retrospectiva e rastreabilidade de commits e revisões.

# REFERÊNCIAS

ANDERSON, David J. Kanban: successful evolutionary change for your technology business. Sequim: Blue Hole Press, 2010.

BECK, Kent et al. Manifesto para Desenvolvimento Ágil de Software. 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html. Acesso em: 4 nov. 2026.

DESIGN COUNCIL. The Double Diamond. London: Design Council, 2019. Disponível em: https://www.designcouncil.org.uk/resources/the-double-diamond/. Acesso em: 1 nov. 2026.

FOWLER, Martin. UML distilled: a brief guide to the standard object modeling language. 3. ed. Boston: Addison-Wesley Professional, 2003.

FUNDAÇÃO DE ESTUDOS SOCIAIS DO PARANÁ (FESP). Aula 2: engenharia de requisitos e modelagem inicial de sistemas. Curitiba: FESP, 2026. Texto de apoio didático.

FUNDAÇÃO DE ESTUDOS SOCIAIS DO PARANÁ (FESP). Aula 3: metodologias ágeis, engenharia de fluxo e planejamento. Curitiba: FESP, 2026. Texto de apoio didático.

FUNDAÇÃO DE ESTUDOS SOCIAIS DO PARANÁ (FESP). Aula 4: desenvolvimento colaborativo, controle de versão e prototipação. Curitiba: FESP, 2026. Texto de apoio didático.

FUNDAÇÃO DE ESTUDOS SOCIAIS DO PARANÁ (FESP). Aula 5: validação, garantia da qualidade e apresentação técnica. Curitiba: FESP, 2026. Texto de apoio didático.

GUERRA, André Roberto. Aula 1: fundamentos da Engenharia de Software, ciclos de vida e qualidade. Curitiba: Fundação de Estudos Sociais do Paraná (FESP), 2026. Material didático da disciplina Engenharia de Software.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO); INTERNATIONAL ELECTROTECHNICAL COMMISSION (IEC). ISO/IEC 25010:2011: Systems and software engineering — System and software quality models. Geneva: ISO, 2011. Disponível em: https://www.iso.org/standard/35733.html. Acesso em: 4 nov. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO); INTERNATIONAL ELECTROTECHNICAL COMMISSION (IEC). ISO/IEC 25010:2023: Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: ISO, 2023. Disponível em: https://www.iso.org/standard/78176.html. Acesso em: 4 nov. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 9241-11:2018: Ergonomics of human-system interaction — Usability: definitions and concepts.

Geneva: ISO, 2018. Disponível em: https://www.iso.org/standard/63500.html. Acesso em: 1 nov. 2026.

LITTLE, John D. C. A proof for the queuing formula L = λW. Operations Research, v. 9, n. 3, p. 383–387, 1961.

NIELSEN, Jakob. 10 usability heuristics for user interface design. Nielsen Norman Group, 1994. Rev. 30 jan. 2024. Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics/. Acesso em: 1 nov. 2026.

NORMAN, Donald A. The design of everyday things: revised and expanded edition. New York: Basic Books, 2013.

OBJECT MANAGEMENT GROUP. Business Process Model and Notation (BPMN), version 2.0.2. 2014. Disponível em: https://www.omg.org/spec/BPMN/2.0.2. Acesso em: 4 nov. 2026.

OBJECT MANAGEMENT GROUP. Unified Modeling Language (UML), version 2.5.1. 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: 4 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Software engineering: a practitioner’s approach. 9. ed. New York: McGraw-Hill Education, 2020.

RODDEN, Kerry; HUTCHINSON, Hilary; FU, Xin. Measuring the user experience on a large scale: user-centered metrics for web applications. In: CHI 2010 — Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. New York: ACM, 2010. p. 2395-2398. DOI: 10.1145/1753326.1753687.

ROGERS, Yvonne; SHARP, Helen; PREECE, Jenny. Interaction design: beyond human-computer interaction. 5. ed. Indianapolis: Wiley, 2019.

SCHWABER, Ken; SUTHERLAND, Jeff. The Scrum Guide: the definitive guide to Scrum — the rules of the game. 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 4 nov. 2026.

SNOWDEN, David J.; BOONE, Mary E. A leader’s framework for decision making. Harvard Business Review, v. 85, n. 11, p. 68–76, 2007.

SOMMERVILLE, Ian. Software engineering. 10. ed. Boston: Pearson, 2016.

WAKE, Bill. INVEST in good stories, and SMART tasks. XP123, 2003. Disponível em: https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/. Acesso em: 4 nov. 2026.

WORLD WIDE WEB CONSORTIUM (W3C). Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 5 out. 2023. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 1 nov. 2026.

# APÊNDICE A — EVIDÊNCIAS E IMAGENS DA VERSÃO ANTERIOR

As 21 imagens seguintes são reproduções do relatório anterior: 1 sitemap, 12 wireframes reconstruídos e 8 capturas de telas. Não evidenciam execução em navegador nem a ordem histórica de construção. Os números de figuras originais foram preservados apenas descritivamente.

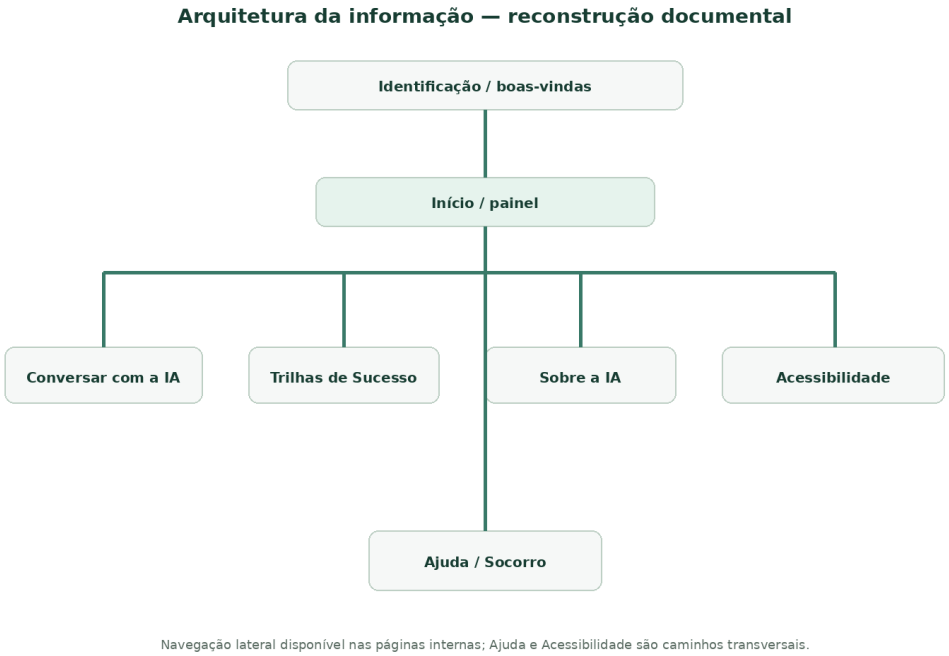



Figura A1 — Sitemap documental preexistente do ConectaIA; reconstrução do relatório UX/IHC.

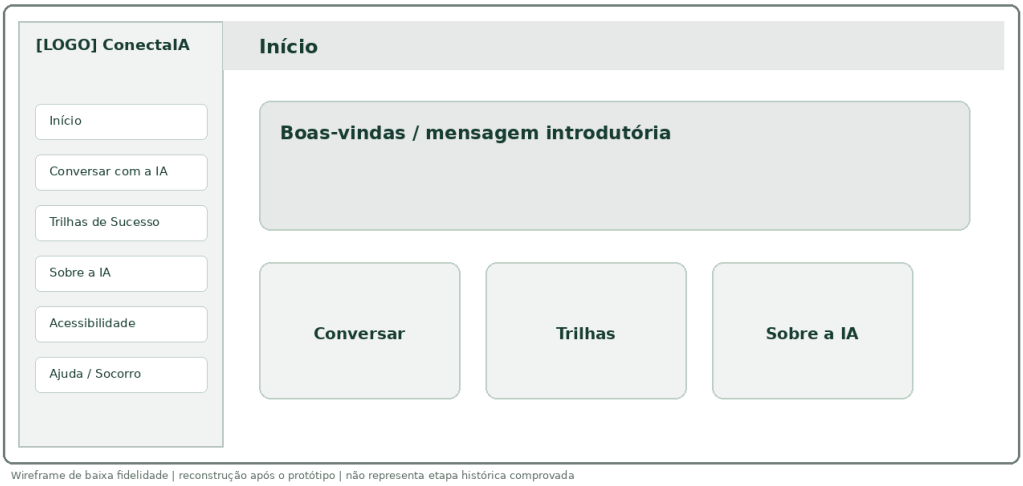



Figura A2 — Início: wireframe de baixa fidelidade (reconstruído na versão anterior).

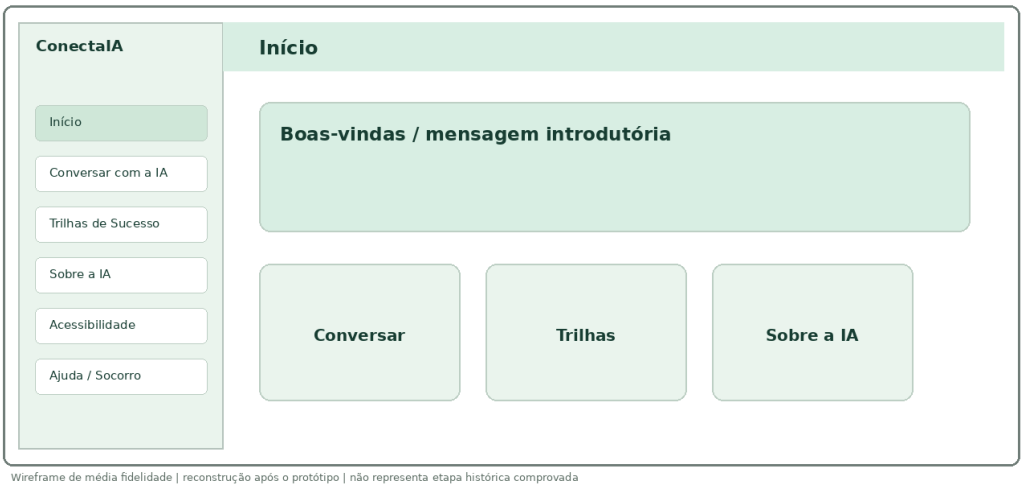



Figura A3 — Início: wireframe de média fidelidade (reconstruído na versão anterior).

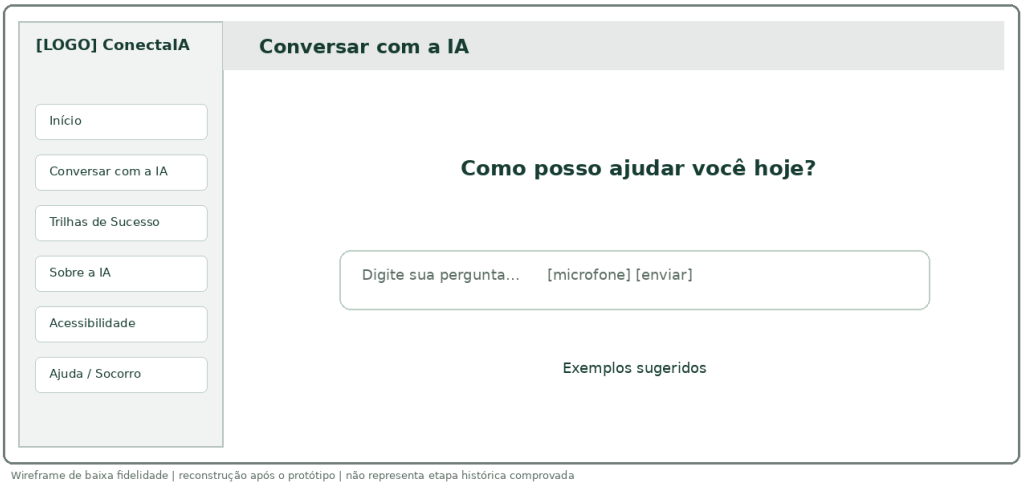



Figura A4 — Conversar com a IA: wireframe de baixa fidelidade (reconstruído na versão anterior).

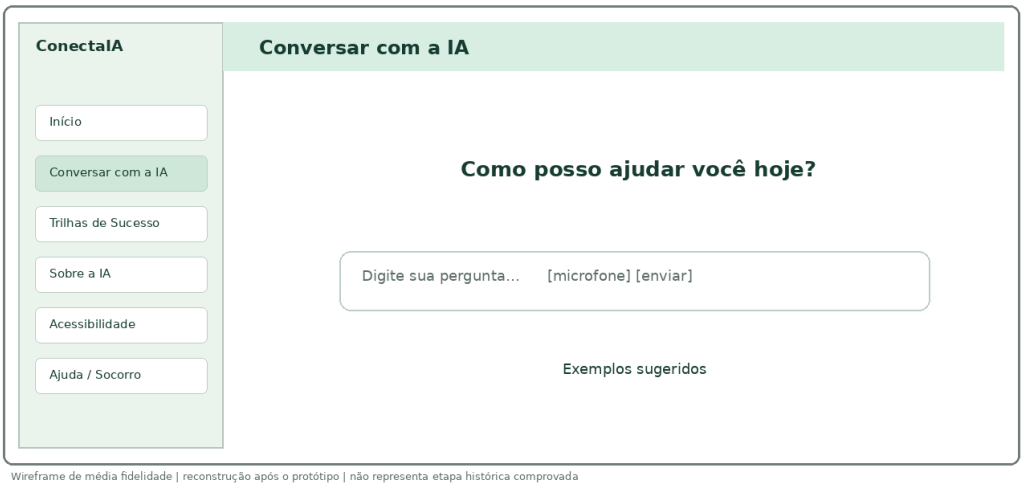



Figura A5 — Conversar com a IA: wireframe de média fidelidade (reconstruído na versão anterior).

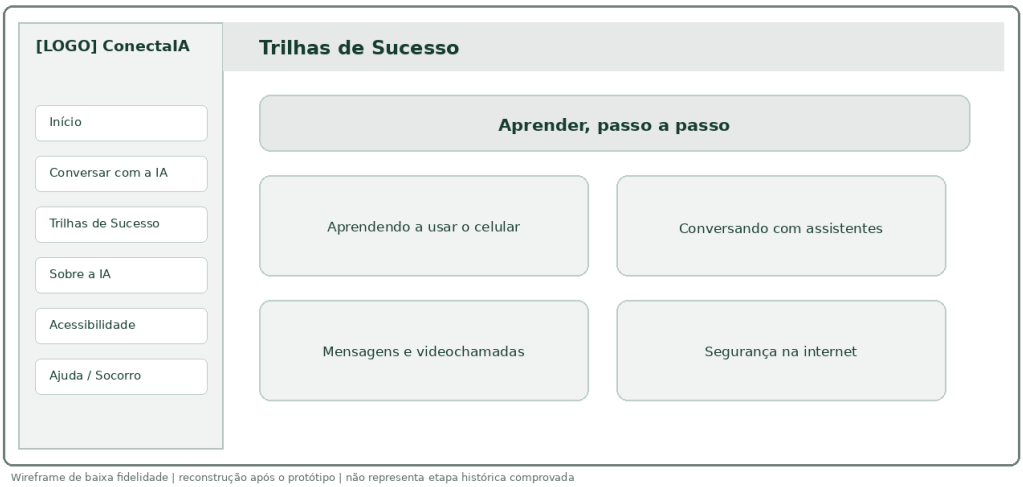



Figura A6 — Trilhas de Sucesso: wireframe de baixa fidelidade (reconstruído na versão anterior).

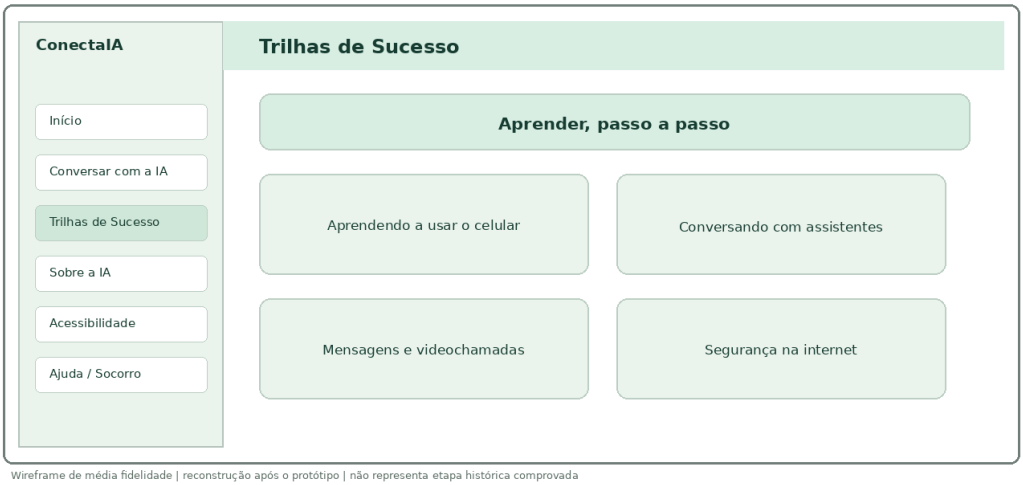



Figura A7 — Trilhas de Sucesso: wireframe de média fidelidade (reconstruído na versão anterior).

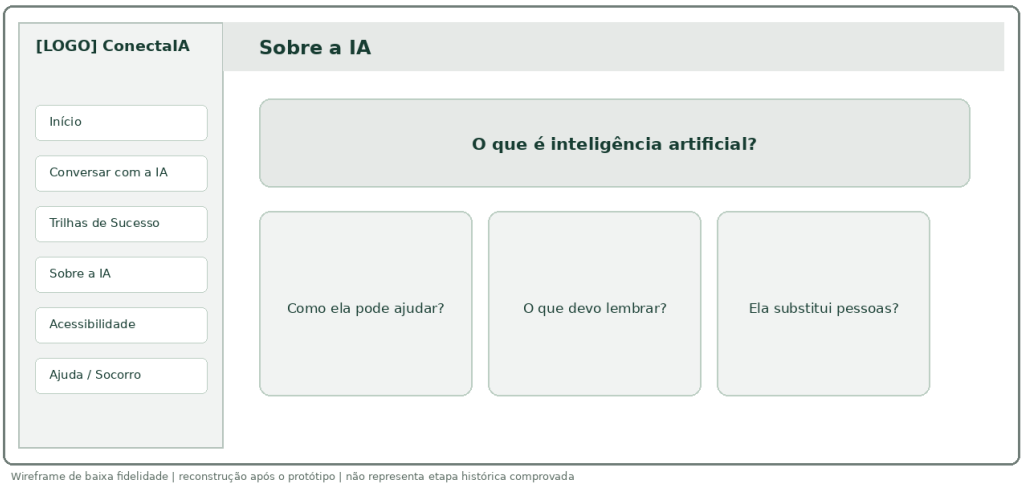



Figura A8 — Sobre a IA: wireframe de baixa fidelidade (reconstruído na versão anterior).

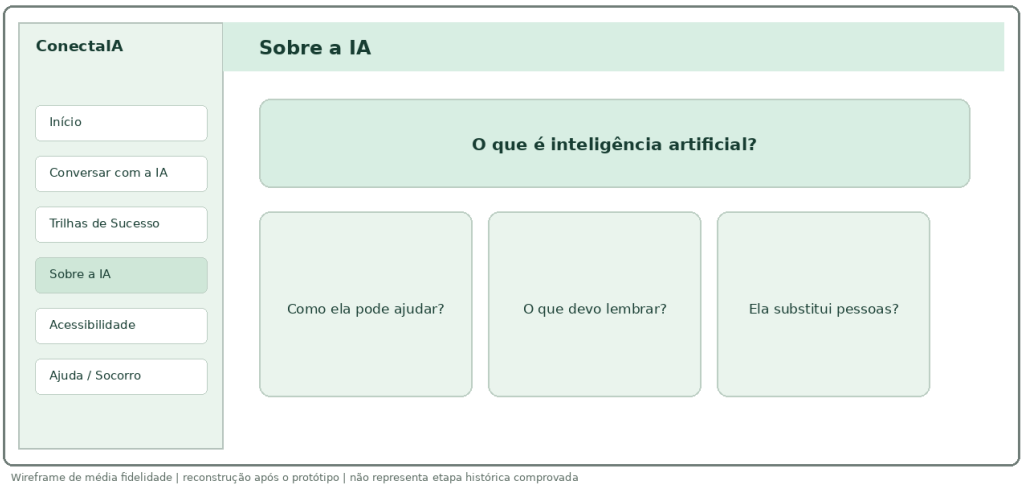



Figura A9 — Sobre a IA: wireframe de média fidelidade (reconstruído na versão anterior).

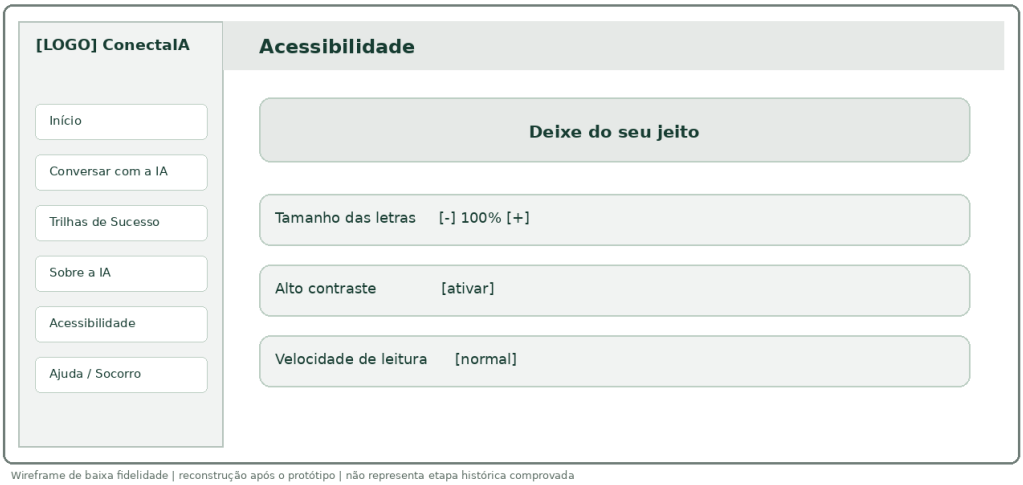



Figura A10 — Acessibilidade: wireframe de baixa fidelidade (reconstruído na versão anterior).

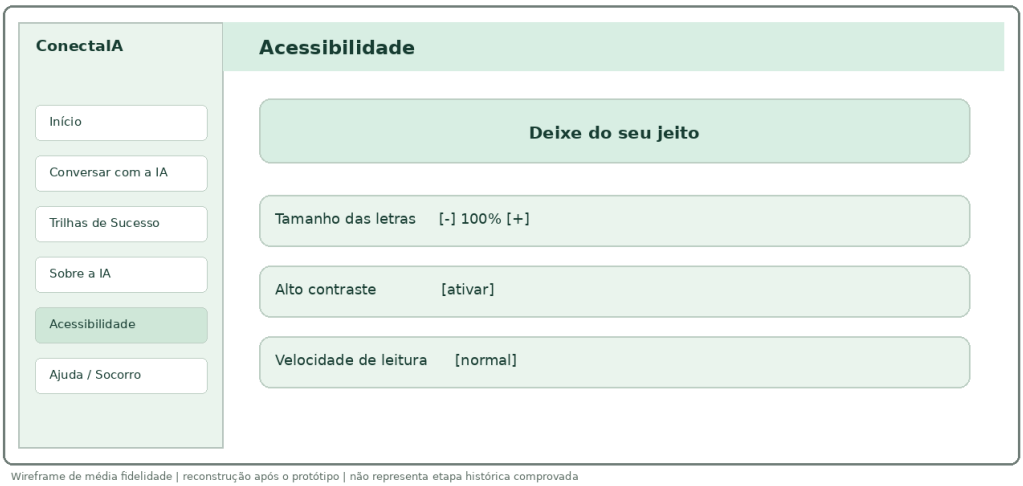



Figura A11 — Acessibilidade: wireframe de média fidelidade (reconstruído na versão anterior).

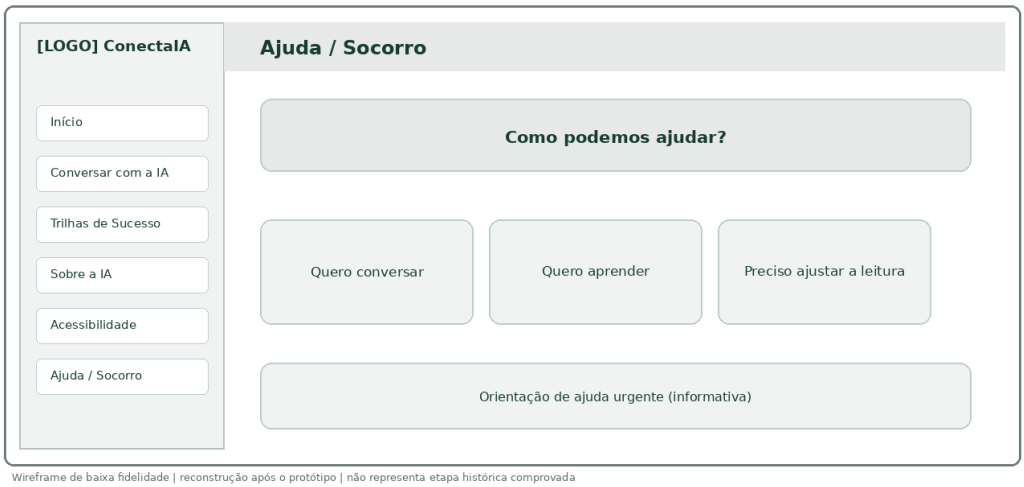



Figura A12 — Ajuda / Socorro: wireframe de baixa fidelidade (reconstruído na versão anterior).

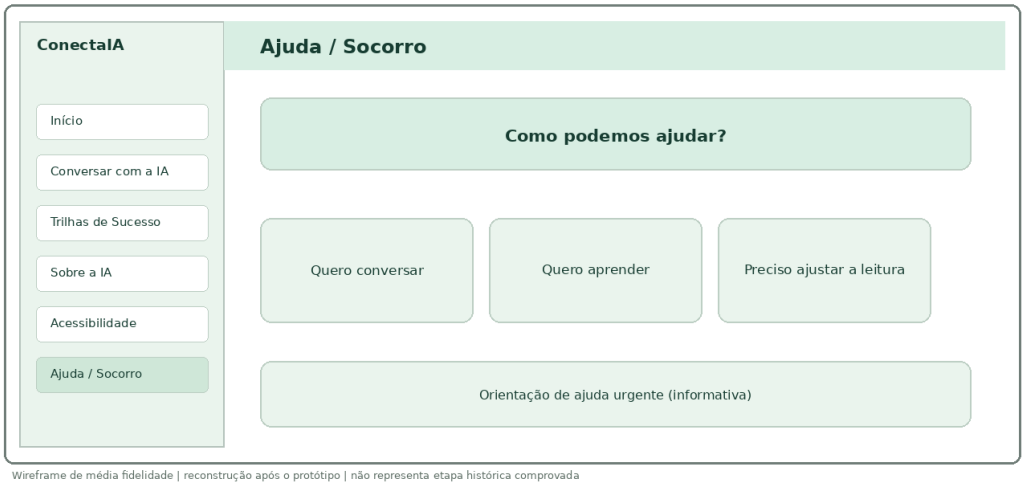



Figura A13 — Ajuda / Socorro: wireframe de média fidelidade (reconstruído na versão anterior).

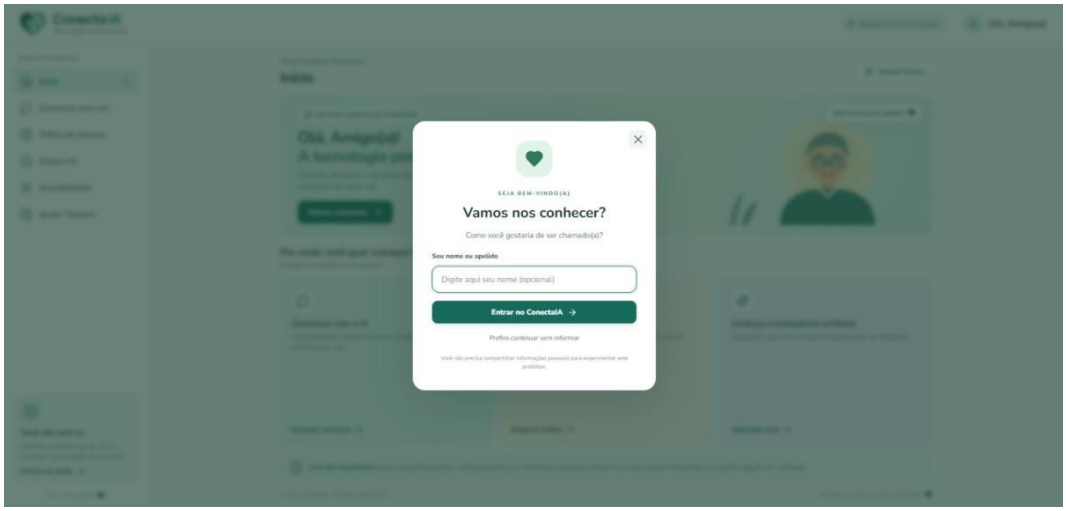



Figura A14 — Identificação inicial: imagem documental do protótipo fornecido anteriormente.

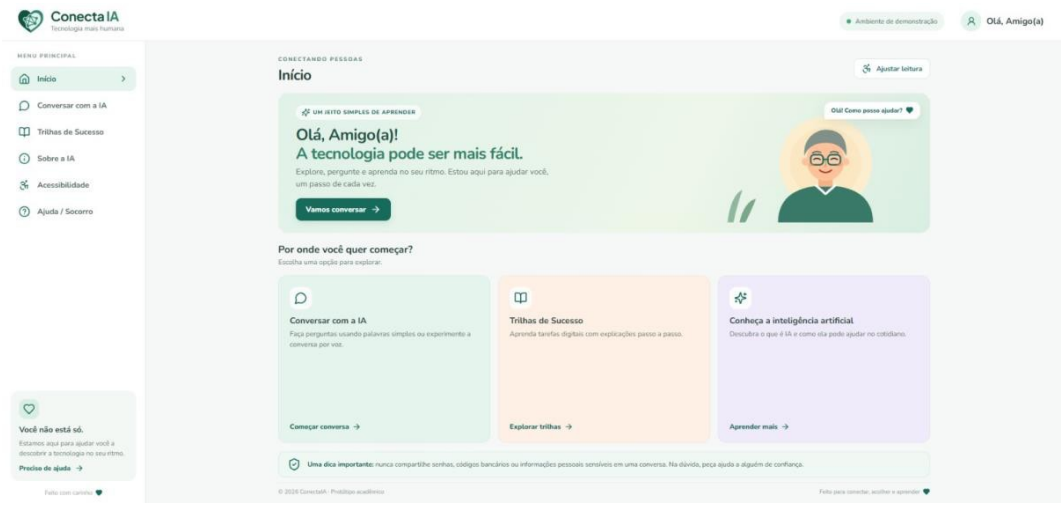



Figura A15 — Início / painel: imagem documental do protótipo fornecido anteriormente.

Figura A16 — Conversa — entrada: imagem documental do protótipo fornecido anteriormente.

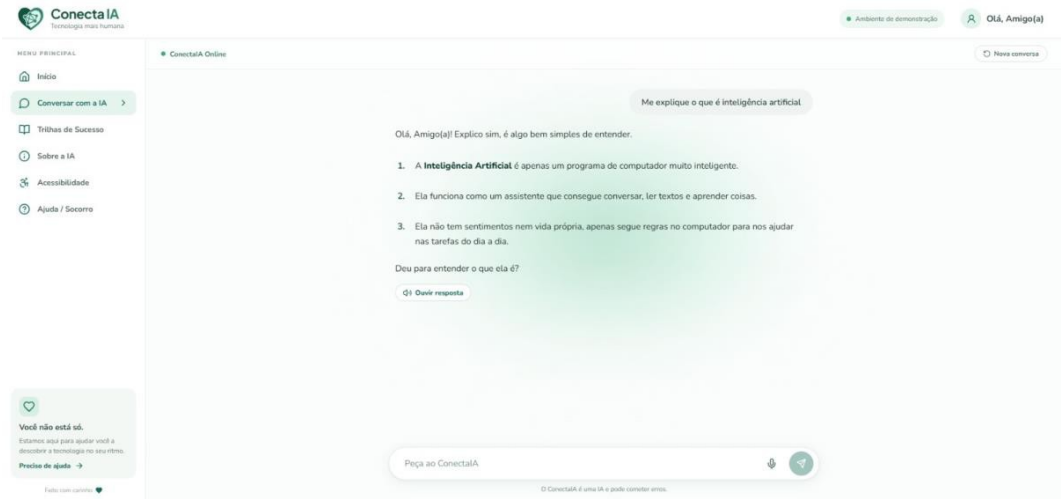



Figura A17 — Conversa — histórico: imagem documental do protótipo fornecido anteriormente.

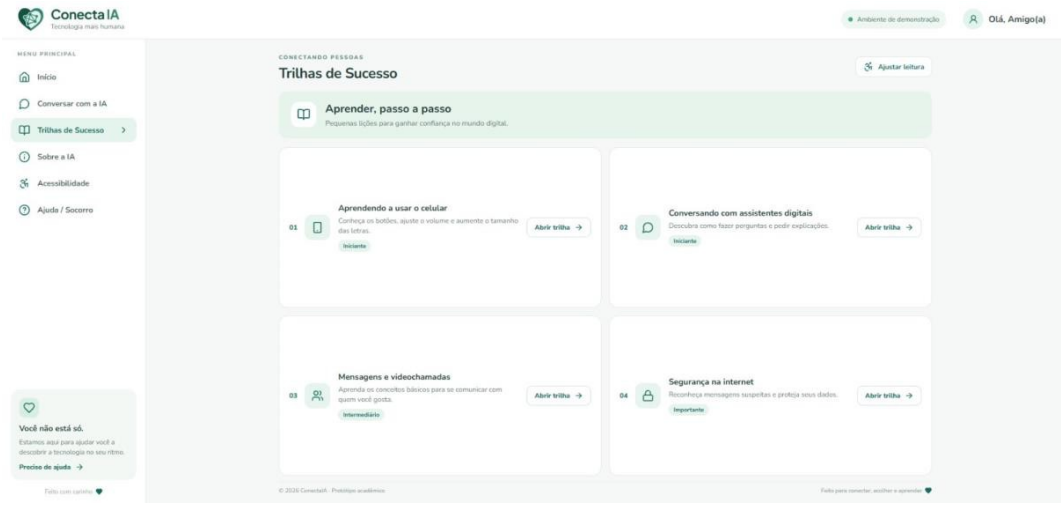



Figura A18 — Trilhas educativas: imagem documental do protótipo fornecido anteriormente.

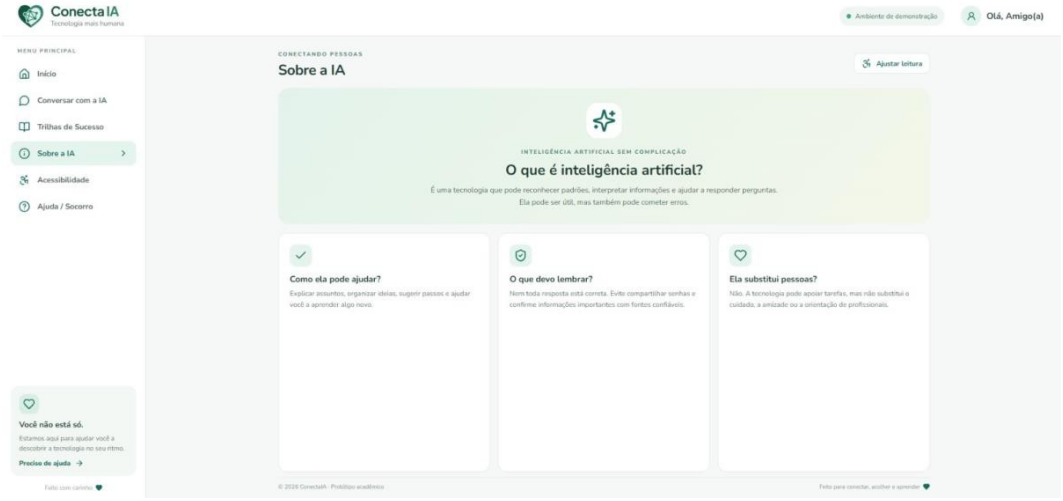



Figura A19 — Sobre a IA: imagem documental do protótipo fornecido anteriormente.

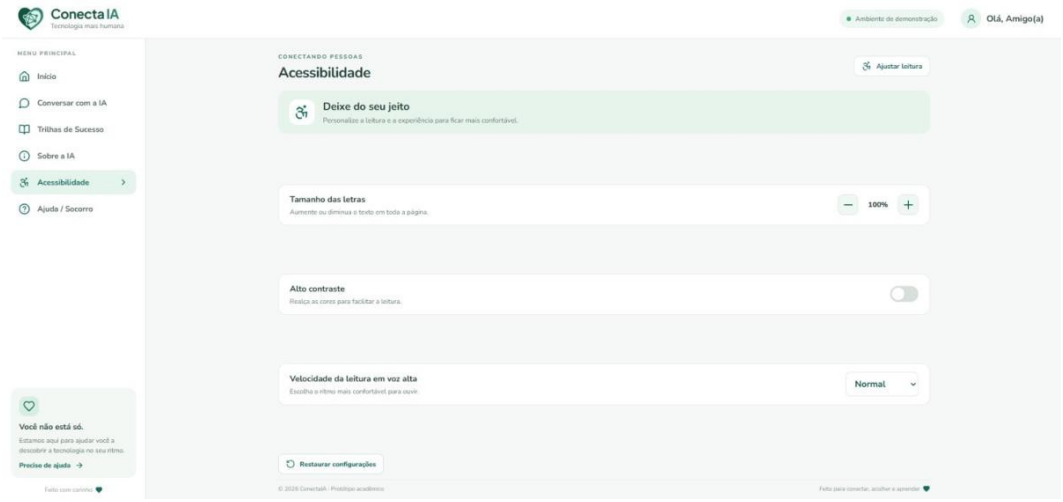



Figura A20 — Acessibilidade: imagem documental do protótipo fornecido anteriormente.

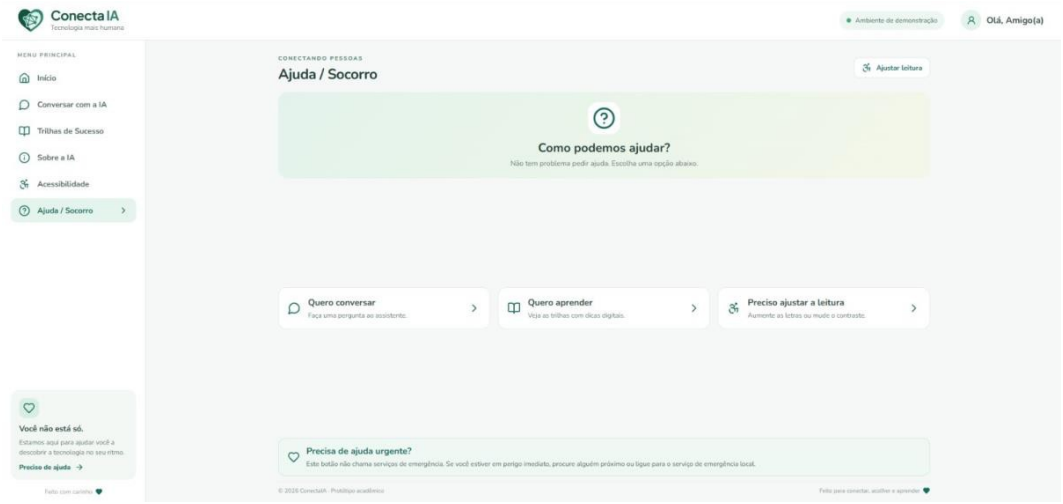



Figura A21 — Ajuda / Socorro: imagem documental do protótipo fornecido anteriormente.

# APÊNDICE B — INSTRUMENTOS DE TESTES E RASTREABILIDADE

Para permitir verificação posterior, mantêm-se os roteiros e os dados agregados originários, identificando seu nível de comprovação. Os controles devem ser preenchidos com dados reais antes de receber status de concluído.

## B.1 Roteiro das seis tarefas já relatadas

- T1 — Abrir o ConectaIA, concluir a identificação e chegar ao Início.
- T2 — Encontrar Conversar com a IA e identificar onde escrever uma pergunta.
- T3 — Abrir Trilhas de Sucesso e escolher um conteúdo útil.
- T4 — Localizar Sobre a IA e explicar um cuidado de segurança.
- T5 — Acessar Acessibilidade, ampliar texto e testar alto contraste.
- T6 — Utilizar Ajuda/Socorro para aprender ou retornar a uma atividade.

## B.2 Registros coletivos e fichas individuais

| Pessoa | ID | Conclusão* | Tempo* | Erros* |
|---|---|---|---|---|
| P1 | T1 | Concluída | 90 | 0 |
| P1 | T2 | Concluída | 90 | 0 |
| P1 | T3 | Concluída | 90 | 0 |
| P1 | T4 | Concluída | 90 | 0 |
| P1 | T5 | Concluída | 90 | 0 |
| P1 | T6 | Concluída | 90 | 0 |
| P2 | T1 | Concluída | 90 | 0 |
| P2 | T2 | Concluída | 90 | 0 |
| P2 | T3 | Concluída | 90 | 0 |
| P2 | T4 | Concluída | 90 | 0 |
| P2 | T5 | Concluída | 90 | 0 |
| P2 | T6 | Concluída | 90 | 0 |
| P3 | T1 | Concluída | 90 | 0 |
| P3 | T2 | Concluída | 90 | 0 |
| P3 | T3 | Concluída | 90 | 0 |
| P3 | T4 | Concluída | 90 | 0 |
| P3 | T5 | Concluída | 90 | 0 |
| P3 | T6 | Concluída | 90 | 0 |

Asteriscos e dados agregados são do relatório de origem: conclusão, duração e erros declarados pela equipe, não ficha observacional independente. Pedidos de ajuda não foram informados.

| Pessoa | SUS* | Tarefas | Total | Tempo* | Erros* |
|---|---|---|---|---|---|
| P1 | 100 | 6/6 | 6 | 90 s | 0 |
| P2 | 100 | 6/6 | 6 | 90 s | 0 |
| P3 | 100 | 6/6 | 6 | 90 s | 0 |
| Média / total | 100 | 18/18 (100%) | 18 | 90 s | 0 |

## B.3 Modelo de registro para novos testes

| Campo | Conteúdo a preencher após execução |
|---|---|
| Identificação | ID de caso; data; executor; versão; ambiente |
| Pré-condições | Dispositivo, navegador, rede e permissões |
| Passos | Ações detalhadas realizadas |
| Resultado esperado | Critério de aceite vinculado |
| Resultado obtido | Valor observado, captura e evidência |
| Conclusão | Aprovado / reprovado / bloqueado |
| Defeito e reteste | ID, severidade, correção, data e resultado |

## B.4 Checklist de segurança, acessibilidade e entregabilidade

| Verificação | Método recomendado | Estado |
|---|---|---|
| Contraste de texto e componentes | Medir pares de cores conforme WCAG 2.2. | VERIFICADO |
| Ampliação de fonte e reflow | Testar zoom, larguras menores e sem perda de conteúdo. | VERIFICADO |
| Operação por teclado | Percorrer com Tab/Shift+Tab/Enter/Espaço. | VERIFICADO |
| Foco visível | Verificar destaque e ordem lógica de foco. | VERIFICADO |
| Leitor de tela | Inspecionar nomes, papéis, estados e cabeçalhos. | VERIFICADO |
| Rótulos e instruções | Avaliar clareza de inputs, botões e mensagens de erro. | VERIFICADO |
| Idioma e semântica | Conferir lang, landmarks, heading hierarchy e labels. | VERIFICADO |
| Controles de acessibilidade | Verificar efeito real das opções exibidas. | VERIFICADO |
| Uso móvel | Verificar ajuste de cards, navegação e alvos sensíveis ao toque. | VERIFICADO |

## APÊNDICE C — CORRESPONDÊNCIA COM OS TEXTOS DE APOIO DIDÁTICOS

O enunciado solicita preservação integral dos cinco Textos de Apoio Didáticos e das respostas dos exercícios. As Aulas 1 a 5 são preservadas nos Anexos A a E, com suas 54 páginas originais, incluindo gráficos, tabelas e referências. Vinte respostas-modelo construídas a partir das entregas previstas no prompt estão integradas aos itens 2.5, 3.6, 4.5, 5.5 e 6.5. Entretanto, não foi disponibilizado o caderno oficial de exercícios; por esse motivo, não é possível verificar equivalência integral com eventual documento anterior ou reproduzir respostas que não tenham sido recebidas.

| Texto / aula solicitada | Cobertura nesta versão | Complementação necessária |
|---|---|---|
| Aula 1 — Fundamentos, ciclos e ISO/IEC 25010 | Etapa 1: fundamentos, modelos, ISO, V&V, DVCP | Texto original no Anexo A; respostas-modelo no item 2.5; execução a comprovar |
| Aula 2 — Requisitos, BPMN e UML | Etapa 2: elicitação, RF/RNF, Gherkin, BPMN, UML, rastreabilidade | Reprodução integral no Anexo B; exemplos FESP preservados como didáticos |
| Aula 3 — Agile, Scrum, Kanban, backlog | Etapa 3: valores, princípios, Cynefin, Scrum, fluxo, INVEST, Planning Poker | Reprodução integral no Anexo C; registros reais de execução ausentes |
| Aula 4 — Colaboração, Git e MVP | Etapa 4: papéis, PR, branching, protótipo e arquitetura | Reprodução integral no Anexo D; histórico Git e PRs ausentes |
| Aula 5 — Testes, validação, Kaizen e apresentação | Etapa 5: tipos de teste, defeitos, dados relatados e proposta de defesa | Reprodução integral no Anexo E; registros técnicos e retrospectiva ausentes |

Critério de fechamento documental: conferir as vinte respostas-modelo com os enunciados originais dos exercícios, caso sejam fornecidos; validar as respostas com a equipe e confrontar artefatos propostos com a versão efetivamente executada do ConectaIA. As reproduções didáticas dos Anexos A a E permanecem fiéis aos arquivos recebidos, sem representar evidência de implantação ou teste.

# APÊNDICE D — PLANO DE COMPROVAÇÕES E REGISTRO DAS ATIVIDADES

Este apêndice apresenta pendências verificáveis e um formulário de registro, em complemento às 20 atividades-modelo integradas às Etapas 1 a 5. Todos os campos de execução devem ser preenchidos somente mediante evidências produzidas pela equipe.

## D.1 Evidências ainda necessárias

As respostas-modelo anteriores podem ser incluídas no trabalho acadêmico. Para transformá-las em evidência efetiva do processo, deverão ser anexados os registros abaixo, caso a atividade tenha ocorrido. Onde não houver registro, realizar o procedimento e coletar provas reais.

| Entrega pendente | Como produzir a evidência | Onde anexar |
|---|---|---|
| Exercícios oficiais | Conferir o DOCX original do professor e vincular cada enunciado à resposta | Antes das respostas-modelo, preservando o enunciado |
| Elicitação | Inserir roteiros, síntese validada e consentimento pertinente | Atividade 2.1 |
| Modelagem | Comparar diagramas com os fluxos de telas e assinatura/revisão da equipe | Atividade 2.4 |
| Versionamento | Exportar histórico Git, PRs, revisões e referências às US | Atividade 4.2 |
| MVP funcional | Executar em navegador; registrar versão, ambiente, vídeo/capturas e resultados | Atividade 4.4 |
| Testes | Executar CT01–CT10; anexar registros de aprovado/reprovado e correções | Atividade 5.1 |
| SUS e tarefas | Anexar respostas individuais e fichas P1–P3, se existirem | Atividade 5.2 |
| Retrospectiva | Registrar reunião, participantes, aprendizados e ações priorizadas | Atividade 5.3 |
| Segurança | Rotacionar segredo de API e retestar exposição no frontend | Atividade 4.4/5.1 |

## D.2 Modelo de registro de evidências

**Atividade e requisito vinculado:**

_____

**Data e responsáveis reais:**

_____

**Ambiente / versão do software:**

_____

**Procedimento executado:**

_____

**Resultado observado (não presumido):**

_____

**Captura, log, commit, PR ou formulário anexado:**

_____

**Pendência / decisão seguinte:**

_____

## ANEXOS — REPRODUÇÃO INTEGRAL DOS CINCO TEXTOS DIDÁTICOS

Em cumprimento à exigência de não retirar palavras ou frases dos textos didáticos, os cinco PDFs de apoio estão integralmente incorporados como páginas fac-similares nos Anexos A–E, preservando a diagramação, gráficos, tabelas e referências. Os capítulos 2 a 6 apresentam a aplicação das cinco aulas ao ConectaIA, incluindo vinte respostas-modelo de atividades. Essas respostas não substituem eventuais registros ou enunciados originais não fornecidos, nem comprovam a execução de atividades práticas. Total de material didático preservado: 54 páginas em cinco anexos.

## ANEXO A — Aula 1 — Fundamentos da Engenharia de Software, Ciclos de Vida e Qualidade

Reprodução integral do texto de autoria identificada do Prof. Dr. André Roberto Guerra. Abrange conceitos, cronologia, ISO/IEC 25010, verificação/validação e modelo de DVCP. A aplicação ao ConectaIA está na Etapa 1.

Arquivo de origem: Texto - Aula 1.pdf. Total de páginas originais: 9. Reprodução sem cortes ou alterações de conteúdo.

**FESP — Fundação de Estudos Sociais do Paraná**
Cursos de Computação (TADS, IA, BSI e Eng. de Software) - 2º Período
Disciplina de Eng. de Software - Autor**: Prof. Dr. André Roberto Guerra, MSc.**

# Aula 1: Fundamentos da Engenharia de Software, Ciclos de Vida e Qualidade

## 1.1 Introdução à Engenharia de Software e o Cenário Profissional

A consolidação da Engenharia de Software como campo formal do conhecimento computacional decorre de um fenômeno histórico diagnosticado na década de 1960: a chamada **"Crise do Software"**. Identificada formalmente nas conferências da OTAN em Garmisch (1968) e Roma (1969), essa conjuntura derivou do descompasso entre a rápida evolução do poder de processamento do hardware e a incapacidade estrutural de produzir sistemas confiáveis, dentro do prazo estipulado e dos orçamentos orçados. O desenvolvimento artesanal, baseado em improvisação individual e ausência de métodos repetíveis, revelou-se inviável à medida que as aplicações migraram de rotinas numéricas simples para plataformas críticas de processamento contínuo.

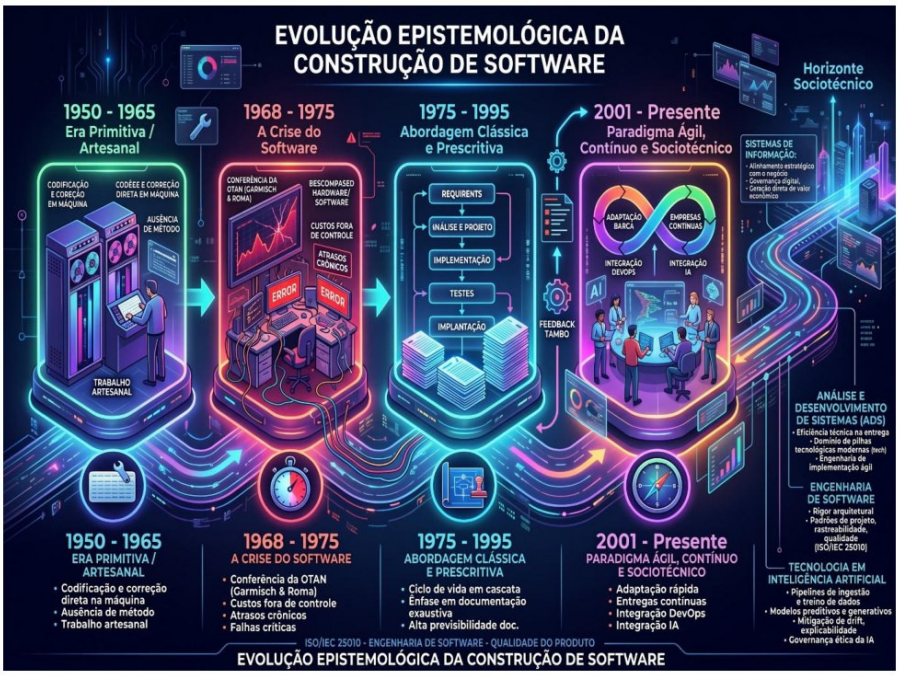
Segundo o padrão **IEEE 610.12** e as definições consagradas por **Pressman e Maxim (2021)**, a Engenharia de Software consiste na aplicação sistemática, disciplinada e quantificável de métodos científicos e de engenharia ao desenvolvimento, à operação e à manutenção de sistemas de computação. Trata-se, portanto, do estabelecimento de princípios consolidados de engenharia com a finalidade de obter, economicamente, softwares confiáveis e que funcionem de forma eficiente em máquinas reais.

### Aprofundamento Conceitual: O que define a maturidade da Engenharia?

A transição de uma atividade puramente empírica para um ramo rigoroso da engenharia baseia-se em quatro pilares fundamentais estabelecidos pelo *Software Engineering Body of Knowledge* (SWEBOK):

- **Repetibilidade e Previsibilidade:** A capacidade de replicar processos bem-sucedidos em diferentes contextos e estimar prazos e custos com desvios estatisticamente controlados.
- **Garantia de Qualidade Contínua (QA):** A inserção de métodos analíticos e empíricos de teste e inspeção desde as fases embrionárias de concepção.
- **Gestão da Manutenibilidade:** O reconhecimento de que o custo total de propriedade (TCO) de um software reside majoritariamente na sua evolução e adaptação operacional, e não apenas na codificação inicial.
- **Sustentabilidade Arquitetural:** O desenho de estruturas computacionais modulares e desacopladas, capazes de acomodar novos requisitos sem degradação do desempenho ou colapso estrutural.

**Evolução Epistemológica da Construção de Software**




- **1950 – 1965 | Era Primitiva / Artesanal:** Caracterizada pela codificação e correção direta em máquina, ausência quase total de métodos estruturados, centralização nas mãos de programadores individuais (*hero programming*) e dependência extrema de linguagens de máquina ou montadores elementares.
  - **Paradigma Epistemológico:** O código como expressão estritamente individual e técnica mecânica (*Code-and-Fix*).
  - **Práticas Centrais:**
    - Codificação e correção direta na máquina (cartões perfurados, painéis e linguagens de baixo nível).
    - Ausência de método formal, modelos de processo ou padronização.
    - Trabalho essencialmente artesanal centrado no programador.
  - **Limitação Chave:** Dependência extrema de especialistas pontuais e impossibilidade de escala ou manutenibilidade de longo prazo.

- **1968 – 1975 | A Crise do Software:** Período de choque com custos fora de controle, atrasos crônicos, falhas críticas em produção e baixa confiabilidade dos sistemas entregues. Surge a exigência premente de formalização teórica e metodológica nas conferências internacionais da OTAN.
  - **Marco Histórico:** Conferências da OTAN em Garmisch (1968) e Roma (1969), onde se cunhou formalmente a disciplina de **Engenharia de Software**.
  - **Fatores Geradores:** Descompasso brutal entre a evolução exponencial do poder de hardware e a incapacidade de desenvolver software complexo de maneira previsível.

- **Sintomas Críticos**:
  - Custos sistematicamente fora de controle e orçamentos estourados.
  - Atrasos crônicos e entregas que não atendiam às necessidades do usuário.
  - Falhas catastróficas em produção e baixa confiabilidade operacional.

- **1975 – 1995 | Abordagem Clássica e Prescritiva:** Consolidação do ciclo de vida em cascata, formalização da análise estruturada e orientada a objetos, com ênfase dominante em documentação exaustiva e processos pesados e lineares.
  - **Paradigma Epistemológico:** O software como processo de engenharia fabril linear e determinístico (Mecânica e Construção Civil aplicadas à computação).
  - **Modelo Central:** Ciclo de vida em cascata (*Waterfall Model*) com fases estanques:
    Requisitos ⟶ Análise e Projeto ⟶ Implementação ⟶ Testes ⟶ Implantação
  - **Práticas Centrais:**
    - Ênfase em documentação exaustiva e contratos de requisitos congelados.
    - Rigor prescritivo e busca por alta previsibilidade documental.
  - **Gargalo Principal:** *Feedback tardio* — erros e mudanças de escopo descobertos apenas nas fases finais geravam retrabalho oneroso.

- **2001 – Presente | Paradigma Ágil, Contínuo e Sociotécnico:** Rompimento com a rigidez documental com o advento do Manifesto Ágil, priorizando adaptação rápida, entregas contínuas de valor, integração de práticas DevOps e incorporação progressiva de Inteligência Artificial aos pipelines de entrega.
  - **Paradigma Epistemológico:** O software como sistema complexo adaptativo, vivo e sociotécnico (pessoas, processos, plataformas e modelos estocásticos).
  - **Marcos e Práticas Centrais:**
    - **Agilidade (Manifesto Ágil):** Ciclos curtos, inspeção, adaptação rápida e foco em colaboração e valor ao cliente.
    - **Engenharia Contínua (DevOps / DevSecOps):** Integração contínua (CI), entrega contínua (CD), observabilidade e automação de ponta a ponta.
    - **Inteligência Artificial Integrada:** Engenharia assistida por LLMs, pipelines de MLOps, agentes autônomos e geração dinâmica de código.

**Trilha de Integração: Horizonte Sociotécnico**

- **Sistemas de Informação:** Alinhamento estratégico com o negócio, governança digital, modelagem de fluxos corporativos e geração direta de valor econômico.
- **Análise e Desenvolvimento de Sistemas (ADS):** Eficiência técnica na entrega, domínio de pilhas tecnológicas modernas (*tech stacks*) e engenharia de implementação ágil.
- **Engenharia de Software:** Rigor arquitetural, padrões de projeto (*design patterns*), rastreabilidade de requisitos, testes automatizados e garantia de qualidade (ISO/IEC 25010).
- **Tecnologia em Inteligência Artificial:** Pipelines de ingestão e treino de dados, modelos preditivos e generativos, mitigação de *drift*, explicabilidade e governança ética da IA.

### O Engenheiro de Software no Cenário Contemporâneo

O papel do engenheiro de software no ecossistema corporativo contemporâneo transcende a escrita de sintaxe executável. Exige-se uma compreensão sociotécnica que articula o modelo de negócio das organizações, as necessidades dos usuários finais, a conformidade regulatória (como LGPD e governança de dados) e a sustentabilidade arquitetural do código-fonte. No contexto de cursos integrados de BSI, TADS, Engenharia de Software e Tecnologia em IA, essa visão se desdobra em **quatro** ênfases complementares:

- **Sistemas de Informação (BSI):** Foco no alinhamento estratégico, fluxos corporativos e geração de valor econômico. Prioriza a integração entre os objetivos de negócio, a governança corporativa de TI e a eficiência dos processos organizacionais.
- **Análise e Desenvolvimento de Sistemas (TADS):** Foco na engenharia de implementação, eficiência técnica de código e integração com pilhas tecnológicas modernas. Concentra-se na solução prática de problemas por meio de frameworks robustos, bibliotecas contemporâneas e entrega rápida de software funcional.
- **Engenharia de Software:** Foco no rigor de processos, arquitetura, rastreabilidade, garantia de qualidade e métricas de confiabilidade. Enfatiza a modelagem formal de sistemas de grande escala, auditoria de código, padrões de projeto (*design patterns*) e sustentabilidade de longo prazo.
- **Tecnologia em Inteligência Artificial:** Foco em pipelines orientados a dados, modelos estocásticos, monitoramento de degradação algorítmica (*model drift*) e rastreabilidade de decisões automatizadas. Alia a computação tradicional às práticas de MLOps, ética em inteligência artificial e observabilidade contínua de predições.

## 1.2 Modelos de Processo e Ciclos de Vida de Software

O ciclo de vida de um software (*Software Development Life Cycle* - SDLC) define a estrutura de fases pelas quais o artefato computacional transita desde sua concepção inicial até a descontinuação operacional. Os modelos de processo institucionalizam as atividades de comunicação, planejamento, modelagem, construção e implantação:

1. **Comunicação:** Identificação de necessidades, levantamento inicial de requisitos e alinhamento com stakeholders.
2. **Planejamento:** Estimativas de esforço, dimensionamento de cronograma, análise de riscos e alocação de recursos técnicos e humanos.
3. **Modelagem:** Elaboração de arquiteturas, diagramas funcionais, fluxogramas, modelos de domínio e esquemas de dados.
4. **Construção:** Atividades coordenadas de codificação de regras e interfaces, combinadas com testes unitários e de integração.
5. **Implantação (*Deployment*):** Entrega do software no ambiente de produção, treinamento de usuários, monitoramento e coleta contínua de feedback.

## Modelos de Ciclo de Vida: Análise Estrutural

**1. Modelo Cascata (Linear / Preditivo)**

[Requisitos] → [Análise/Projeto] → [Implementação] → [Testes] → [Implantação]

*(Feedback tardio; alto custo de correção em fases avançadas)*

**2. Modelo Iterativo e Incremental**

Ciclo 1: [Req] → [Proj] → [Cod] → [Test] —> Incremento 1 (Funcionalidade Core)
Ciclo 2: [Req] → [Proj] → [Cod] → [Test] —> Incremento 2 (+ Autenticação e Regras)
Ciclo 3: [Req] → [Proj] → [Cod] → [Test] —> Incremento 3 (+ Relatórios e IA)

## Detalhamento dos Modelos de Ciclo de Vida

**1. Modelo Cascata Clássico (Waterfall)**

Proposto formalmente por Winston Royce em 1970 (embora com ressalvas explícitas ao seu uso estritamente linear), organiza o processo em fases sequenciais estanques, nas quais uma etapa só se inicia após a homologação e o congelamento documental da anterior.

- **Vantagens:** Alta previsibilidade documental, clareza nas metas contratuais e facilidade de fiscalização gerencial em projetos com escopo totalmente estável.
- **Limitações:** Rigidez excessiva diante de alterações nos requisitos, adiamento do contato do usuário com o sistema executável para os estágios finais e propagação silenciosa de erros de especificação, cujo custo de resolução cresce exponencialmente à medida que a entrega avança.

**2. Modelos Iterativos e Incrementais**

Diferenciam-se da linearidade ao subdividir o escopo em blocos gerenciáveis.

- O **aspecto iterativo** pressupõe a revisão e o refinamento cíclico dos artefatos à medida que o entendimento do problema amadurece.
- O **aspecto incremental** refere-se à entrega sucessiva de subconjuntos funcionais do sistema que agregam valor imediato ao usuário.

**3. Abordagens Especializadas**

- **Modelo Espiral (Boehm):** Orienta o desenvolvimento em torno de ciclos sucessivos de avaliação e mitigação explícita de riscos técnicos e de negócio. Cada volta na espiral percorre quatro quadrantes fundamentais: determinação de objetivos e restrições, identificação e resolução de riscos (com prototipação), desenvolvimento e validação do produto do ciclo, e planejamento da próxima fase.
- **Desenvolvimento Rápido de Aplicações (RAD) e Abordagens Evolucionárias:** Priorizam a construção rápida de versões executáveis a partir do estreitamento de laços com os usuários finais. O feedback prático obtido com os primeiros protótipos direciona a evolução dos requisitos e a arquitetura definitiva do produto.

## Matriz Comparativa dos Modelos de Ciclo de Vida

Abaixo, a representação visual de alta definição sintetiza a comparação estrutural entre os modelos de ciclo de vida:

**Tabela Detalhada de Comparação de Critérios**

| Critério de Análise | Modelo Cascata (Waterfall) | Modelo Incremental | Modelo Evolucionário / Espiral |
|---|---|---|---|
| **Estabilidade de Requisitos** | Exige congelamento prévio e baixa volatilidade | Tolera variações controladas entre incrementos | Projetado para ambientes de alta incerteza |
| **Custo da Mudança** | Exponencial ao longo do tempo | Moderado e contido no incremento | Baixo a moderado devido à mitigação de riscos |
| **Envolvimento do Usuário** | Concentrado no início e na fase de aceite final | Contínuo a cada entrega funcional parcial | Intenso em todas as iterações de avaliação |
| **Visibilidade do Produto** | Apenas no término do ciclo de desenvolvimento | Frequente, com software executável periódico | Frequente, através de protótipos e versões piloto |
| **Gestão de Risco Técnico** | Reativa, identificada tardiamente em testes | Gradual, mitigada pelo avanço dos incrementos | Proativa, sendo o motor de transição de ciclos |

## 1.3 Processos de Desenvolvimento, Verificação, Validação e Qualidade

A qualidade de software é um construto multidimensional que não pode ser reduzido à simples ausência de erros sintáticos de execução. Como postula a norma internacional **ISO/IEC 25010** (*Systems and Software Quality Requirements and Evaluation* - SQuaRE), a qualidade divide-se em modelos de qualidade em uso e modelos de qualidade de produto de software.

Abaixo, o infográfico apresenta a taxonomia do modelo de qualidade de produto de software da ISO/IEC 25010:

ANEXO A — Fonte original, página 6 de 9. Reprodução integral FESP, 2026.

## Detalhamento das 8 Características e Subcaracterísticas da ISO/IEC 25010

**1. Adequação Funcional:** Grau em que o produto atende às funções especificadas ao ser usado sob condições preestabelecidas.

- *Completude Funcional:* Cobertura de todas as tarefas e objetivos especificados pelo usuário.
- *Correção Funcional:* Fornecimento de resultados precisos e esperados.
- *Pertinência Funcional:* Capacidade de fornecer apenas recursos que de fato agregam valor e facilitam a realização de tarefas.

**2. Eficiência de Desempenho:** Desempenho relativo à quantidade de recursos utilizados em condições estabelecidas.

- *Tempo de Resposta:* Latência, tempo de processamento e taxas de vazão (*throughput*) durante a execução.
- *Utilização de Recursos:* Uso eficiente de CPU, memória, I/O e largura de banda.

**3. Compatibilidade:** Grau em que um produto pode trocar informações com outros sistemas e/ou compartilhar o mesmo ambiente de hardware e software.

- *Coexistência:* Capacidade de compartilhar um ambiente de execução comum com outros softwares sem impacto adverso.
- *Interoperabilidade:* Capacidade de dois ou mais sistemas trocarem dados e utilizarem mutuamente a informação trocada.

**4. Usabilidade:** Facilidade com que um produto pode ser utilizado por usuários específicos para alcançar metas com eficácia, eficiência e satisfação.

- *Reconhecimento de Adequação:* Facilidade com que o usuário entende se o software é apropriado para suas necessidades.
- *Aprendizagem:* Facilidade de aprender a operar a aplicação de maneira independente e intuitiva.
- *Operabilidade:* Facilidade de navegação, controle e manipulação das interfaces.
- *Proteção contra Erros de Uso:* Prevenção ativa e resiliência diante de comandos equivocados inseridos pelo operador.

**5. Confiabilidade:** Grau em que um sistema executa funções especificadas sob condições pré-definidas durante um intervalo de tempo estabelecido.

- *Maturidade:* Frequência reduzida de falhas decorrentes de defeitos residuais no código.
- *Tolerância a Falhas:* Capacidade de continuar operando adequadamente mesmo diante de exceções ou falhas parciais.
- *Recuperabilidade:* Agilidade e capacidade de restaurar o estado consistente dos dados após incidentes operacionais.

**6. Segurança:** Capacidade de proteger as informações e os dados para que indivíduos ou outros sistemas tenham o grau de acesso aos dados compatível com seus níveis de autorização.

- *Confidencialidade:* Garantia de que os dados são acessíveis apenas por entidades expressamente autorizadas.
- *Integridade:* Prevenção de modificações ou deleções não autorizadas de código ou dados.
- *Não Repúdio:* Prova da ocorrência de uma ação ou evento, sem a negação de sua autoria.
- *Autenticidade:* Confirmação indiscutível da identidade de usuários, processos ou dispositivos.
- *Responsabilidade (Accountability):* Rastreabilidade inequívoca de ações executadas no sistema até a entidade causadora.

**7. Manutenibilidade:** Eficácia e eficiência com as quais o software pode ser modificado para ser corrigido, aprimorado ou adaptado.

- *Modularidade:* Divisão do sistema em componentes independentes, minimizando o acoplamento.
- *Reusabilidade:* Grau em que ativos de software podem ser aplicados na construção de múltiplos componentes ou produtos.
- *Analisabilidade:* Facilidade de diagnosticar defeitos, causas de falhas ou identificar partes a serem alteradas.
- *Modificabilidade:* Facilidade de implementar alterações sem degradar a estabilidade do sistema existente.
- *Testabilidade:* Nível de facilidade para estabelecer critérios de validação e executar testes automatizados sobre o código.

8. **Portabilidade:** Eficácia e facilidade com que o sistema pode ser transferido de um ambiente operacional de hardware, software ou rede para outro.
   - *Adaptabilidade:* Habilidade de acomodação a diferentes ambientes técnicos sem necessidade de alterações extensivas.
   - *Facilidade de Instalação:* Sucesso e simplicidade com os quais o sistema é instalado ou desinstalado em ambientes-alvo.
   - *Substituibilidade:* Capacidade de substituir outro software concorrente ou legado no mesmo ambiente para o mesmo propósito.

### Verificação versus Validação (V&V)

A distinção teórica proposta por **Barry Boehm** permanece basilar para a engenharia contemporânea:

**Verificação:** *"Are we building the product right?"* (Estamos construindo o produto corretamente?)

Avalia se os produtos de uma determinada fase de desenvolvimento satisfazem as condições impostas pelo início dessa fase.

Consiste em garantir que o software está sendo construído em estrita conformidade com as suas especificações arquiteturais, padrões de codificação e modelos projetados. Envolve revisões técnicas formais, análises estáticas de código, inspeções de arquitetura e execução de testes unitários.

**Validação:** *"Are we building the right product?"* (Estamos construindo o produto certo?)

Avalia se o software atende ao seu uso pretendido e às reais necessidades operacionais do usuário e do negócio no ambiente operacional. Trata-se da confirmação de valor prático da solução. É comprovada por meio de testes de aceitação do usuário (*UAT*), testes de carga e desempenho em ambiente realista, validação de hipóteses de negócio e monitoramento do comportamento dos usuários em produção.

## 1.4 Governança de Artefatos e Documentação Técnica Básica

A documentação em projetos de software não deve ser encarada como uma burocracia inerte, mas sim como um instrumento de comunicação entre stakeholders e uma salvaguarda para a sustentabilidade da manutenção do sistema. Sistemas desprovidos de documentação estruturada acumulam débito técnico imediato, tornando qualquer transição de equipe um processo crítico e custoso.

A documentação divide-se em **três** domínios elementares:

- **Documentação de Requisitos e Negócio:** Explicita o problema, os objetivos organizacionais, os requisitos funcionais, os requisitos não funcionais e as regras de negócio regulatórias. Define os limites contratuais do projeto e estabelece os critérios formais de aceitação.

- **Documentação Técnica e de Arquitetura:** Descreve diagramas estruturais e comportamentais, esquemas de banco de dados, especificações de rotas de API (via padrões como OpenAPI/Swagger) e decisões arquiteturais documentadas em ADRs (*Architectural Decision Records*). Garante a longevidade técnica e a rastreabilidade das decisões de engenharia.

- **Documentação de Usuário e Operação:** Manuais de instalação, guias contextuais de uso, tutoriais e procedimentos de restauração do sistema em incidentes (playbooks e runbooks de suporte). Assegura a autonomia de operadores, administradores de infraestrutura e usuários finais.

## Template Operacional: Documento de Visão e Contexto do Projeto (DVCP)

O template a seguir constitui a linha de base para a especificação inicial de projetos nos cursos integrados de tecnologia:

**DOCUMENTO DE VISÃO E CONTEXTO DO PROJETO (DVCP)**

**Identificação Geral**
- Nome do Projeto: [Nome da Solução]
- Equipe / Turma: [Identificação dos Estudantes e Cursos - BSI/TADS/Eng/IA]
- Versão do Documento: 1.0 | Data de Emissão: [DD/MM/AAAA]

**1. Problema de Negócio e Contexto Real**
- O Problema: [Descrição detalhada da dor operacional, ineficiência ou gap de mercado]
- Afeta: [Quais departamentos, usuários ou empresas sofrem com o problema]
- Impacto Operacional: [Custos financeiros, perdas de tempo ou vulnerabilidades geradas]
- Solução Atual Adotada: [Como o problema é tratado hoje paliativamente (planilhas, processos manuais)]

**2. Declaração de Posicionamento do Produto**
- Para: [Público-alvo / Usuários finais primários]
- Que: [Declaração da necessidade crítica de operação ou oportunidade de inovação]
- O: [Nome da Solução de Software]
- É um: [Categoria de sistema: web app, pipeline de IA, microsserviço, ERP setorial]
- Que entrega: [Benefício-chave de valor, eficiência mensurável ou automação]
- Ao contrário de: [Alternativa legada ou concorrência direta no mercado]
- Nosso produto: [Diferencial técnico, escalabilidade ou inteligência aplicada]

**3. Partes Interessadas (Stakeholders) e Personas**
- Patrocinador (Sponsor): [Responsável econômico/estratégico pelo projeto]
- Operador Primário: [Perfil técnico ou administrativo do usuário diário]
- Usuário Beneficiário: [Cliente final impactado pela entrega do serviço]

**4. Restrições Arquiteturais e Premissas Tecnológicas**
- Linguagens e Frameworks Homologados: [Ex: Node.js, Python, TypeScript, Spring Boot]
- Infraestrutura de Banco de Dados: [Ex: PostgreSQL, MongoDB, Redis]
- Plataforma de Hospedagem: [Ambiente Cloud ou servidor local de testes]
- Conformidade Regulatória: [Adequação à LGPD, diretrizes éticas de IA e ISO/IEC 25010]

# ANEXO B — Aula 2 — Engenharia de Requisitos e Modelagem Inicial de Sistemas

Reprodução integral do texto institucional, incluindo os exemplos originais FESP de triagem de chamados, módulo acadêmico, tabelas de requisitos, BPMN, UML e rastreabilidade. Esses exemplos são didáticos, não módulos comprovados do ConectaIA. A aplicação ao projeto está na Etapa 2.

Arquivo de origem: Texto da Aula 2 - Eng Software.pdf. Total de páginas originais: 19. Reprodução sem cortes ou alterações de conteúdo.

## Engenharia de Software e Desenvolvimento Ágil de Soluções

**Cursos Integrados:** 2º Semestre dos Bacharelados em Sistemas de Informação (BSI) e em Engenharia de Software, Tecnologia em Análise e Desenvolvimento de Sistemas (TADS) e Tecnologia em Inteligência Artificial

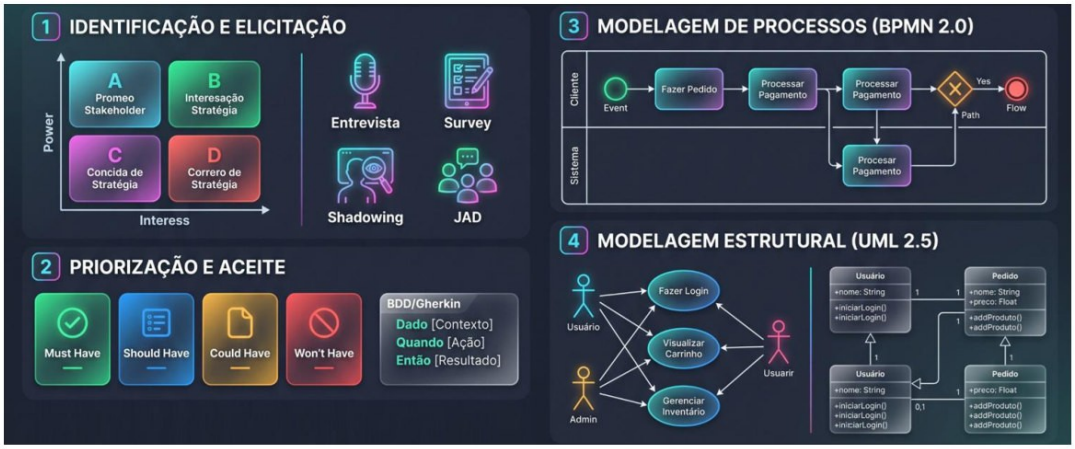
# Aula 2: Engenharia de Requisitos e Modelagem Inicial de Sistemas

*Orientar a identificação, classificação, priorização e modelagem das necessidades de software para mitigar o risco técnico e evitar o fracasso do projeto.*

## Visão Geral e Alinhamento Estrutural

O desenvolvimento de software corporativo de missão crítica opera sobre uma cadeia rigorosa de transformações epistêmicas: converte necessidades humanas difusas em modelos conceituais estruturados e, subsequentemente, em arquiteturas computacionais determinísticas e código executável testável.

A representação sintética a seguir condensa os quatro macroestágios que estruturam esta Aula 2:




- **Transição Metodológica As-Is -> To-Be**
- **Diagrama de Casos de Uso (Fronteiras, Include, Extend)**
- **Pools, Lanes, Eventos, Tarefas**
- **Diagrama de Classes de Domínio (Entidades e Tipagem)**
- **Gateways Decisórios (XOR, AND, OR)**
- **Rastreabilidade Ponta a Ponta: "O Fio de Ouro"**

## O Custo Oculto da Ambiguidade e o Efeito Cascata de Falhas

A literatura clássica de Engenharia de Software — desde os estudos empíricos seminais de Barry Boehm até as diretrizes contemporâneas do SWEBOK e da norma ISO/IEC/IEEE 29148 — demonstra que o custo de correção de uma falha cresce de forma exponencial conforme o ciclo de vida avança.

Uma inconsistência de domínio ignorada na fase de elicitação corrompe os modelos arquiteturais subsequentes, traduzindo-se em componentes e bancos de dados desalinhados da realidade operacional.

**Gráfico 1: A Curva Exponencial de Custo de Mudança (Boehm vs. Práticas Modernas de Engenharia)**

## 2.1 Elicitação de Requisitos e Mapeamento de Stakeholders

A **Engenharia de Requisitos** consiste na fase fundamental em que os limites do sistema computacional são determinados, negociados e acordados entre os agentes de interesse.

Requisitos imperfeitos, incompletos ou contraditórios constituem a principal causa de fracasso em projetos de software industriais.

Antes de escrever qualquer linha de código ou selecionar pilhas tecnológicas, a equipe de engenharia deve diagnosticar com precisão a rede de atores humanos, organizacionais e técnicos que influenciam e são impactados pela solução.

A identificação precisa das partes interessadas (*stakeholders*) demanda o emprego da **Matriz Poder versus Interesse**, instrumento analítico derivado do gerenciamento estratégico que classifica os agentes em quatro quadrantes distintos para otimizar os canais, a cadência e o teor da comunicação:

ANEXO B — Fonte original, página 4 de 19. Reprodução integral FESP, 2026.

## O Arsenal Metodológico de Elicitação de Requisitos

As técnicas de elicitação de requisitos variam conforme a natureza, a complexidade técnica e a maturidade do domínio de negócio sob investigação. O engenheiro contemporâneo não pode se restringir a um único método; deve aplicar uma abordagem multifacetada para confrontar discursos teóricos com a realidade operacional da empresa:

- **Entrevistas Estruturadas e Semiestruturadas:** Viabilizam a extração de necessidades profundas, bloqueios operacionais e expectativas individuais de atores estratégicos. O formato estruturado estabelece perguntas padronizadas para fins comparativos; o formato semiestruturado combina um roteiro-base com liberdade para explorar ramificações não previstas que emergem durante o diálogo técnico.
- **Questionários Quantitativos:** Permitem a coleta padronizada de informações de grandes bases amostrais de usuários quando as equipes atuam de modo distribuído. Constituem instrumentos eficientes para mensurar frequências de uso, prioridades de mercado e graus de satisfação, embora não capturem nuances de comportamento ou processos informais.
- **Observação Etnográfica e *Shadowing*:** O engenheiro acompanha o usuário em seu ambiente real de trabalho para identificar lacunas não verbalizadas, atalhos informais e desvios de processo que raramente são reportados nas entrevistas formais. É a ferramenta mais eficiente para desmascarar a discrepância entre a "norma oficial da empresa" e a "prática operacional cotidiana".
- **Joint Application Development (JAD):** Sessões intensivas e altamente colaborativas que reúnem analistas, desenvolvedores, arquitetos e tomadores de decisão em uma sala de trabalho para convergir requisitos divergentes de modo rápido e transparente. Mediadas por um facilitador neutro, as oficinas JAD eliminam rodadas morosas de aprovação de documentos individuais e reduzem semanas de negociação a poucos dias de trabalho concentrado.

### Tabela 2: Matriz Comparativa do Arsenal de Elicitação

| Técnica de Elicitação | Foco Central / Ponto Forte | Custo e Esforço | Cenário Ideal de Aplicação | Armadilha / Risco Metodológico |
|---|---|---|---|---|
| **Entrevistas** | Dores profundas e motivações estratégicas dos atores-chave. | Alto | Entender bloqueios operacionais de diretores, gerentes e especialistas de domínio. | Viés pessoal do entrevistado e omissão inconsciente de rotinas cotidianas triviais. |
| **Questionários** | Volume, alcance geográfico e tabulação estatística. | Baixo | Coletar dados de grandes bases amostrais de usuários remotos ou clientes finais. | Perguntas ambíguas geram dados estéreis; baixa taxa de engajamento no preenchimento. |
| **Observação (*Shadowing*)** | Processos ocultos, rotinas reais e atalhos não documentados. | Alto | Mapear atalhos informais, sistemas paralelos de controle (ex: planilhas) e vícios operacionais que o usuário não verbaliza. | Efeito Hawthorne: o trabalhador altera seu comportamento padrão pelo fato de estar sendo explicitamente observado. |
| **Workshops (JAD)** | Convergência rápida e resolução direta de conflitos de interesse. | Médio a Alto | Colocar analistas, desenvolvedores e executivos na mesma sala para destravar impasses técnicos e de negócio. | Dominação da sessão por figuras hierarquicamente superiores, inibindo a contribuição técnica da base. |

### Classificação Estrutural de Requisitos: O Cérebro e o Esqueleto

A especificação rigorosa de requisitos exige distinguir com clareza o comportamento dinâmico que o sistema executa das restrições arquiteturais e de qualidade dentro das quais esse comportamento é viabilizado:

- **Requisitos Funcionais (RF):** Representam o "Cérebro" do software. Definem o comportamento do sistema, especificando as transformações de entrada em saída, as regras de processamento, os serviços oferecidos e as reações a estímulos específicos.
  - *Exemplo canônico:* "O sistema deve calcular o score de crédito do solicitante processando seu histórico financeiro via inferência bayesiana em menos de dois segundos".
- **Requisitos Não Funcionais (RNF):** Representam o "Esqueleto" da solução. Descrevem restrições e propriedades de qualidade que o sistema precisa observar em sua operação global, agrupados de acordo com os pilares da **ISO/IEC 25010**.
  - *Exemplos canônicos:* Tempo máximo de latência de resposta de requisições de API fixado em 2 segundos; tráfego protegido integralmente por criptografia em trânsito sob TLS 1.3; interface em estrita conformidade com as diretrizes de acessibilidade da WCAG 2.1 nível AA; e disponibilidade mínima acordada em SLA de 99,5% no regime 24x7.

### Tabela 3: Matriz de Desdobramento RNF baseada na Norma ISO/IEC 25010

| Característica ISO 25010 | Subcaracterística de Engenharia | Métrica Operacional Verificável | Exemplo de Requisito Não Funcional Específico |
|---|---|---|---|
| **Eficiência de Desempenho** | Comportamento temporal | Percentil 95 (P95) | O microsserviço de autenticação deve processar requisições JWT em até 200ms sob carga nominal. |
| **Segurança** | Confidencialidade e Integridade | Algoritmo criptográfico mínimo | Todas as chaves e dados sensíveis no PostgreSQL devem utilizar cifragem AES-256 em repouso. |
| **Usabilidade** | Acessibilidade Digital | Padrão W3C / WCAG 2.1 | A interface web institucional deve alcançar 100% de conformidade com os critérios AA de acessibilidade para leitores de tela. |
| **Confiabilidade** | Disponibilidade e Tolerância | Taxa de *Uptime* e MTTR | O barramento de mensageria deve manter 99,9% de disponibilidade mensal, com tempo médio de recuperação inferior a 15 minutos. |
| **Manutenibilidade** | Modularidade e Analisabilidade | Cobertura de Testes e Débito | O repositório deve exigir cobertura de código automatizada mínima de 80% antes de autorizar o *merge* para a *branch main*. |
| **Portabilidade** | Adaptabilidade de Execução | Conteinerização OCI | O sistema deve ser empacotado em imagens de contêiner compatíveis com Docker e orquestráveis via Kubernetes. |

## 2.2 Análise, Priorização e Critérios de Aceitação

Nem todos os requisitos levantados durante as rodadas de elicitação inicial possuem a mesma relevância técnica ou viabilidade econômica.

Uma equipe de engenharia que aceita todas as solicitações sem triagem técnica sucumbe rapidamente ao fenômeno do *Scope Creep* (inchaço descontrolado de escopo), comprometendo prazos, orçamento e estabilidade arquitetural.

A priorização racional assegura que os esforços de engenharia concentrem-se naquilo que maximiza o retorno sobre o investimento e mitiga os riscos centrais da arquitetura.

### O Framework de Priorização MoSCoW

Originado na metodologia DSDM (*Dynamic Systems Development Method*), o framework MoSCoW estabelece fronteiras contratuais precisas entre as entregas indispensáveis de curto prazo e os incrementos que podem aguardar ciclos futuros:

**Tabela 4: Matriz de Decisão e Classificação MoSCoW para o Projeto Acadêmico FESP**

## Critérios de Aceitação e Definição de Comportamento (BDD / Gherkin)

A ambiguidade na linguagem natural é a principal fonte de desalinhamento entre o que o analista documenta e o que a equipe técnica implementa. Descrições como "o sistema deve ser rápido" ou "o módulo antifraude deve ser inteligente" são tecnicamente imprestáveis. Os critérios de aceitação formalizam as condições verificáveis sob as quais uma funcionalidade é declarada concluída e aprovada, utilizando com frequência a estrutura comportamental **Gherkin** (*Given-When-Then*), associada ao *Behavior-Driven Development* (BDD)

**PREVENINDO A AMBIGUIDADE: ANATOMIA DA NOTAÇÃO GHERKIN**

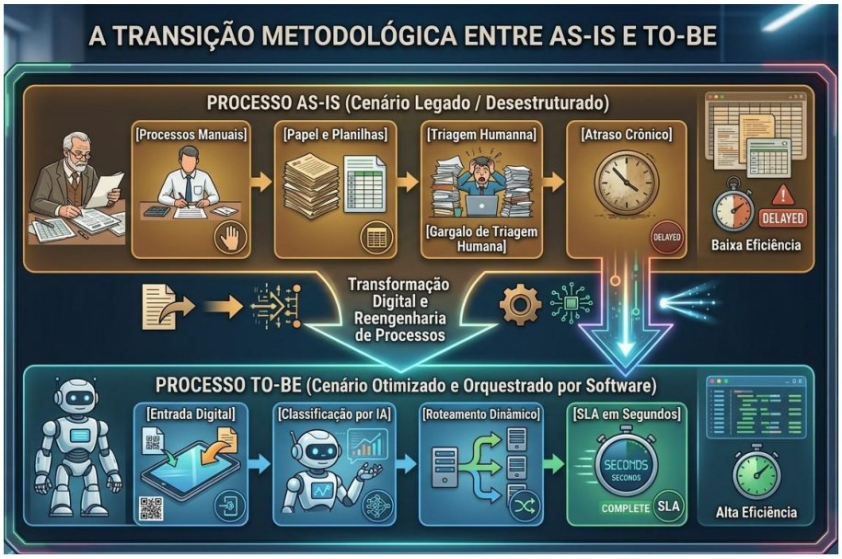
**Espcificação Canônica em Gherkin: Cenário Antifraude e Limites Operacionais**

**Tabela 5: Mapeamento de Engenharia do Cenário de Teste de Aceitação**

| Cláusula Gherkin | Expressão de Negócio | Mapeamento Técnico de Implementação | Asserção de Teste Automatizado |
|---|---|---|---|
| **Dado que** | Usuário autêntico com limite de R$ 5.000,00 | `User.findById(id).getLimit() == 5000.00` | `assertEquals(5000.00, user.getLimit())` |
| **E** | Dispositivo classificado com risco alto | `FraudEngine.evaluateDevice(fingerprint).riskLevel == HIGH` | `assertTrue(deviceRisk.isHighRisk())` |
| **Quando** | Submete transação de R$ 6.200,00 | Chamada HTTP `POST /api/v1/transactions` com payload: `{ amount: 6200.00 }` | Requisição enviada via cliente de teste |
| **Então** | Bloqueia processamento imediato | `TransactionStatus.REJECTED_SUSPICIOUS` | `assertEquals(HttpStatus.FORBIDDEN, response.status)` |
| **E** | Emite código de recusa 'SEC-403' | Retorno do corpo da resposta com `ErrorCode: "SEC-403"` | `assertEquals("SEC-403", response.json().errorCode)` |
| **E** | Dispara token biométrico em até 30s | Publicação de evento no RabbitMQ/Kafka: `auth.biometrics.challenge` | `verify(notificationService, timeout(30000)).sendToken()` |

## 2.3 Modelagem de Processos de Negócio com BPMN

A notação **BPMN 2.0** (*Business Process Model and Notation*) serve como linguagem padronizada entre a estratégia corporativa e a engenharia de software. Ela previne que a equipe de tecnologia cometa o erro clássico de automatizar processos inerentemente ineficientes, burocráticos ou disfuncionais. A modelagem deve sempre distinguir a análise da situação vigente (**As-Is**) da projeção otimizada mediada pela tecnologia (**To-Be**).




ANEXO B — Fonte original, página 10 de 19. Reprodução integral FESP, 2026.

## Elementos Básicos da Notação BPMN 2.0

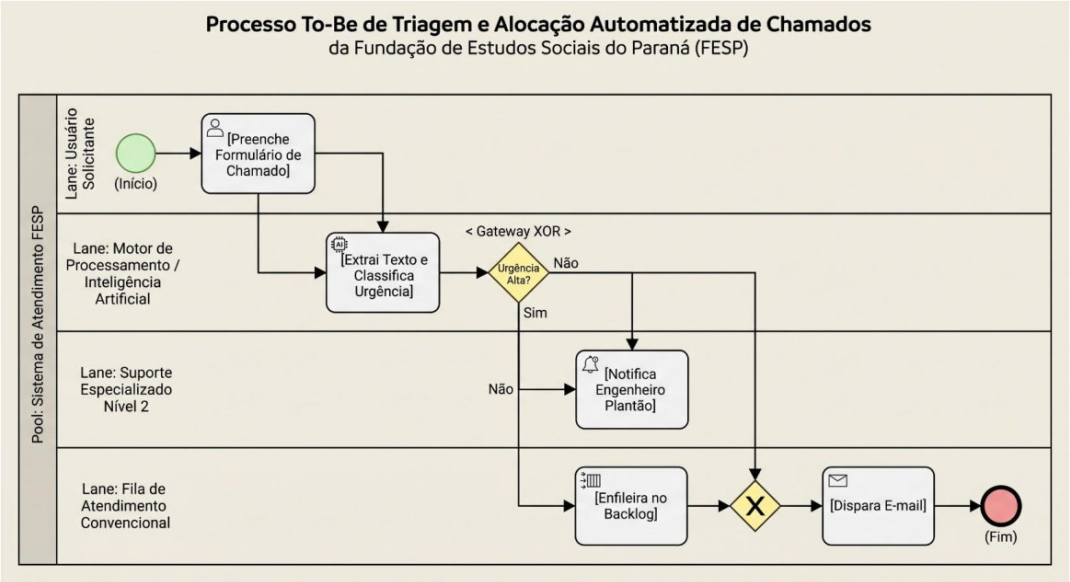
Para expressar a dinâmica de fluxos operacionais sem ambiguidades, o padrão BPMN define cinco categorias essenciais de modelagem:

1. **Pools:** Entidades corporativas, organizações ou sistemas autônomos participantes do processo.
2. **Lanes:** Raias internas de segregação de funções, papéis de usuários, departamentos ou módulos de software responsáveis pela execução das atividades.
3. **Eventos:** Marcadores temporais que sinalizam mudanças de estado no processo:
    - *Início:* Representado por um círculo com borda fina.
    - *Intermediário:* Representado por um círculo duplo, indicando espera por mensagem, tempo ou gatilho externo.
    - *Fim:* Representado por um círculo de borda espessa, indicando o encerramento do ciclo de execução.
4. **Atividades / Tarefas:** Retângulos com bordas arredondadas indicando uma ação de trabalho concreta que deve ser desempenhada por um ator humano ou por um serviço computacional.
5. **Gateways:** Losangos de controle que governam a divergência e a convergência de caminhos no fluxo de trabalho com base em condições lógicas:
    - *Exclusivo (XOR - marcador 'X'):* Bifurcação condicional estrita em que apenas uma alternativa de saída pode ser seguida.
    - *Paralelo (AND - marcador '+'):* Todas as ramificações de saída são disparadas de forma concorrente e simultânea, sem necessidade de validação condicional.
    - *Inclusivo (OR - marcador 'O'):* Uma ou mais ramificações de saída podem ser ativadas concomitantemente, dependendo da avaliação de condições independentes.

## Diagrama BPMN: Processo To-Be de Triagem e Alocação Automatizada de Chamados

O diagrama a seguir especifica o fluxo otimizado da Fundação de Estudos Sociais do Paraná (FESP), integrando a requisição do usuário, a análise via modelo preditivo de Inteligência Artificial e o direcionamento estruturado de filas técnicas:

**Pool: Sistema de Atendimento FESP**




```mermaid
flowchart TD
  subgraph Pool_Sistema_Atendimento_FESP["Pool: Sistema de Atendimento FESP"]
    subgraph Lane_Usuario["Lane: Usuário Solicitante"]
      Start((Início)) --> A1[Preenche Formulário de Chamado]
    end
    subgraph Lane_Motor_IA["Lane: Motor de Processamento / Inteligência Artificial"]
      A1 --> B1[Extrai Texto e Classifica Urgência]
      B1 --> G1{Urgência Alta?}
    end
    subgraph Lane_Suporte_N2["Lane: Suporte Especializado Nível 2"]
      G1 -- Sim --> C1[Notifica Engenheiro de Plantão]
    end
    subgraph Lane_Fila_Convencional["Lane: Fila de Atendimento Convencional"]
      G1 -- Não --> D1[Enfileira no Backlog Padrão]
      C1 --> M1((Merge))
      D1 --> M1
      M1 --> D2[Dispara E-mail de Confirmação]
      D2 --> EndEvent(((Fim)))
    end
  end
```

ANEXO B — Fonte original, página 12 de 19. Reprodução integral FESP, 2026.

**Tabela 6: Catálogo Semântico dos Elementos BPMN 2.0 e suas Implicações de Software**

| Elemento Gráfico | Notação BPMN | Semântica de Processo de Negócio | Mapeamento no Código de Software |
|---|---|---|---|
| **Pool** | Retângulo horizontal delimitador | Fronteira institucional ou sistema independente. | Domínio isolado, microsserviço ou contêiner autocontido. |
| **Lane** | Subdivisão interna horizontal | Papel de usuário, departamento ou subsistema. | Camada de permissão (RBAC), fila de serviço ou módulo. |
| **Tarefa de Serviço** | Caixa com engrenagem | Ação executada automaticamente por código | Chamada a uma função, API REST ou rotina *backend*. |
| **Gateway XOR (X)** | Losango com 'X' | Decisão mutuamente exclusiva (Sim / Não) | Instrução de fluxo condicional if...else ou switch. |
| **Gateway AND (+)** | Losango com '+' | Execução estritamente simultânea / paralela | Concorrência assíncrona, *threads* paralelas ou *Fork-Join*. |
| **Gateway OR (O)** | Losango com 'O' | Múltiplas ramificações ativadas conforme condições | Avaliação de múltiplos predicados independentes em cascata. |

## 2.4 Modelagem Básica de Sistemas com UML

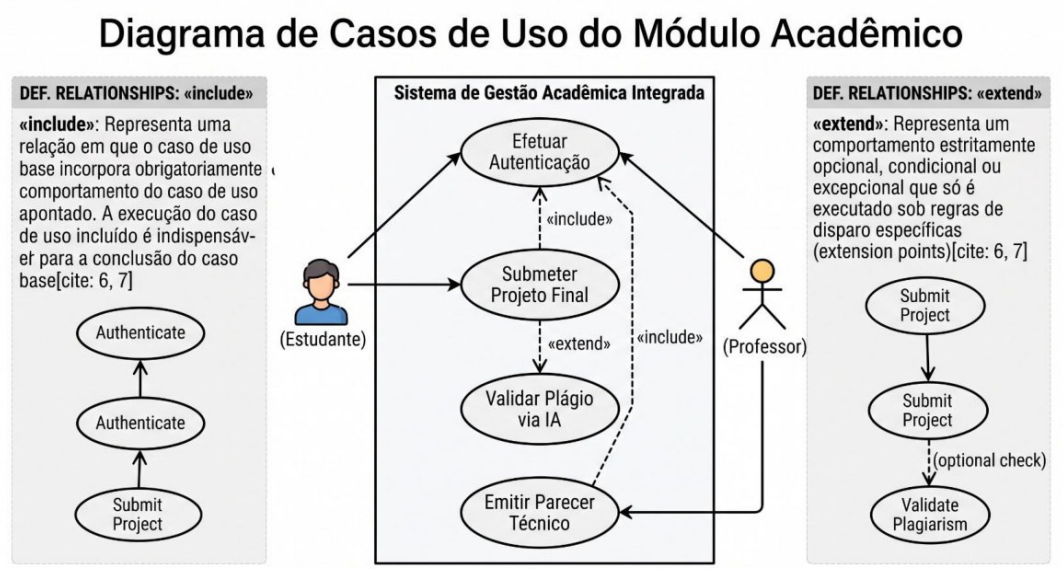
A **Unified Modeling Language (UML 2.5)** permanece como a principal notação técnica padronizada pela OMG (*Object Management Group*) para expressar a arquitetura lógica, os esquemas estruturais e os comportamentos de sistemas orientados a objetos. Nesta fase introdutória, destacam-se os **Diagramas de Casos de Uso** (visão externa funcional e limites de escopo) e os **Diagramas de Classes Simplificados** (visão estática estrutural e contratos de domínio).

### Diagrama de Casos de Uso

Evidencia as fronteiras funcionais do sistema, os atores externos que interagem com a solução e os relacionamentos de inclusão obrigatória («include») e extensão condicional («extend»):

- **«include»:** Representa uma relação em que o caso de uso base incorpora obrigatoriamente o comportamento do caso de uso apontado. A execução do caso de uso incluído é indispensável para a conclusão do caso base.
- **«extend»:** Representa um comportamento estritamente opcional, condicional ou excepcional que só é executado sob regras de disparo específicas (*extension points*).

**Diagrama de Casos de Uso do Módulo Acadêmico**




```mermaid
flowchart LR
  subgraph SGB["Fronteira do Sistema: Gestão Acadêmica"]
    UC1([Efetuar Autenticação])
    UC2([Submeter Projeto Final])
    UC3([Validar Plágio via IA])
    UC4([Emitir Parecer Técnico])
  end

  ActorEstudante((Estudante)) --> UC1
  ActorEstudante --> UC2
  ActorProfessor((Professor)) --> UC1
  ActorProfessor --> UC4

  UC2 -.->|«include»| UC1
  UC2 -.->|«extend»| UC3
  UC4 -.->|«include»| UC1

  ActorIA((Motor de IA)) --> UC3
```

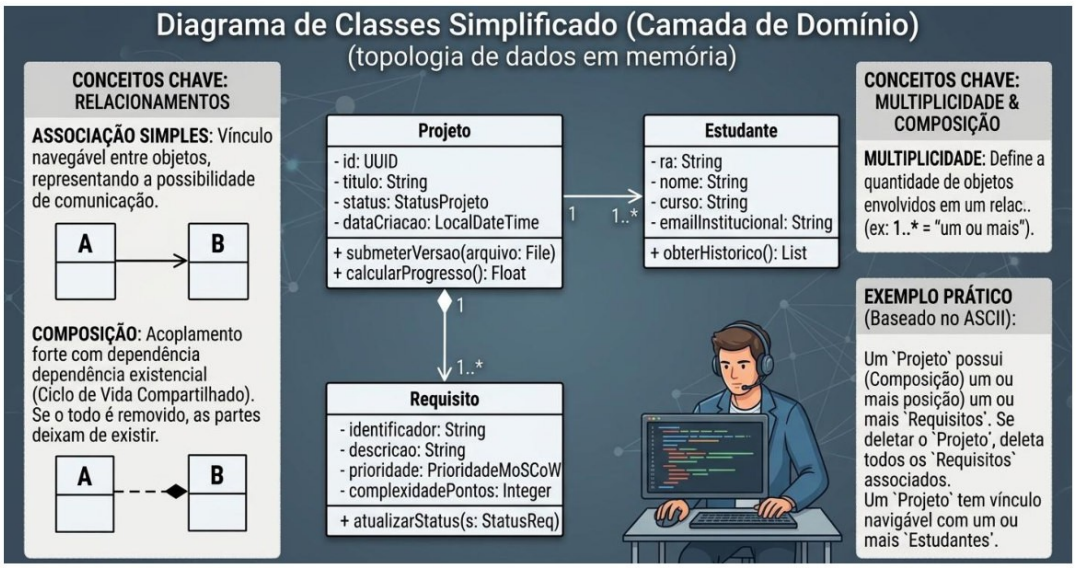
**Tabela 7: Dicionário Técnico de Especificação dos Casos de Uso**

| Identificador | Caso de Uso | Atores Primários | Tipo de Dependência | Regra de Disparo / Condição Arquitetural |
|---|---|---|---|---|
| **UC-01** | Efetuar Autenticação | Estudante, Professor | Inclusão («include») | Obrigatório em todas as transações que exigem identificação de perfil e controle de sessão JWT |
| **UC-02** | Submeter Projeto Final | Estudante | Caso de uso base | O estudante anexa o arquivo digital e formaliza a entrega antes do prazo de encerramento da disciplina. |
| **UC-03** | Validar Plágio via IA | Motor de IA (Ator Secundário) | Extensão («extend») | Acionado condicionalmente se o arquivo for submetido em formato PDF/DOCX elegível para análise de similaridade textual. |
| **UC-04** | Emitir Parecer Técnico | Professor | Caso de uso base | O orientador analisa o relatório acadêmico, insere notas e homologa o parecer final da banca avaliadora. |

## Diagrama de Classes Simplificado (Camada de Domínio)

Modela a topologia de dados em memória e as relações de associação, multiplicidade e composição entre os objetos do sistema.

Uma associação simples indica vínculo navegável; uma **composição** (representada pelo losango preenchido na extremidade do todo) define um acoplamento forte em que as partes dependem existencialmente do elemento proprietário: se o Projeto for removido, todos os Requisitos a ele associados deixam de existir.




```
classDiagram
  class Projeto {
    -UUID id
    -String titulo
    -StatusProjeto status
    -LocalDateTime dataCriacao
    +submeterVersao(File arquivo)
    +calcularProgresso() Float
  }

  class Estudante {
    -String ra
    -String nome
    -String curso
    -String emailInstitucional
    +obterHistorico() List
  }

  class Requisito {
    -String identificador
    -String descricao
    -PrioridadeMoSCoW prioridade
    -Integer complexidadePontos
    +atualizarStatus(StatusReq s)
  }

  Projeto "1" --> "1..*" Estudante : alocado para
  Projeto "1" *-- "1..*" Requisito : composto por
```

**Tabela 8: Dicionário Estrutural de Dados da Camada de Domínio**

| Classe de Entidade | Atributo e Visibilidade | Tipo de Dado Primitivo / Enun | Restrição de Integridade / Regra de Negócio |
|---|---|---|---|
| **Projeto** | - id: UUID | java.util.UUID | Identificador universal único gerado automaticamente na instanciação. |
| **Projeto** | - titulo: String | java.lang.String | Campo obrigatório; tamanho de 5 a 120 caracteres alfanuméricos. |
| **Projeto** | - status: StatusProjeto | Enum (EM_ANDAMENTO, HOMOLOGADO) | Máquina de estados que controla se novas versões podem ser enviadas. |
| **Estudante** | - ra: String | java.lang.String | Registro Acadêmico institucional único; chave candidata indexada. |
| **Estudante** | - email Institucional: String | java.lang.String | Formato padrão @fesp.edu.br validado via expressão regular (Regex). |
| **Requisito** | - prioridade: Prioridade MoSCoW | Enum (MUST, SHOULD, COULD, WONT) | Define o peso de execução no *backlog* e a ordenação de entrega. |
| **Requisito** | - complexidade Pontos: Integer | java.lang.Integer | Escala baseada na sequência de Fibonacci modificada (1, 2, 3, 5, 8, 13). |

## O Fio de Ouro da Engenharia de Software: Rastreabilidade Bidirecional

Um artefato isolado é apenas burocracia documental inerte; artefatos rastreáveis formam a verdadeira Engenharia de Software.

A rastreabilidade bidirecional (*Forward and Backward Traceability*) garante que toda linha de código em produção responda indiscutivelmente a uma necessidade humana e de negócio formalmente validada.

Inversamente, ela permite auditar como cada alteração de requisito impacta diagramas de processos, modelos estruturais de classes e suítes de testes automatizados.

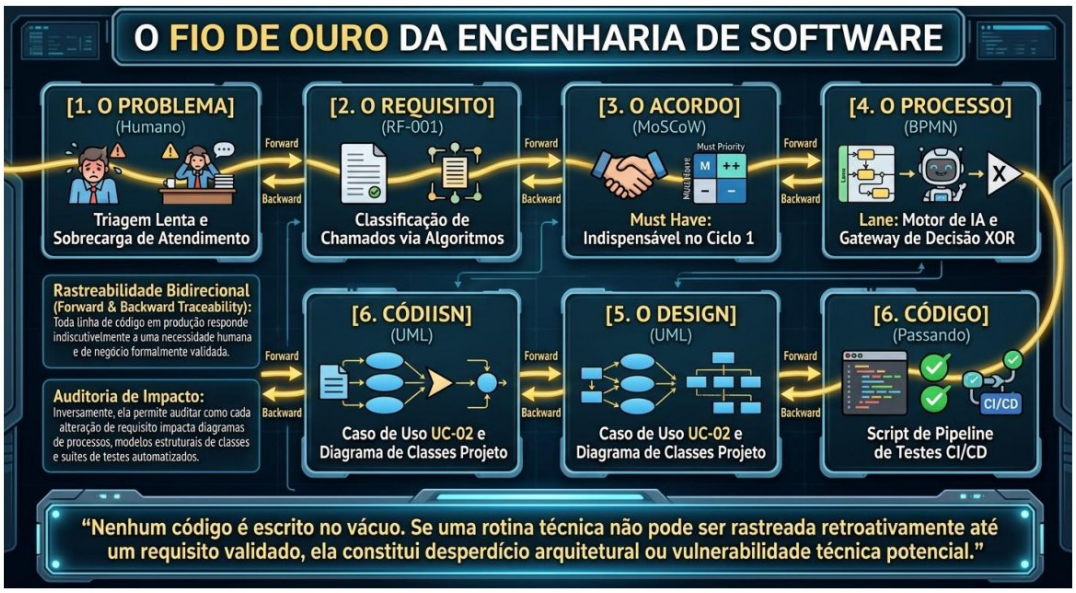



**Tabela 9: Matriz de Rastreabilidade Bidirecional Ponta a Ponta**

TABELA 9: MATRIZ DE RASTREABILIDADE BIDIRECIONAL PONTA A PONTA

| ID Problema | Requisito Funcional (RF) | Prioridade MoSCoW | Elemento de Processo (BPMN 2.0) | Artefato de Design (UML 2.5) | Critério de Aceitação (Gherkin) | Artefato Executável / Teste |
|---|---|---|---|---|---|---|
| **PRB-01** (Triagem lenta de chamados) | **RF-001:* O sistema deve classificar a urgência textual de solicitações via IA | **Must Have** (Indispensável no MVP) | Lane *Motor de IA*, Tarefa *Extrai Texto*, Gateway XOR | Caso de Uso *Validar Plágio/Urgência via IA* | `Dado que o chamado é crítico, Quando a IA processa, "Então notifica N2" | ``testUrgencyInference()" passando no pipeline de CI/CD. |
| **PRB-02** (Inconsistência de entregas) | **RF-002:* O sistema deve registrar e versionar entregas acadêmicas com hash criptográfico. | **Must Have** (Requisito regulatório institucional) | Lane *Usuário Solicitante*, Atividade *Preenche Formulário* | Caso de Uso *Submeter Projeto Final*; Classe `Projeto` | `Dado que o estudante é ativo, Quando anexa arquivo, "Então salva UUID" | ``ProjetoService.submeterVersao()` com teste unitário. |
| **PRB-03** (Atraso na correção docente) | **RF-003:* O sistema deve disponibilizar painel de avaliação e parecer técnico para orientadores. | **Should Have** (Contornável via relatórios manuais) | Lane *Suporte/Docente*, Atividade *Dispara E-mail* | Caso de Uso *Emitir Parecer Técnico*; Classe `Estudante` | `Dado que a versão é válida, Quando o orientador avalia, "Então emite nota" | ``DocenteController.emitirParecer()` homologado em homologação. |

Modern Management Solutions

## Desdobramento Técnico para os Cursos Integrados

A aplicação prática dos fundamentos abordados nesta Aula 2 manifesta-se de forma complementar conforme o perfil formativo dos estudantes:

- **Bacharelado em Sistemas de Informação (BSI):** Foco prioritário na governança de partes interessadas e no alinhamento estratégico via BPMN 2.0, assegurando que o software desenhado resolva gargalos operacionais e gere valor econômico comprovado para a organização contratante.
- **Tecnologia em Análise e Desenvolvimento de Sistemas (TADS):** Concentração no refinamento da especificação Gherkin e na modelagem de classes de domínio em UML, transformando contratos de casos de uso em estruturas de banco de dados relacionais e endpoints de API RESTful de alta eficiência.
- **Engenharia de Software:** Domínio absoluto da matriz de rastreabilidade bidirecional, controle formal de mudanças contra o *Scope Creep* via MoSCoW e estabelecimento de métricas quantitativas de qualidade baseadas na norma ISO/IEC 25010.
- **Tecnologia em Inteligência Artificial:** Estruturação de requisitos estocásticos em sistemas híbridos, identificando as fronteiras entre lógica determinística de software tradicional e modelos de aprendizado de máquina, com especial ênfase na monitoração contínua de latência e degradação de inferências.

## Referências Bibliográficas Fundamentais


- ANDERSON, David J. **Kanban: Mudança Tecnológica Bem-sucedida para Seu Negócio de Tecnologia**. Sequim: Blue Hole Press, 2010.
- BECK, Kent et al. **Manifesto para o Desenvolvimento Ágil de Software**. Snowbird: Agile Alliance, 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html.
- BOEHM, Barry W. **Software Engineering Economics**. Englewood Cliffs: Prentice-Hall, 1981.
- FOWLER, Martin. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3. ed. Boston: Addison-Wesley Professional, 2003.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models**. Genebra: ISO/IEC, 2011
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering**. Genebra: ISO/IEC/IEEE, 2018.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH / McGraw-Hill, 2021.
- SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum: As Regras do Jogo**. Scrum.org, 2020. Disponível em: https://scrumguides.org[cite: 7].
- SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019

# ANEXO C — Aula 3 — Metodologias Ágeis, Engenharia de Fluxo e Planejamento

Reprodução integral do texto institucional sobre Manifesto Ágil, Cynefin, Scrum, Kanban, métricas, Lei de Little, INVEST e Planning Poker. A adaptação ao ConectaIA está na Etapa 3.

Arquivo de origem: Texto da Aula 3 - Eng Software.pdf. Total de páginas originais: 18. Reprodução sem cortes ou alterações de conteúdo.

## Engenharia de Software e Desenvolvimento Ágil de Soluções

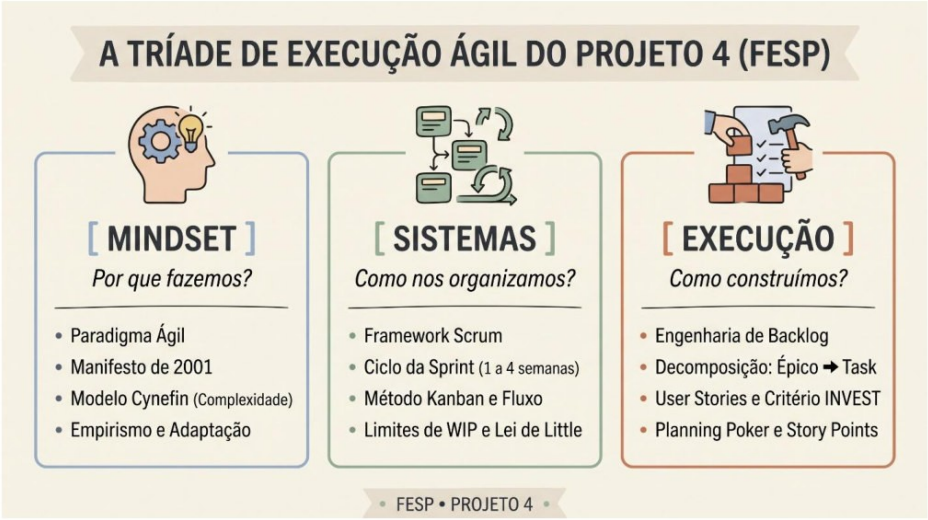
- **Cursos:** 2º Semestre - Bacharelado em Sistemas de Informação (BSI), Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), Engenharia de Software e Tecnologia em Inteligência Artificial

# Aula 3: Metodologias Ágeis, Engenharia de Fluxo e Planejamento

*Dominar os fundamentos da agilidade, a mecânica empírica do Scrum, a física de fluxo do Kanban e o rigor analítico da decomposição de requisitos para viabilizar a entrega contínua de software funcional.*

## O Playbook de Execução para o Projeto 4

O desenvolvimento profissional de software moderno opera na convergência entre cultura organizacional, modelos de controle empírico e técnicas de decomposição analítica.
Para o desenvolvimento prático do Projeto da disciplina (Projeto 4), a governança técnica estrutura-se sobre uma tríade sequencial e interdependente, na qual cada camada sustenta operacionalmente a seguinte:

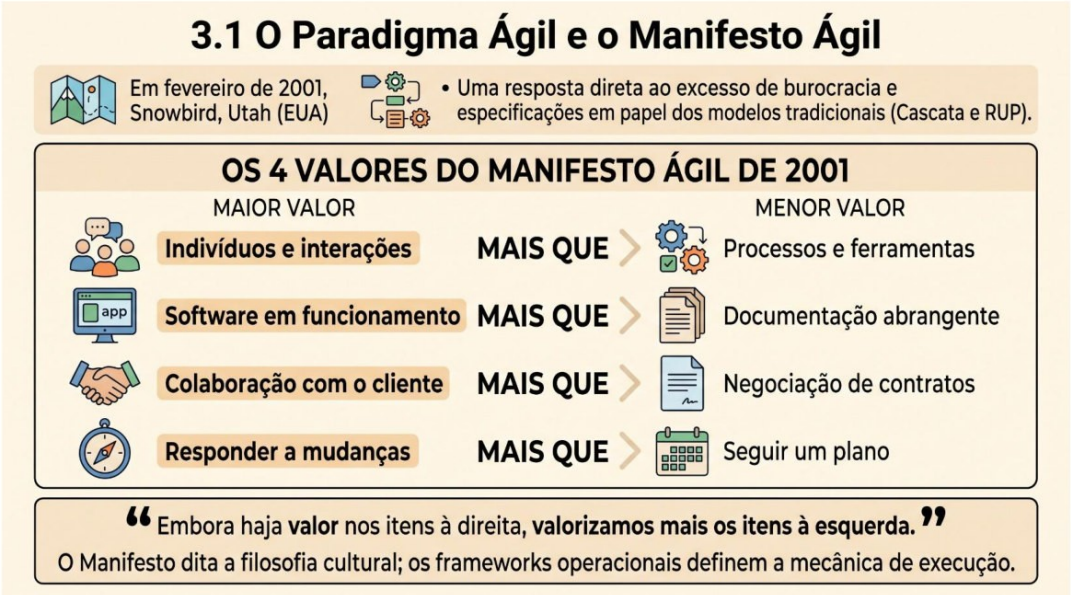



A visão sinóptica a seguir sintetiza a integração desses componentes operacionais, servindo como mapa de navegação didática para esta aula:

## 3.1 O Paradigma Ágil e o Manifesto Ágil

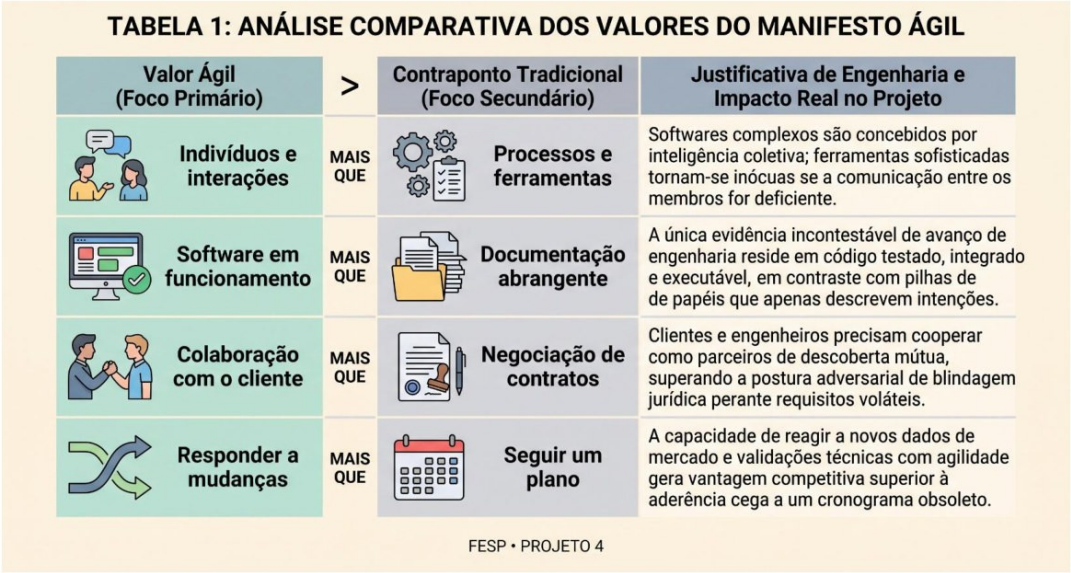
O movimento ágil consolidou-se como uma resposta direta às deficiências causadas pelo excesso de prescrição dos modelos burocráticos tradicionais (como o **Rational Unified Process - RUP** e os modelos cascata com auditorias documentais infindáveis), que tentavam tratar o desenvolvimento de software como uma linha de montagem determinística industrial .
Durante as décadas de 1980 e 1990, a crença dominante postulava que o sucesso de um sistema dependia da exaustão prévia de especificações em papel.
Essa premissa revelou-se contraproducente quando confrontada com ambientes econômicos dinâmicos, volatilidade contínua de mercados e a emergência da computação distribuída em rede, onde os requisitos sofrem mutações antes mesmo que o design inicial seja concluído.

Em fevereiro de 2001, dezessete signatários reuniram-se em Snowbird, Utah, e redigiram o **Manifesto para o Desenvolvimento Ágil de Software**, estabelecendo quatro valores centrais:




A interpretação técnica desse compromisso exige rigor: o manifesto não elimina o lado direito da balança, visto que ferramentas, documentação arquitetural, contratos corporativos e planos estratégicos continuam sendo indispensáveis para a estabilidade da engenharia.
A revolução reside em submeter tais elementos instrumentais ao propósito final da criação de software, impedindo que a burocracia documental paralise a capacidade de entrega contínua de valor de negócio.

**Tabela 1: Análise Comparativa dos Valores do Manifesto Ágil**




**Os Doze Princípios Operacionais da Agilidade**

A filosofia ágil apoia-se em doze princípios operacionais estabelecidos pelos signatários de 2001, que norteiam a governança de engenharia e os processos de tomada de decisão técnica:

1. **Satisfação do Cliente**: Nossa maior prioridade é satisfazer o cliente por meio da entrega contínua e antecipada de software com valor agregado.
2. **Acolhimento a Mudanças**: Mudanças de requisitos são bem-vindas, mesmo em fases avançadas do desenvolvimento; processos ágeis tiram proveito das alterações para gerar vantagem competitiva
3. **Entregas Frequentes**: Entregar software funcional frequentemente, na escala de poucas semanas a poucos meses, com preferência à menor escala de tempo.
4. **Trabalho Conjunto**: Profissionais de negócio e engenheiros de software devem trabalhar juntos, diariamente, durante todo o ciclo do projeto.
5. **Indivíduos Motivados**: Construir projetos em torno de indivíduos motivados, fornecendo o ambiente, o ferramental e a confiança necessários para que realizem seu trabalho.
6. **Comunicação Direta**: O método mais eficiente e eficaz de transmitir informações tanto para dentro quanto dentro de uma equipe de desenvolvimento é a comunicação face a face ou síncrona.
7. **Métrica Primária**: Software funcional é a medida primária de progresso de um projeto de tecnologia.
8. **Ritmo Sustentável**: Processos ágeis promovem o desenvolvimento sustentável; patrocinadores, desenvolvedores e usuários devem manter um ritmo de trabalho constante indefinidamente.
9. **Excelência Técnica:** A contínua atenção à excelência técnica e ao bom design arquitetural aprimora a agilidade do sistema e mitiga a formação de débito técnico.
10. **Simplicidade**: A simplicidade — definida como a arte de maximizar a quantidade de trabalho não realizado — é fundamental para evitar desperdício estrutural.
11. **Equipes Auto-organizadas**: As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizadas e dotadas de autonomia decisória.
12. **Melhoria Contínua**: Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e refina e ajusta seu comportamento de acordo.

**O Modelo Cynefin e a Teoria da Complexidade**

A adoção de métodos ágeis apoia-se na teoria do Modelo Cynefin , desenvolvido pelo pesquisador britânico Dave Snowden, que classifica os problemas e os contextos operacionais em quatro domínios de causalidade distintos: Simples/Claro, Complicado, Complexo e Caótico.

ANEXO C — Fonte original, página 4 de 18. Reprodução integral FESP, 2026.

**Tabela 2: Matriz dos Domínios Cynefin Aplicada à Engenharia de Software**

**TABELA 2: MATRIZ DOS DOMÍNIOS CYNEFIN APLICADA À ENGENHARIA DE SOFTWARE**

| Domínio Cynefin | Natureza da Relação de Causalidade | Dinâmica Operacional e Ação Recomendada | Adequação Metodológica de Software | Exemplo Real no Ecossistema FESP |
|---|---|---|---|---|
| **Simples / Claro** | Relação de causa e efeito linear, óbvia e evidente para qualquer operador. | **Perceber → Categorizar → Responder**: aplicação de procedimentos operacionais padrão (SOP). | **Modelos clássicos** preditivos e cascata operam com sucesso neste domínio. | **Configuração** de rotas de rede estáticas ou cadastro manual de tabelas de feriados no banco de dados. |
| **Complicado** | Relação de causa e efeito cognoscível, porém separada no espaço e tempo, demandando análise de especialistas. | **Perceber → Analisar → Responder**: diagnósticos técnicos aprofundados para definir boas práticas. | **Métodos analíticos** estruturados, RUP refinado, modelagem formal de dados e engenharia reversa. | Otimização de planos de execução de queries SQL complexas ou dimensionamento de clusters computacionais. |
| **Complexo** | Causa e efeito intrinsecamente entrelaçadas, que só podem ser compreendidas em retrospectiva histórica. | **Sondar → Perceber → Responder**: experimentos curtos, hipóteses testáveis e adaptação frequente. | **Métodos Ágeis** (Scrum, Kanban, Lean): o ciclo iterativo é o único capaz de domar a incerteza. | Projetos integrados de IA, BSI e TADS operam 90% do tempo neste domínio (comportamento de usuários, model drift). |
| **Caótico** | Ausência transitória de relações ordenadas de causa e efeito; turbulência e colapso imediato. | **Agir → Perceber → Responder**: ações imediatas de contenção de danos para restaurar a estabilidade. | **Gestão de Incidentes Críticos**, comitês de crise (war rooms) e protocolos de contenção de desastres. | Incidente severo de vazamento de credenciais em produção ou ataque massivo de ransomware paralisando servidores. |

FESP • PROJETO 4

O desenvolvimento contemporâneo de soluções computacionais opera predominantemente no domínio Complexo, exigindo abordagens empíricas baseadas em experimentação rápida, inspeção contínua e adaptação frequente.
Em sistemas modernos que acoplam pipelines de inteligência artificial, regras corporativas voláteis e múltiplos perfis de usuários, o comportamento do sistema não decorre do empilhamento analítico prévio de requisitos, mas sim da reação observada no ambiente real de operação.

## 3.2 O Framework Scrum: Arquitetura de Papéis, Eventos e Artefatos

O  Scrum , sistematizado por Ken Schwaber e Jeff Sutherland, constitui um framework adaptativo projetado para gerar valor por meio de soluções transparentes para problemas complexos.
Longe de ser um conjunto rígido de receitas prescritivas, o Scrum fornece fronteiras estruturais mínimas que viabilizam o florescimento de processos emergentes.

### O Controle Empírico de Processos

O Scrum apoia-se formalmente no tripé do controle empírico, cujos pilares sustentam a integridade e a previsibilidade técnica do desenvolvimento:

**Arquitetura de Papéis no Scrum Team (*Accountabilities*)**

O Scrum aboliu hierarquias corporativas internas na equipe de desenvolvimento para instituir o Scrum Team unificado: uma unidade coesa, auto-organizável e multidisciplinar de profissionais comprometidos conjuntamente com o mesmo objetivo de produto.

Não existem subtimes ou castas funcionais; todos os membros compartilham a mesma responsabilidade ética e técnica pelo sucesso da entrega.

ANEXO C — Fonte original, página 6 de 18. Reprodução integral FESP, 2026.

* **Product Owner (PO):** É o responsável direto por maximizar o valor econômico do produto decorrente do trabalho do time. Detém a autoridade e a responsabilidade final pela gestão e ordenação do Product Backlog, atuando como a ponte entre o negócio, os usuários e a equipe técnica. Suas decisões tornam-se visíveis no conteúdo e na sequência de prioridades do backlog, exigindo que toda a organização respeite sua autonomia estratégica.
* **Scrum Master (SM):** Atua como líder servidor e responsável pela eficácia global do Scrum Team. Auxilia o time no cumprimento das práticas, atua na remoção de impedimentos organizacionais que bloqueiam a fluidez do trabalho técnico e protege a equipe de interferências externas indevidas. Não atua como chefe funcional ou gerente de projetos tradicional, mas como facilitador de maturidade técnica e comportamental.
* **Desenvolvedores**: Profissionais multidisciplinares (engenheiros de software, arquitetos de dados, especialistas em interface e analistas de QA) que assumem o compromisso de construir qualquer aspecto de um Incremento utilizável e testado a cada Sprint. Detêm a soberania sobre o "como" construir, definindo as abordagens arquiteturais, estimativas de esforço e decomposições de tarefas necessárias para atingir o objetivo técnico.

> Diretriz de Aplicação no Projeto 4 (FESP): Na execução prática do projeto acadêmico integrado, o grupo de estudantes assume integralmente o papel coletivo de Desenvolvedores (revezando a facilitação do Scrum Master entre os membros), enquanto o professor/orientador assume a prerrogativa do cliente e Product Owner , validando critérios de aceitação e ordenando o backlog da disciplina .

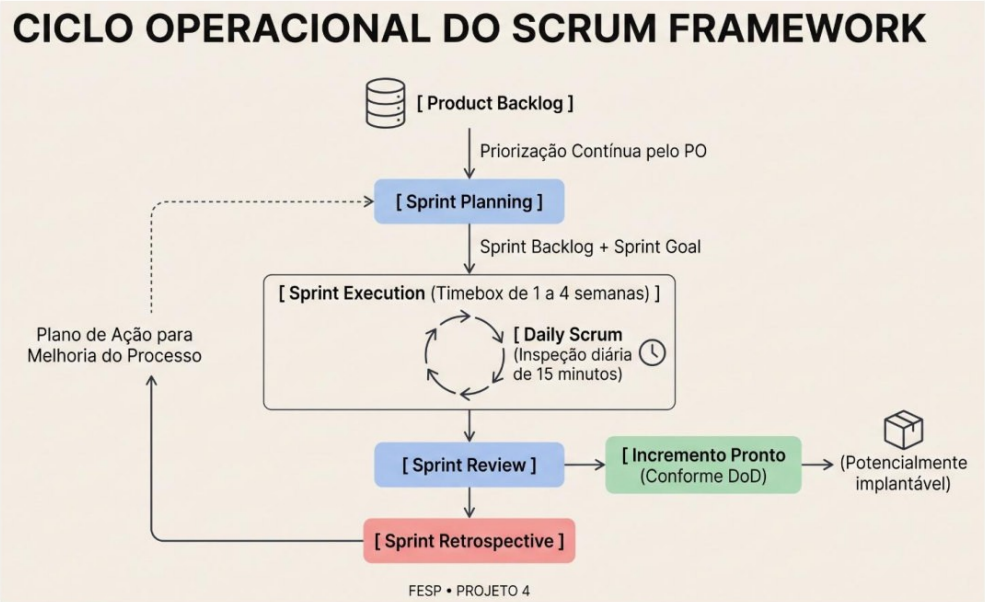
**Ciclo Operacional e Eventos Formais do Scrum**

Cada evento dentro do framework Scrum é concebido formalmente como um contêiner com tempo limite estabelecido (***timebox***), cujo propósito central consiste em possibilitar a inspeção e a adaptação transparente dos artefatos de desenvolvimento:

```
flowchart TD
  PB[(Product Backlog)] -->|Priorização Contínua pelo PO| SP[Sprint Planning]
  SP -->|Sprint Backlog + Sprint Goal| SE[Sprint Execution: 1 a 4 semanas]

  subgraph Execution_Loop [Ciclo Diário de Execução]
    SE --> DS[Daily Scrum: 15 min]
    DS --> SE
  end

  SE --> SR[Sprint Review]
  SR --> Inc[Incremento Pronto - Conforme DoD]
  SR --> Ret[Sprint Retrospective]
  Ret -->|Plano de Melhoria do Processo| SP
```




* **Sprint**: O evento contêiner de duração regular (1 a 4 semanas) no qual as ideias são transformadas em valor prático. Durante a Sprint, o escopo permanece protegido contra alterações que coloquem em risco a meta acordada. Uma nova Sprint inicia-se imediatamente após a conclusão da anterior.
* **Sprint Planning**: Sessão no início da Sprint em que o time define coletivamente três tópicos estruturantes:
    1. **Por que a Sprint é valiosa**? Formalização da Meta da Sprint (Sprint Goal).
    2. **O que pode ser entregue**? Seleção dos itens prioritários do Product Backlog com base na capacidade real do time.
    3. **Como o trabalho será realizado**? Decomposição técnica dos itens selecionados em tarefas operacionais e arquiteturais.
* **Daily Scrum**: Alinhamento diário de no máximo 15 minutos entre os desenvolvedores para inspecionar o progresso rumo ao Sprint Goal e adaptar o plano do dia. Não se trata de uma reunião de prestação de contas à liderança, mas de um sincronismo tático entre pares para identificar bloqueios técnicos imediatos.
* **Sprint Review**: Sessão colaborativa com partes interessadas para demonstrar o Incremento gerado e debater as próximas prioridades do produto. Exige a apresentação de software executável e validado em ambiente representativo, banindo demonstrações teóricas ou baseadas em slides estáticos.
* **Sprint Retrospective**: Reunião interna da equipe para avaliar as relações interpessoais, processos, ferramentas e a **Definição de Pronto (DoD),** estabelecendo melhorias práticas imediatas para o ciclo seguinte. É o mecanismo que materializa a melhoria contínua da engenharia.

**Tabela 3: Matriz Operacional dos Artefatos do Scrum e seus Compromissos**

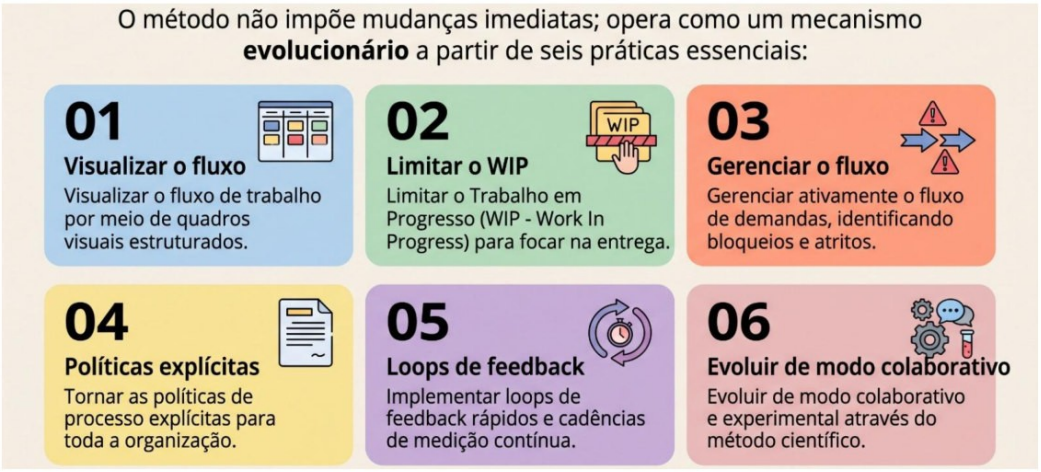
| Artefato Formal | Definição Estrutural de Engenharia | Compromisso Obrigatório (Commitment) | Critério de Integridade Técnica |
|---|---|---|---|
| **Product Backlog** | Lista ordenada e emergente de tudo o que é necessário para aperfeiçoar o produto. | **Meta do Produto (Product Goal):** O estado futuro de longo prazo que o backlog visa construir. | Deve ser dinâmico, refinado continuamente e mantido sob custódia transparente do PO. |
| **Sprint Backlog** | Conjunto de itens selecionados do Product Backlog somado ao plano técnico de entrega. | **Meta da Sprint (Sprint Goal):** O objetivo unificado e inegociável assumido pelo time no ciclo. | Pertence exclusivamente aos desenvolvedores; atualizado diariamente durante a execução. |
| **Incremento** | Passo concreto em direção à Meta do Produto, integrado à totalidade dos incrementos passados. | **Definição de Pronto (Definition of Done - DoD):** Critérios formais de qualidade técnica exigidos. | Se um item não cumpre 100% da DoD, não pode ser lançado nem demonstrado na Sprint Review. |

## 3.3 Método Kanban e Engenharia de Gestão de Fluxo

Enquanto o Scrum estrutura-se em iterações com escopo congelado durante o timebox, o Kanban apoia-se no fluxo contínuo. Derivado dos princípios do Sistema Toyota de Produção (**Toyota Production System - TPS**) e da manufatura enxuta (**Lean**), e adaptado ao trabalho intelectual da computação por David J. Anderson, o Kanban busca expor gargalos e otimizar o fluxo de entrega por meio de um sistema puxado (**pulled system**).

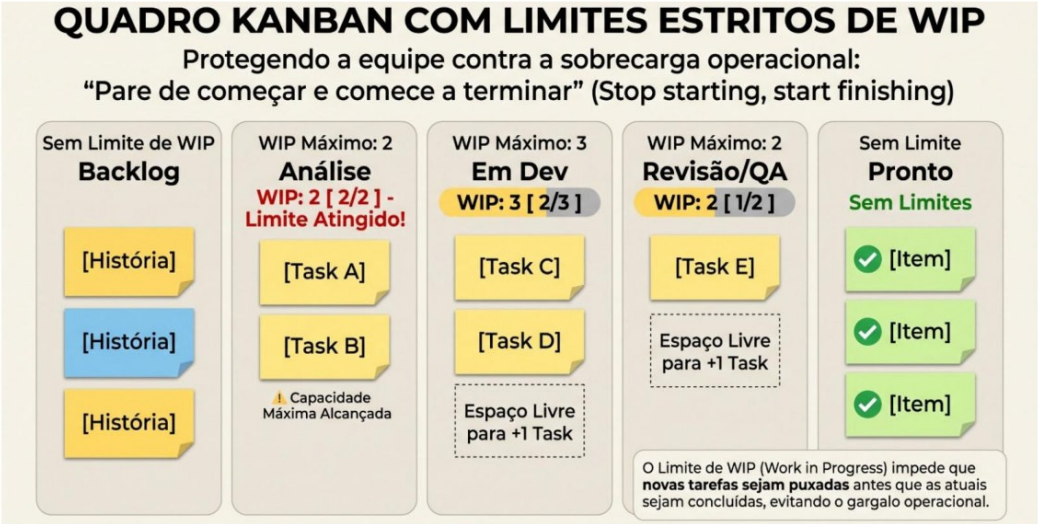
### As Seis Práticas Fundamentais do Kanban

O método não impõe mudanças estruturais imediatas em papéis ou hierarquias; opera como um mecanismo evolucionário que transforma os processos vigentes a partir de seis práticas essenciais:

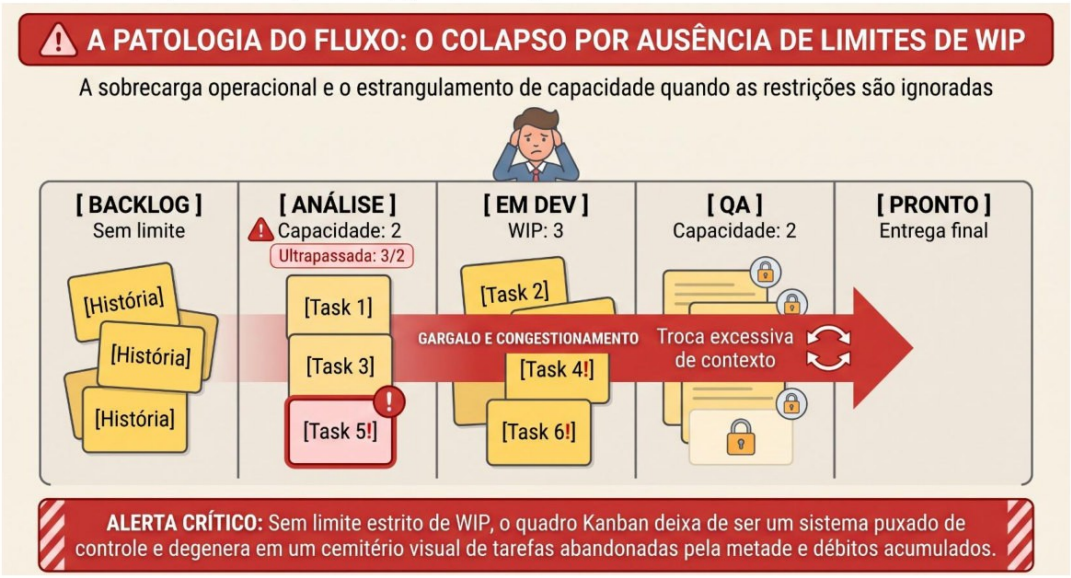


**Gestão Visual e a Dinâmica dos Limites de WIP (Work In Progress)**

A imposição de limites de trabalho em progresso é a regra de ouro que protege a equipe contra a sobrecarga operacional e o estrangulamento de capacidade . Seu fundamento reside na máxima da engenharia enxuta: **"Pare de começar e comece a terminar"** (***Stop starting, start finishing***).




Quando uma coluna atinge seu limite de WIP estipulado, os desenvolvedores daquela etapa ficam terminantemente impedidos de puxar novas tarefas do estágio anterior. Diante desse bloqueio temporário, a equipe deve se mobilizar para atuar como força-tarefa no desimpedimento das etapas subsequentes que estão represadas, garantindo que o fluxo retome sua cadência natural.

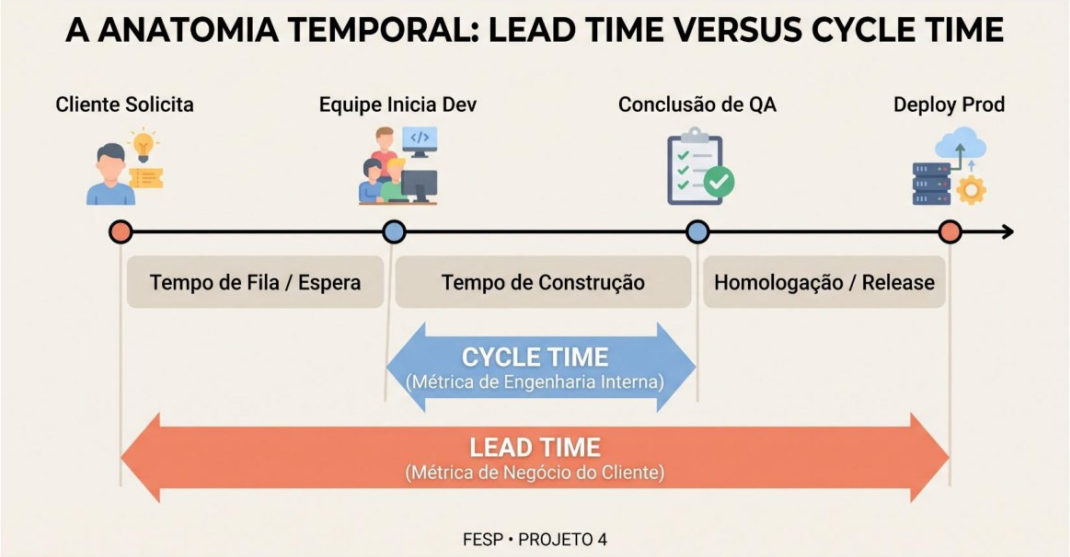



ANEXO C — Fonte original, página 10 de 18. Reprodução integral FESP, 2026.

**Métricas de Fluxo e Engenharia Estatística**

A eficiência do fluxo de trabalho é aferida através de métricas de dispersão temporal que substituem estimativas subjetivas por análise de dados históricos:

* **Lead Time**: O intervalo temporal total transcorrido entre a solicitação inicial da demanda pelo cliente (sua inserção formal no sistema) e a sua disponibilização efetiva em ambiente de produção. Representa a métrica sentida pelo mercado e pelos patrocinadores do negócio.
* **Cycle Time**: O tempo no qual a equipe técnica atua ativamente na execução da demanda, medido desde o instante em que o item entra na coluna de desenvolvimento ativo até a sua aprovação final na validação de qualidade (QA) . Representa a métrica direta de eficiência da engenharia.
* **Throughput (Vazão):** O volume numérico de itens de trabalho concluídos e entregues em estrita conformidade com a DoD por unidade temporal estabelecida (ex: histórias entregues por semana ou por ciclo).

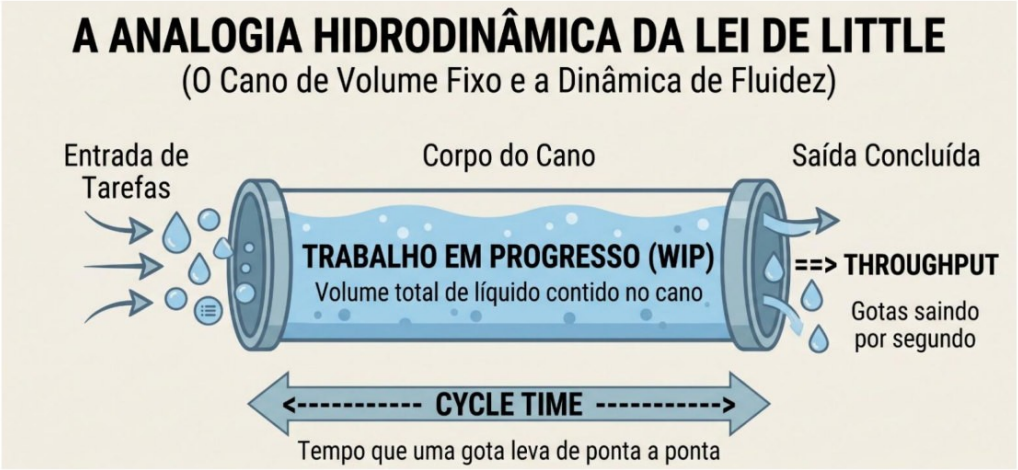



**A Lei de Little e a Modelagem Matemática do Fluxo**

A base matemática que governa a mecânica do fluxo em sistemas enxutos deriva da Lei de Little , formulada originalmente no âmbito da teoria matemática das filas por John Little em 1961 . Aplicada à engenharia de software e ao método Kanban, sua formulação expressa a relação entre três variáveis operacionais fundamentais:

Tempo Médio de Ciclo (Cycle Time) = Trabalho Médio em Progresso (WIP) Vazão Média (Throughput)

A representação física dessa igualdade matemática apoia-se na analogia do cano de água :




*** WIP (Trabalho em Progresso):** Representa o volume total de água retido simultaneamente no interior da tubulação.
*** Throughput (Vazão):** Representa a quantidade de gotas de água que conseguem verter da extremidade do cano por unidade de tempo, dimensionada pela secção de saída da tubulação.
*** Cycle Time:** Representa o tempo que uma gota singular necessita para percorrer toda a extensão longitudinal do cano.

Essa expressão demonstra que a tentativa de acelerar entregas inserindo mais tarefas simultâneas na equipe sem aumentar a capacidade operacional eleva o trabalho em progresso e inflaciona diretamente o tempo de ciclo, gerando lentidão e desperdício por troca de contexto.

Matematicamente, se uma equipe possui um Throughput estabilizado de 4{tarefas/semana} e mantém 12 {tarefas} abertas concomitantemente no quadro, seu Cycle Time médio será de:

**{Cycle Time} = {12{tarefas}}{4{tarefas/semana}} = 3{semanas}**

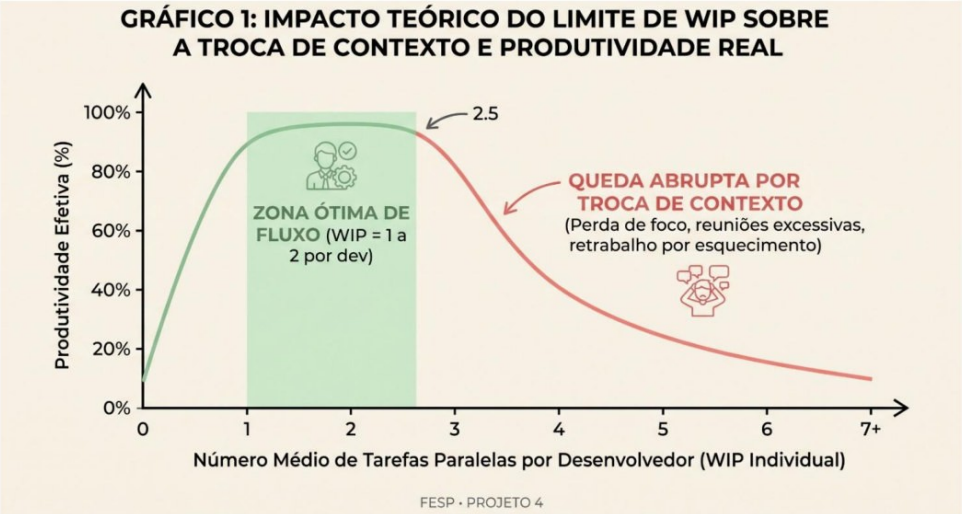
Se essa mesma equipe, sob pressão gerencial indevida, injetar mais trabalho e elevar o WIP para 20 { tarefas} sem alterar sua capacidade estrutural, o resultado não será mais entregas, mas sim a dilatação imediata do tempo de conclusão:

**{Cycle Time} = {20{tarefas}}{4{tarefas/semana}} = 5{semanas}**

A elevação do WIP produz fragmentação cognitiva, custos de alternância de tarefas **(context switching penalty)** e expansão descontrolada de débitos técnicos residuais. O caminho analítico correto para reduzir o tempo de resposta consiste em reduzir o limite de WIP, forçando o fluxo a escoar as tarefas represadas.

ANEXO C — Fonte original, página 12 de 18. Reprodução integral FESP, 2026.

**Gráfico 1: Impacto Teórico do Limite de WIP sobre a Troca de Contexto e Produtividade Real**




**Matriz de Batalha Estrutural: Scrum versus Kanban**

A seleção do modelo de governança para o Projeto 4 exige compreender com clareza as distinções operacionais e os contextos ideais de aplicação de cada abordagem:

**MATRIZ DE BATALHA ESTRUTURAL: SCRUM VERSUS KANBAN**

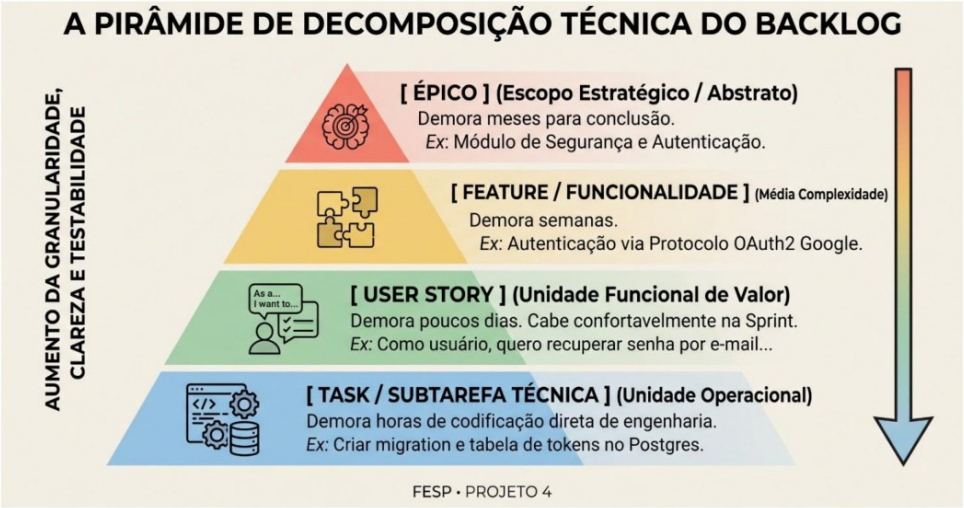
**Tabela 4:** Matriz Comparativa Scrum versus Kanban para a seleção do modelo de governança

| Critério de Análise | Framework Scrum | Método Kanban | Decisão Recomendada para o Projeto 4 |
|---|---|---|---|
| • Cadência / Iteração | Baseia-se em timeboxes regulares fixos de 1 a 4 semanas. | Opera em fluxo contínuo; liberações sob demanda ou por funcionalidade pronta. | **Abordagem Híbrida (Scrumban):** Timebox formal de Sprint para alinhamento pedagógico, associado a limites de WIP nas colunas do quadro. |
| • Papéis Definidos | Papéis estritos e mandatórios: Product Owner, Scrum Master e Desenvolvedores. | Sem papéis pré-determinados; atua sobre as funções organizacionais já existentes. | Manter papéis explícitos do Scrum para evitar diluição da responsabilidade entre os estudantes. |
| • Gestão de Mudança | Escopo congelado durante o ciclo; mudanças são redirecionadas para o próximo planejamento. | Novas tarefas podem ingressar a qualquer instante, desde que respeitados os limites de WIP. | Proteger o escopo da Sprint contra mudanças estruturais; aceitar refinamentos táticos menores. |
| • Métrica Chave | Velocidade de entrega (Velocity): Story Points entregues por Sprint. | Lead Time, Cycle Time, Throughput e diagrama de fluxo cumulativo (CFD). | Medir tanto o Cycle Time individual das tarefas quanto os Story Points consolidados por ciclo. |
| • Reset do Quadro | O Sprint Backlog zera ao término do ciclo; itens não concluídos retornam ao Product Backlog. | O quadro é persistente e perpétuo; itens continuam fluindo sem corte artificial. | Manter persistência física das colunas para permitir rastreamento contínuo da disciplina. |

FESP • PROJETO 4

### 3.4 Engenharia de Backlog, Histórias de Usuário e Dimensionamento

O Product Backlog organiza as entregas do sistema de forma contínua e iterativa. Não deve ser encarado como um depósito estático de ideias arquivadas, mas como um organismo vivo de engenharia que passa por decomposição contínua de granularidade à medida que os itens se aproximam da implementação física.




**Anatomia da User Story (História de Usuário)**

Para viabilizar a granularidade técnica e inverter o foco técnico isolado para o benefício real percebido pelo usuário, adota-se o modelo conceitual das User Stories , estruturadas sob a ótica da pessoa usuária:

{**Como**} [{Perfil / Persona}],
{**eu quero**} [{Ação / Capacidade Computacional}],
{**para que**} [{Benefício de Negócio / Valor Mensurável}].

Essa síntese garante que a implementação responda simultaneamente a três pilares: a identidade do ator que demanda o serviço, o comportamento funcional que o sistema precisa executar e a justificativa econômica ou operacional que valida a construção daquele ativo de software.

**O Critério INVEST de Qualidade em Histórias de Usuário**

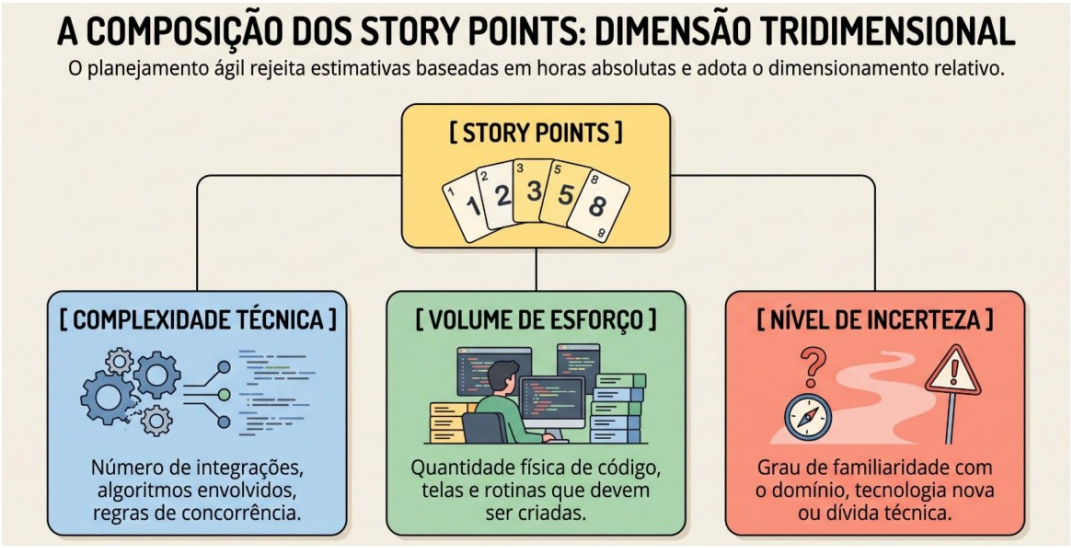
Proposto por Bill Wake, estabelece que uma história de usuário tecnicamente madura deve ser:

**Tabela 5: Guia de Auditoria e Refatoração de Histórias com INVEST**

| Dimensão INVEST | Sintoma de Violação no Projeto Acadêmico | Risco de Engenharia | Ação de Refatoração Recomendada |
|---|---|---|---|
| I - Independent | A História B só pode ser iniciada se a tela da História A estiver concluída no frontend. | Bloqueio em cascata e paralisia do fluxo de entrega. | Introduzir interfaces mockadas ou contratos de API para desacoplar as frentes de trabalho. |
| N - Negotiable | Especificação de 10 páginas definindo antecipadamente até as classes CSS internas. | Perda de autonomia técnica dos engenheiros e rigidez nociva. | Documentar a intenção funcional e debater a solução técnica durante a sessão de planejamento. |
| V - Valuable | História cadastrada como: "Configurar banco de dados relacional". | Trabalho técnico executado sem visibilidade de valor para o PO. | Reformular: "Persistir perfis cadastrais para permitir consulta e login no sistema acadêmico". |
| E - Estimable | A equipe declara que não sabe se a história consome dois dias ou seis meses de esforço. | Alta incerteza arquitetural e inviabilidade de compromisso. | Criar uma tarefa técnica de investigação (Spike) com tempo limitado para estudar o problema. |
| S - Small | Uma história recebe pontuação máxima e precisa de duas Sprints completas para rodar. | Estouro crônico de timebox e quebra do princípio de entrega frequente. | Fatiar a história em fluxos menores (ex: caminho feliz vs fluxos alternativos/exceções). |
| T - Testable | Critério de aceite vago: "O módulo deve responder com boa performance visual". | Impossibilidade de validação objetiva e desacordo no aceite. | Especificar critérios em Gherkin: "O tempo de renderização no navegador deve ser inferior a 1,5s" |

**Dimensionamento Relativo e a Técnica do Planning Poker**

O planejamento ágil rejeita estimativas baseadas em horas absolutas devido aos altos níveis de viés cognitivo e incerteza intrínsecos ao desenvolvimento de software intelectual.
Em seu lugar, adota o dimensionamento relativo mensurado em Story Points, que expressam uma combinação tridimensional indivisível de fatores técnicos:




1. **Apresentação do Item:** O Product Owner apresenta a história e esclarece dúvidas de negócio.
2. **Avaliação Individual e Privada**: Cada desenvolvedor seleciona privadamente uma carta de pontuação com base em seu julgamento técnico.
3. **Revelação Simultânea**: Todas as cartas são reveladas de modo simultâneo para mitigar o viés de ancoragem social (situação em que engenheiros sêniores influenciam indevidamente as estimativas dos novatos).
4. **Debate e Convergência**: As pontuações divergentes nos extremos explicam suas razões técnicas para o restante da equipe. Uma nova rodada de votação é conduzida até que ocorra convergência em torno de uma estimativa consensual.

ANEXO C — Fonte original, página 16 de 18. Reprodução integral FESP, 2026.

```
flowchart LR
  E1[Épicos de Negócio] --> US[Histórias INVEST]
  US --> PP[Planning Poker: Story Points]
  PP --> SP[Sprint Planning]
  SP --> SB[Sprint Backlog + Sprint Goal]

  subgraph Execution [Execução Puxada via Kanban]
    SB --> KB1[To Do]
    KB1 --> KB2[In Dev - WIP: 3]
    KB2 --> KB3[QA / Review - WIP: 2]
  end

  KB3 --> DoD{Atende 100% da DoD?}
  DoD -- Sim --> Inc[Incremento de Software Executável]
  DoD -- Não --> KB2
```

**Síntese: O Ecossistema Ágil em Ação para o Projeto Integrador**

A operacionalização bem-sucedida das práticas desta aula no Projeto 4 exige que as ferramentas e cerimônias atuem de forma sincronizada, conectando a governança estratégica à rotina operacional do repositório de código:

**Referências Bibliográficas Fundamentais**

* ANDERSON, David J. **Kanban: Mudança Tecnológica Bem-sucedida para Seu Negócio de Tecnologia**. Sequim: Blue Hole Press, 2010 .

* BECK, Kent et al. **Manifesto para o Desenvolvimento Ágil de Software**. Snowbird: Agile Alliance, 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html

* FOWLER, Martin. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3. ed. Boston: Addison-Wesley Professional, 2003 .

* INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models**. Genebra: ISO/IEC, 2011.

* LITTLE, John D. C. **A Proof for the Queuing Formula: L = lambdaW** . Operations Research, v. 9, n. 3, 1961.

* PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH / McGraw-Hill, 2021.

* SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum: As Regras do Jogo. Scrum.org, 2020**. Disponível em: https://scrumguides.org](https://scrumguides.org.

* SNOWDEN, David J.; BOONE, Mary E. **A Leader's Framework for Decision Making**. Harvard Business Review, v. 85, n. 11, p. 68-76, 2007.

* SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

* WAKE, Bill. **INVEST in Good Stories, and SMART Tasks**. XP123, 2003. Disponível em: https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/

## ANEXO D — Aula 4 — Desenvolvimento Colaborativo, Controle de Versão e Prototipação

Reprodução integral do texto institucional sobre colaboração, Git/GitHub, Gitflow, Trunk-Based, revisão por pares e arquitetura de camadas. A aplicação ao ConectaIA está na Etapa 4.

Arquivo de origem: Texto da Aula 4 - Eng Software.pdf. Total de páginas originais: 4. Reprodução sem cortes ou alterações de conteúdo.

## Engenharia de Software e Desenvolvimento Ágil de Soluções

**Cursos:** 2º Semestre - Bacharelado em Sistemas de Informação (BSI), Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), Engenharia de Software e Tecnologia em Inteligência Artificial

# Aula 4: Desenvolvimento Colaborativo, Controle de Versão e Prototipação

### 4.1 Cooperação Técnica em Equipes Multidisciplinares

Em ambientes de computação contemporâneos, a produção de software é uma atividade essencialmente coletiva e distribuída. Projetos integrados que congregam estudantes e profissionais das áreas de BSI, TADS, Engenharia de Software e IA dependem de clareza de papéis, governança na comunicação e padronização técnica rigorosa.

A comunicação em equipes de alta performance deve privilegiar o modo **assíncrono documentado** para decisões estruturais (via registros arquiteturais, revisões de código e issues rastreáveis), reservando a comunicação síncrona para desimpedimento rápido e deliberações de design que exijam validação imediata. Reuniões excessivas e desordenadas introduzem fragmentação cognitiva contínua, penalizando a produtividade dos desenvolvedores.

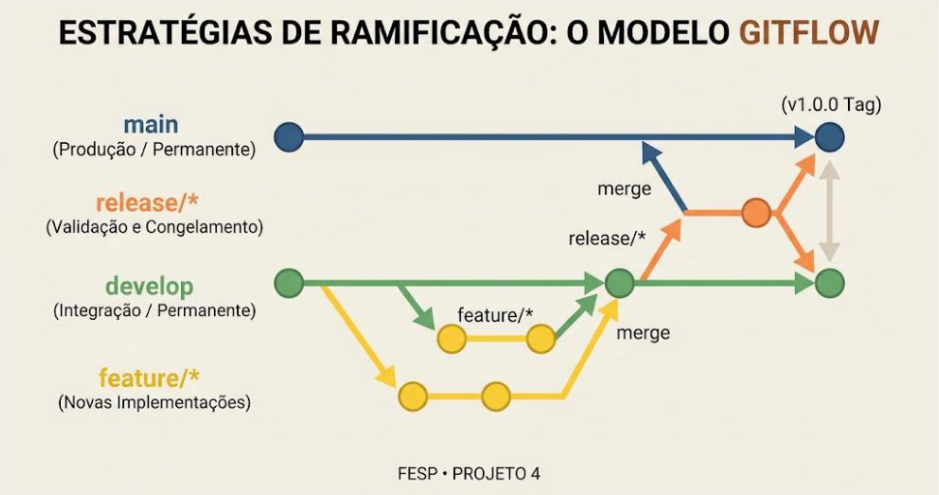
### 4.2 Controle de Versão Distribuído com Git e GitHub

O Git opera internamente como um sistema de arquivos versionado orientado a um Grafo Acíclico Dirigido (Directed Acyclic Graph - DAG), gerenciando nós imutáveis calculados por hashes criptográficos SHA. Diferente de ferramentas legadas centralizadas como o SVN, cada nó no Git possui uma cópia integral do repositório, garantindo autonomia operacional e segurança contra perda de dados.

### Estratégias de Ramificação (Branching Strategies): Gitflow e Trunk-Based

A escolha da estratégia de ramificação determina a velocidade de entrega e a estabilidade da base de código:

- **Gitflow:** modelo estruturado composto por branches permanentes (main para produção e develop para integração) e branches temporárias (feature/* para novas implementações, release/* para validação e congelamento de versão e hotfix/* para reparos urgentes em produção). Indicado para produtos com ciclos de liberação tradicionais.
- **Trunk-Based Development:** modelo lean no qual todos os desenvolvedores mesclam suas alterações diariamente diretamente em uma ramificação central (main/trunk), empregando *feature flags* (alternadores de funcionalidade em tempo de execução) para isolar códigos incompletos. Trata-se da fundação essencial para a operação de pipelines modernos de Integração e Entrega Contínuas (CI/CD).



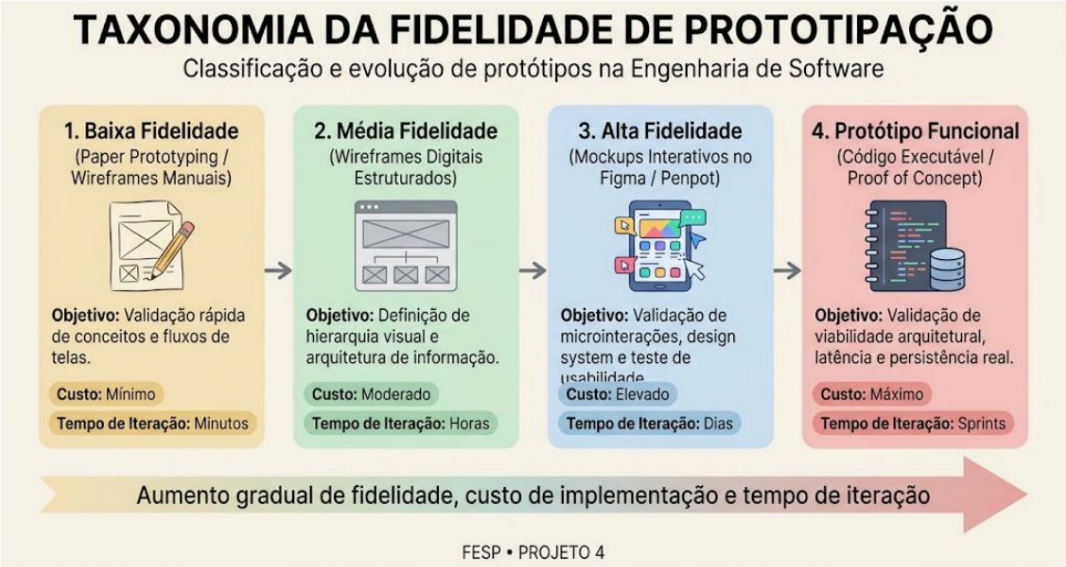

### Anatomia da Revisão por Pares (Pull Request / Code Review)

O Pull Request (PR) atua como barreira de contenção de defeitos e mecanismo pedagógico de disseminação de conhecimento técnico. A submissão de um PR deve observar os padrões de **Commits Semânticos** (*Conventional Commits*):

- **feat**: implementar autenticação JWT via Refresh Tokens
- **fix**: corrigir divergência de cálculo de juros compostos em atraso
- **refactor**: modularizar serviço de inferência do modelo Random Forest
- **test**: adicionar casos de teste de limite na rotina de parcelamento

## 4.3 Prototipação Incremental e Produto Mínimo Viável (MVP)

A construção de protótipos na Engenharia de Software atende a dois objetivos epistemológicos centrais: mitigar incertezas técnicas relativas à arquitetura e mitigar incertezas de mercado relativas à usabilidade do produto.




O **Produto Mínimo Viável (MVP)**, popularizado na metodologia Lean Startup e incorporado à Engenharia de Software contemporânea, não representa um produto incompleto ou defeituoso, mas sim a menor versão viável de um software que viabiliza o ciclo de aprendizado validado junto a usuários reais com o menor esforço e custo de desenvolvimento.

## 4.4 Desenvolvimento do Protótipo Funcional e Arquitetura de Camadas

A transição do protótipo interativo de telas para o protótipo executável funcional exige a definição de uma arquitetura limpa de software. Para o escopo dos projetos acadêmicos e integradores, preconiza-se a separação estrita de responsabilidades em camadas lógicas desacopladas:

# Referências Bibliográficas Fundamentais

**Referências Bibliográficas Fundamentais**

* ANDERSON, David J. **Kanban: Mudança Tecnológica Bem-sucedida para Seu Negócio de Tecnologia**. Sequim: Blue Hole Press, 2010 .

* BECK, Kent et al. **Manifesto para o Desenvolvimento Ágil de Software**. Snowbird: Agile Alliance, 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html

* FOWLER, Martin. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3. ed. Boston: Addison-Wesley Professional, 2003 .

* INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models**. Genebra: ISO/IEC, 2011.

* PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH / McGraw-Hill, 2021.

* SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum: As Regras do Jogo. Scrum.org, 2020**. Disponível em: https://scrumguides.org](https://scrumguides.org.

* SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

**Fontes**

1. https://dev.to/albersonfbarbosa/parte-01-agile-test-53go

# ANEXO E — Aula 5 — Validação, Garantia da Qualidade e Apresentação Técnica

Reprodução integral do texto institucional sobre testes, validação SUS, melhoria contínua, defesa técnica e a rubrica oficial da disciplina. A aplicação ao ConectaIA está na Etapa 5.

Arquivo de origem: Texto da Aula 5 - Eng Software.pdf. Total de páginas originais: 4. Reprodução sem cortes ou alterações de conteúdo.

# Engenharia de Software e Desenvolvimento Ágil de Soluções

**Cursos:** 2º Semestre - Bacharelado em Sistemas de Informação (BSI), Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), Engenharia de Software e Tecnologia em Inteligência Artificial

# Aula 5: Validação, Garantia da Qualidade e Apresentação Técnica

## 5.1 Fundamentos de Testes de Software e Garantia da Qualidade

A testagem de software constitui uma atividade analítica e experimental orientada a revelar discrepâncias entre o comportamento esperado de um sistema e seu comportamento real em execução. Martin Fowler esquematiza a distribuição ideal de esforços de garantia de qualidade na **Pirâmide de Testes**:

### Técnicas de Projeto de Testes: Caixa-Preta versus Caixa-Branca

- **Testes de Caixa-Preta (Testes Funcionais):** a estrutura interna e a implementação lógica do código-fonte permanecem ocultas ao analista. As entradas são injetadas e as saídas inspecionadas com base na especificação. Empregam-se técnicas como o **Particionamento por Equivalência** (divisão do domínio de entrada em classes de dados válidos e inválidos) e a **Análise de Valores Limite** (testagem rigorosa das bordas numéricas e condições limítrofes dos intervalos).
- **Testes de Caixa-Branca (Testes Estruturais):** inspecionam diretamente as estruturas internas do código executável. Analisam a complexidade ciclomática de McCabe e mensuram a cobertura de branches, caminhos e linhas de instrução (*code coverage*).

### Ciclo de Vida do Registro de Defeitos (Bugs)

Quando um teste falha ou uma anomalia em produção é detectada, o apontamento técnico deve conter rastreabilidade rigorosa. A matriz de avaliação separa estritamente a **Severidade** (o impacto operacional da falha na estabilidade do sistema) da **Prioridade** (a urgência de reparo exigida pelas metas de negócio).

## 5.2 Validação da Solução com Usuários e Avaliação de Usabilidade

A verificação formal de código e o cumprimento dos critérios técnicos da **Definição de Pronto (Definition of Done - DoD)** certificam que o sistema foi construído sem erros estruturais, mas não comprovam que o software resolve o problema do usuário final de modo eficiente.
Para validar o produto em sua dimensão sociotécnica, conduzem-se testes formais de usabilidade empregando a **Escala de Usabilidade de Sistema (SUS - System Usability Scale)**, padrão composto por dez assertivas avaliadas em escala Likert de 1 (discordo totalmente) a 5 (concordo totalmente):

1. Eu gostaria de utilizar este sistema frequentemente.
2. Achei o sistema desnecessariamente complexo.
3. Achei o sistema fácil de utilizar.
4. Acho que precisaria do auxílio de um suporte técnico para usar este sistema.
5. As funções deste sistema estão muito bem integradas.
6. Achei que houve muita inconsistência neste sistema.
7. Imagino que a maioria das pessoas aprenderia a usar este sistema rapidamente.
8. Achei o sistema excessivamente complicado de usar.
9. Me senti muito confiante ao usar este sistema.
10. Precisei aprender uma série de coisas novas antes de conseguir usar o sistema.

A pontuação total calculada varia de 0 a 100 pontos; resultados acima de 68 pontos situam a aplicação acima da média aceitável de facilidade de uso, enquanto médias inferiores a 50 pontos sinalizam problemas estruturais na arquitetura de navegação e design de interação.

## 5.3 Retrospectiva do Projeto e Melhoria Contínua (Kaizen)

A retrospectiva consiste no mecanismo de aprendizagem organizacional do ciclo ágil, onde o time inspeciona a si próprio e estabelece compromissos de melhoria prática para os períodos seguintes. Ela difere da Sprint Review porque seu foco de análise não reside nas funcionalidades entregues, mas nas dinâmicas operacionais, relações entre pessoas e gargalos de ferramentas.

A dinâmica **Starfish (Estrela do Mar)** permite categorizar o comportamento operacional da equipe técnica em cinco quadrantes complementares:

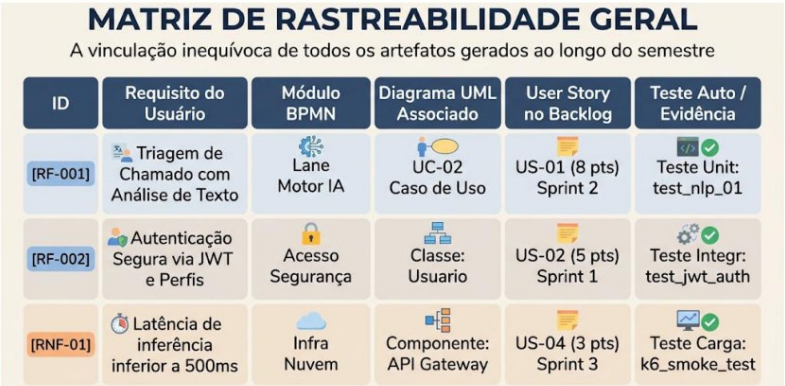
1. **Começar a Fazer (Start):** práticas e técnicas de engenharia ausentes que devem ser incorporadas imediatamente (ex: introduzir testes automatizados de integração nas rotinas de commit).
2. **Parar de Fazer (Stop):** condutas disfuncionais ou desperdícios identificados que devem ser eliminados (ex: mesclar branches na branch principal sem revisão por pares).
3. **Fazer Mais (More):** atividades benéficas que foram realizadas em volume insuficiente e devem ser ampliadas (ex: refinamento mais detalhado dos critérios de aceitação com o Product Owner).
4. **Fazer Menos (Less):** práticas aceitáveis, mas cujo excesso gera atritos e lentidão (ex: discussões teóricas excessivamente longas na Daily Scrum).
5. **Continuar Fazendo (Keep):** acordos de trabalho e padrões técnicos consolidados que funcionaram com excelência e devem ser preservados como ativos do time.

## 5.4 Apresentação Técnica, Defesa do Projeto e Entrega Final

A defesa de uma solução de software diante de uma banca examinadora acadêmica ou de investidores industriais exige uma articulação entre o problema do negócio, a conformidade de processos e a excelência arquitetural. O tempo de apresentação deve ser distribuído equilibrado:

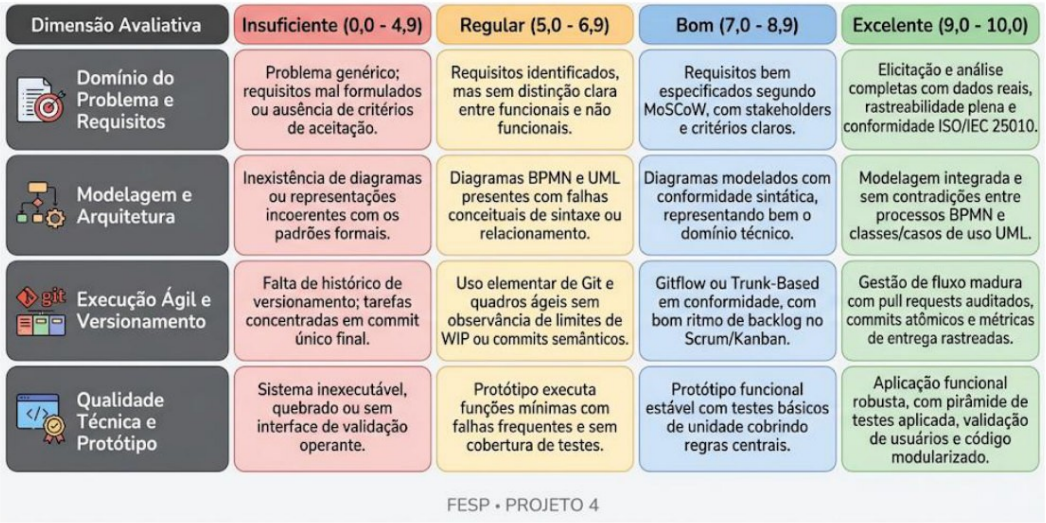
## Matriz de Rastreabilidade e Auditoria de Entrega

A entrega final da disciplina demanda a vinculação inequívoca de todos os artefatos gerados ao longo do semestre:



## Rubrica de Avaliação Integrada da Disciplina

Para alinhar o processo pedagógico às diretrizes curriculares nacionais e aos critérios institucionais de avaliação, os projetos finais dos cursos de BSI, TADS, Engenharia de Software e IA da FESP são pontuados conforme as quatro faixas de desempenho descritas a seguir:



| Dimensão Avaliativa | Insuficiente (0,0 - 4,9) | Regular (5,0 - 6,9) | Bom (7,0 - 8,9) | Excelente (9,0 - 10,0) |
|---|---|---|---|---|
| Domínio do Problema e Requisitos | Problema genérico; requisitos mal formulados ou ausência de critérios de aceitação. | Requisitos identificados, mas sem distinção clara entre funcionais e não funcionais. | Requisitos bem especificados segundo MoSCoW, com stakeholders e critérios claros. | Elicitação e análise completas com dados reais, rastreabilidade plena e conformidade ISO/IEC 25010. |
| Modelagem e Arquitetura | Inexistência de diagramas ou representações incoerentes com os padrões formais. | Diagramas BPMN e UML presentes com falhas conceituais de sintaxe ou relacionamento. | Diagramas modelados com conformidade sintática, representando bem o domínio técnico. | Modelagem integrada e sem contradições entre processos BPMN e classes/casos de uso UML. |
| Execução Ágil e Versionamento | Falta de histórico de versionamento; tarefas concentradas em commit único final. | Uso elementar de Git e quadros ágeis sem observância de limites de WIP ou commits semânticos. | Gitflow ou Trunk-Based em conformidade, com bom ritmo de backlog no Scrum/Kanban. | Gestão de fluxo madura com pull requests auditados, commits atômicos e métricas de entrega rastreadas. |
| Qualidade Técnica e Protótipo | Sistema inexecutável, quebrado ou sem interface de validação operante. | Protótipo executa funções mínimas com falhas frequentes e sem cobertura de testes. | Protótipo funcional estável com testes básicos de unidade cobrindo regras centrais. | Aplicação funcional robusta, com pirâmide de testes aplicada, validação de usuários e código modularizado. |

FESP • PROJETO 4

## Referências Bibliográficas


- ANDERSON, David J. **Kanban: Mudança Tecnológica Bem-sucedida para Seu Negócio de Tecnologia**. Sequim: Blue Hole Press, 2010.
- BECK, Kent et al. **Manifesto para o Desenvolvimento Ágil de Software**. Snowbird: Agile Alliance, 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html.
- FOWLER, Martin. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3. ed. Boston: Addison-Wesley Professional, 2003.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models**. Genebra: ISO/IEC, 2011.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH / McGraw-Hill, 2021.
- SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum: As Regras do Jogo**. Scrum.org, 2020. Disponível em: https://scrumguides.org.
- SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.


**Fontes**

1. https://dev.to/albersonfbarbosa/parte-01-agile-test-53go

## ADENDO TÉCNICO — REVISÃO DO PROTÓTIPO E EVIDÊNCIAS DE TESTE

Versão revisada em 06/11/2026. Este adendo complementa o relatório-base sem apagar o histórico das limitações identificadas na auditoria anterior. As conclusões abaixo se restringem aos arquivos recebidos e aos testes executados nesta revisão.

### 1. Escopo efetivamente revisado

Foi inspecionado o pacote CONECTA_IA ON (1).zip. A aplicação existente utiliza React, TypeScript e Vite, com interface em src/App.tsx, estilos em src/styles.css e serviço de conversa em src/services/gemini.ts. Foram mantidos os componentes e fluxos já presentes, sem reconstrução integral da interface.

A integração foi reorganizada: o navegador chama /api/chat; um servidor Node local em server/index.mjs faz a mediação com a API Gemini. A credencial é lida exclusivamente pelo servidor a partir de GEMINI_API_KEY. O frontend não contém VITE_GEMINI_API_KEY. Se não houver chave ou a chamada falhar, a aplicação recorre às respostas locais já existentes. Esse modo de contingência não deve ser apresentado como resposta gerada por IA.

### 2. Segurança e limites da arquitetura

O servidor valida o corpo da requisição, limita a pergunta a 2.000 caracteres, restringe o histórico aos seis últimos itens, limita o tamanho do corpo HTTP, aplica limite simples de requisições por endereço IP e timeout da chamada externa. A escuta está limitada a 127.0.0.1 para demonstração local. Esses controles reduzem exposição acidental no protótipo, mas não constituem arquitetura pronta para publicação pública. Uma implantação pública exigiria autenticação, proteção antiabuso robusta, gestão de segredos, observabilidade, revisão de privacidade e configuração de infraestrutura.

Se uma chave real foi incluída em versões anteriores do código ou enviada a repositórios, ela deve ser revogada no painel do provedor e substituída. A análise do ZIP não permite confirmar se a credencial histórica já foi revogada.

### 3. Testes executados nesta revisão

| ID | Teste | Resultado | Status |
|---|---|---|---|

| | | | |
|---|---|---|---|
| | | observado | |
| UT-01 | Rejeição de payload inválido/pergunta vazia | Retornou null para entradas inválidas | APROVADO |
| UT-02 | Limites de pergunta, nome e histórico | Limites e truncamento confirmados | APROVADO |
| UT-03 | Montagem do payload Gemini | Papéis user/model e instrução de sistema confirmados | APROVADO |
| UT-04 | Extração de resposta Gemini | Texto concatenado; ausência de texto tratada | APROVADO |
| INT-01 | Endpoint local /api/health sem chave | HTTP 200; aiConfigured=false | APROVADO |

Os sete testes automatizados foram executados com node --test server/*.test.mjs: 7 aprovados, 0 reprovados (quatro testes unitários e três testes HTTP do servidor). O conjunto cobre validação de payload, limites de entrada, transformação do histórico, extração de resposta, endpoint de saúde, rejeição de pergunta vazia e indisponibilidade controlada sem chave. Esse teste não chama o Gemini real e não comprova a integração externa.

### 4. Testes não executados e evidências ainda necessárias

A instalação limpa das dependências npm expirou por timeout do registry nesta estação. Por esse motivo, npm run build não pôde ser concluído e não foram executados testes de navegador/E2E. A integração real com Gemini não foi testada porque nenhuma chave válida foi fornecida. Não foram realizados novos testes com

participantes, auditoria independente WCAG 2.2, nem validação de leitor de tela. O histórico Git/PRs e a ata de retrospectiva também não constam do ZIP.

Os resultados de usabilidade anteriores (18/18 tarefas, SUS 100/100 por participante, 90 segundos e zero erros) permanecem relatos atribuídos à equipe, pois os formulários SUS originais e fichas individuais não foram apresentados. A planilha de usuários contém marcações de checklist como “VERIFICADO”, mas não há anexos de medição ou relatórios independentes suficientes para converter essas marcações em certificação WCAG.

**5. Procedimento para validação final no ambiente da equipe**

1. Na pasta 06_APP/ConectaIA_Proto1, executar npm.cmd ci.
2. Copiar .env.example para .env e inserir GEMINI_API_KEY localmente. Nunca compartilhar o arquivo .env.
3. Em um terminal, executar npm.cmd run api; em outro, executar npm.cmd run dev.
4. Executar npm.cmd test e npm.cmd run build; guardar os logs completos.
5. Com a aplicação aberta, executar CT01–CT09 e retestar CT10, registrando ambiente, passos, resultado esperado, resultado observado e evidência.
6. Testar a integração Gemini com pergunta não sensível e registrar resposta/erro sem expor a chave.
7. Realizar avaliação de teclado, foco, contraste, ampliação, leitor de tela e telas menores; anexar evidências.
8. Realizar ou localizar os registros originais de testes com usuários, SUS e retrospectiva; não reconstruir dados ausentes.
9. Exportar histórico real de commits, pull requests e revisões para comprovar desenvolvimento colaborativo.

**6. Conclusão técnica**

A versão revisada melhora a segurança da integração e acrescenta testes unitários automatizados reproduzíveis. A entrega permanece um protótipo acadêmico, não um produto de produção homologado. Os testes aprovados neste adendo são somente os explicitamente executados. Build frontend, Gemini externo, E2E, acessibilidade formal, validação com usuários, histórico Git e retrospectiva continuam pendentes de comprovação. A matriz detalhada está em 04_TESTES_E_VALIDACAO/Registro_Execucao_Tecnica.xlsx.